



Java Learning Center

No.1 In Java Training & placement

DSA

Module 1

Introduction to DSA

Author

Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

1. Data types

- ♦ Data type represents:
 - Type of data you want to use
 - Amount of memory allocation required for your data.

There are two types of data types

- 1) Primitive Data Types
- 2) User Defined Data Types

Primitive Data Types

- ♦ There are 8 primitive types available in Java.
- ♦ 8 keywords are defined to represent 8 primitive data types.
- ♦ Following are primitive data types:

boolean	byte	char	short
int	long	float	double

User-Defined Data Types

- ♦ There are four types of User Defined Data types:
 - Class type
 - Interface type
 - Enum type (From JAVA 5)
 - Annotation type (From JAVA 5)

2. Variables

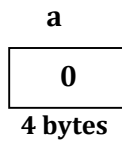
- ♦ Variable is the container which holds user data.
- ♦ Memory will be allocated for the variable while executing the program.
- ♦ Value of the variable can be changed any number of times during the program execution.

Syntax:

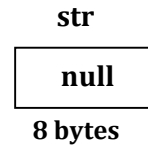
```
<Data type> <varName>;  
<Data type> <varName> = <value>;
```

Ex:

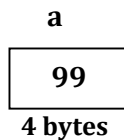
`int a ;`



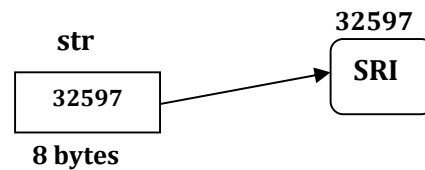
`String str;`



`int a = 99;`



`String str="SRI";`



Types of Variables

There are two types of variables based on data type used to declare the variable.

- 1) Primitive Variables
- 2) Reference Variables

2.1 Primitive Variables

- ♦ Variables declared with primitive data types are called as primitive variables.

Ex:

`int a;`
`int b = 99;`

2.2 Reference Variables

- ♦ Variables declared with user defined data types are called as reference variables.

Ex:

`String str1;`
`String str2 = "JLC";`

3. Data Structures

- ♦ Data Structure is a special way for storing and organizing data so that it can be used efficiently.
- ♦ Following are Types of Data Structures
 - Arrays
 - Linked Lists
 - Stacks
 - Queues
 - Trees
 - Graphs
 - etc
- ♦ We can divide the Data Structures into two Types depending on the organization of the elements
 1. Linear Data Structures
 2. Non - Linear Data Structures

3.1 Linear Data Structures

- ♦ Elements are accessed in a sequential order but it is not compulsory to store all elements sequentially.

Ex:

Linked Lists

Stacks

Queues

3.2 Non - Linear Data Structures

- ♦ Elements of this data structure are stored/accessed in a non-linear order.

Ex:

Trees

Graphs

4. Abstract Data Types

- ♦ Define Data Type by Combining the data structures with their operations which is called as Abstract Data Type (ADT).
- ♦ An Abstract Data Type(ADT) is an abstraction of a Data Structure that provides only the interface to which the Data Structure must adhere.
- ♦ The interface does not give any specific details about how something should be implemented or in what programming language.
- ♦ ADT consists of two parts:
 1. Declaration of data
 2. Declaration of operations
- ♦ Following are Commonly used ADTs:
 - Linked Lists
 - Stacks
 - Queues
 - Priority Queues
 - Binary Trees
 - Dictionaries
 - Disjoint Sets
 - Hash Tables
 - Graphs
 - etc
- ♦ Different kinds of ADTs are suited to different kinds of applications, and some are highly specialized to specific tasks.
- ♦ By the end of this Course, we will go through many of them and you will be in a position to relate the data structures to the kind of problems they solve.

5. Exploring Algorithms

- ♦ An algorithm is the step-by-step unambiguous instructions to solve a given problem.
- ♦ There are two main criteria for judging the algorithms:
 - a) Correctness
 - b) Efficiency

a) Correctness

- ♦ Does the algorithm give solution to the problem

b) Efficiency

- ♦ How much resources (in terms of memory and time) does it take to execute

Examples:

Ex1: Algorithm for Adding Two Numbers

```
/*  
* @Author : Srinivas Dande  
* @Company: Java Learning Center  
* */
```

Step 1: Start

Step 2: Declare 3 Variables a , b and sum.

Step 3: Read values for a and b

Step 4: Add a and b and assign the result to a variable sum.

Step 5: Display sum

Step 6: Stop



Ex2: Algorithm for Adding Two Numbers

```
/*  
* @Author : Srinivas Dande  
* @Company: Java Learning Center  
* */
```

Step 1: Start

Step 2: Declare 3 Variables a , b and sum.

```
int a;  
int b;  
int sum;
```

Step 3: Read values for a and b

```
a = 10;  
b = 20;
```

Step 4: Add a and b and assign the result to a variable sum.

```
sum = a+b;
```

Step 5: Display sum

```
print(sum);
```

Step 6: Stop

Ex3: Algorithm for Printing Numbers from 1 to 10

```
/*  
* @Author : Srinivas Dande  
* @Company: Java Learning Center  
* */
```

Step 1: Start

Step 2: Declare and Initialize the Variable i to 1

```
int i=1;
```

Step 3: Display i

```
print(i);
```

Step 4: Increment i by 1

```
i = i + 1;
```

Step 5: Check if the value of i is less than or equal to 10.

```
IF i<=10 THEN
```

```
    GOTO Step 3
```

```
otherwise
```

```
    GOTO Step 6
```

Step 6: Stop.

6. Algorithm Analysis

- ♦ Algorithm analysis helps us to determine which algorithm is most efficient in terms of time and space consumed.
- ♦ Goal of the analysis of algorithms is to compare algorithms (or solutions).

How to Compare Algorithms:

1) Execution times?

- ♦ Not a Good Idea because execution time will be changed from Machine to Machine or Programming Language to Programming Language

2) Number of statements executed?

- ♦ Not a Good Idea because Number of statements will be changed from Programming Language to Programming Language

What is Best Solution:

Need the Way to Calculate Time Complexity and Space Complexity of an Algorithm without depending on Machine or Programming Language or Developer

7. Rate of Growth Classes

- ♦ Rate at which the running time of algorithm grows as the input size grows is called rate of growth.

Commonly Used Rates of Growth

Complexity	Name
1	Constant
log n	Logarithmic
n	Linear
n log n	Linear Logarithmic
n ²	Quadratic
n ³	Cubic
2 ⁿ	Exponential
3 ⁿ	Exponential
n!	Factorial

Relationship between Rates of Growth Classes:

$$1 < \log n < n < n * \log n < n^2 < n^3 < 2^n < 3^n < n!$$



Ex1: Find the Time Complexity

```
void sum() {  
    int a= 10;  
    int b=20;  
    int sum = a + b;  
    System.out.println(sum);  
}
```

$f(n) = O(1 + 1 + 1 + 1)$
 $= O(4)$
 $= O(1) \Rightarrow$ **Constant Time Complexity**

Ex2: Find the Time Complexity

```
int sum(int n) {  
    System.out.println("Begin");  
    int sum= 0;  
  
    for(int i= 1; i<=n; i++){  
        sum = sum + i;  
    }  
  
    System.out.println("End");  
    return sum;  
}
```

$f(n) = O(1 + 1 + n + n + 1 + 1)$
 $= O(2n + 4)$
 $= O(n) \Rightarrow$ **Linear Time Complexity**

8. Types of Analysis

- ♦ To analyze the given algorithm, We need to know
 - With which inputs the algorithm takes less time
 - With which inputs the algorithm takes a long time.
- ♦ We represent the algorithm with multiple expressions:
- ♦ One for the case where it takes less time and another for the case where it takes more time.
- ♦ There are three types of analysis:
 - a) Best Case
 - b) Average Case
 - c) Worst Case

a) Best Case Analysis

- ♦ Minimum amount of time the Algorithm takes to execute for the given Input.(Faster Execution time)

b) Average Case Analysis

- ♦ Provides a prediction about the running time of the algorithm.
- ♦ Run the algorithm many times, using many different inputs and compute the Average Running Time

c) Worst Case Analysis

- ♦ Maximum amount of time the Algorithm takes to execute for the given Input.(Slower Execution Time)



9. Asymptotic Notations

- ♦ Asymptotic Notations are the mathematical notations used to describe the Time Complexity or Space Complexity of an algorithm
- ♦ There are 3 such Notations:
 - a) Big-O Notation (Upper Bound Function)
 - b) Omega Notation (Lower Bound Function)
 - c) Theta Notation (Tight Bound Function)

10. Big-O Notation

- ♦ Big-O Notation gives tight upper bound of the given function
- ♦ It is Represented as $f(n) = O(g(n))$
i.e Upper bound of $f(n)$ is $g(n)$ at Larger Values of n .

Function $f(n) = O(g(n))$

if there exists +ve constants C and n_0

such that $f(n) \leq C * g(n)$ for all $n \geq n_0$

Ex1:

$$f(n) = 10n + 7$$

$$10n + 7 \leq 11n + 7n \quad \text{for all } n \geq 1$$

$$10n + 7 \leq 18n \quad \text{for all } n \geq 1$$

$$\text{Here } f(n) = 10n + 7$$

$$C = 18$$

$$g(n) = n$$

$$n_0 = 1$$

$$n=0 \quad \text{-----} \quad 7 \leq 0 \quad // \text{ Not Allowed}$$

$$n=1 \quad \text{-----} \quad 17 \leq 18 \quad // \text{ Allowed}$$

$$n=2 \quad \quad \quad 27 \leq 36 \quad // \text{ Allowed}$$

$$n=3 \quad \quad \quad 37 \leq 54 \quad // \text{ Allowed}$$

SO $n \geq 1$ is Allowed.

So $f(n) = O(n) \Rightarrow$ Linear Complexity

Ex2:

$$f(n) = 6n^2 + 2n + 32n$$

$$6n^2 + 2n + 32n \leq 6n^2 + 2n^2 + 32n^2 \text{ for all } n \geq 0$$

$$6n^2 + 2n + 32n \leq 40n^2 \text{ for all } n \geq 0$$

Here $f(n) = 6n^2 + 2n + 32n$

$$C = 40$$

$$g(n) = n^2$$

$$n_0 = 0$$

n=0	----- 0	≤ 0	// Allowed
n=1	----- 40	≤ 40	// Allowed
n=2	----- 101	≤ 160	// Allowed

SO $n \geq 0$ is Allowed.

So $f(n) = O(n^2) \Rightarrow$ Quadratic Complexity

Ex3:

$$f(n) = 3n^2 + 5n + 9$$

$$3n^2 + 5n + 9 \leq 4n^2 \text{ for all } n \geq 7$$

$$3n^2 + 5n + 9 \leq 4n^2 \text{ for all } n \geq 7$$

Here $f(n) = 3n^2 + 5n + 9$

$$C = 4$$

$$g(n) = n^2$$

$$n_0 = 7$$

n=0	----- 9	≤ 0	// Not Allowed
n=1	----- 17	≤ 4	// Not Allowed
n=2	----- 31	≤ 16	// Not Allowed
....			
n=6	----- 147	≤ 144	// Not Allowed
n=7	----- 191	≤ 196	// Allowed

SO $n \geq 7$ is Allowed.

So $f(n) = O(n^2) \Rightarrow$ Quadratic Complexity



11. Omega Notation

- ♦ Omega Notation gives tight lower bound of the given function
- ♦ It is Represented as $f(n) = \Omega (g(n))$
i.e Lower bound of $f(n)$ is $g(n)$ at Larger Values of n .

Function $f(n) = \Omega (g(n))$
if there exists +ve constants C and n_0
such that $f(n) \geq C * g(n)$ for all $n \geq n_0$

Ex1:

$$f(n) = 3n + 5$$
$$3n + 5 \geq 1n \quad \text{for all } n \geq 0$$
$$3n + 5 \geq 2n \quad \text{for all } n \geq 0$$

Here $f(n) = 3n + 5$

$C = 2$

$g(n) = n$

$n_0 = 0$

$n=0$	-----	$5 \geq 0$	// Allowed
$n=1$	-----	$8 \geq 2$	// Allowed
$n=2$	-----	$11 \geq 4$	// Allowed

SO $n \geq 0$ is Allowed.

So $f(n) = \Omega (n) \Rightarrow$ Linear Complexity

Ex2:

$$f(n) = 6n^2 + 2n + 32$$
$$6n^2 + 2n + 32 \geq 5n^2 \quad \text{for all } n \geq 0$$
$$6n^2 + 2n + 32 \geq 5n^2 \quad \text{for all } n \geq 0$$

Here $f(n) = 6n^2 + 2n + 32$

$C = 5$

$g(n) = n^2$

$n_0 = 0$

$n=0$	-----	$32 \geq 0$	// Allowed
$n=1$	-----	$40 \geq 5$	// Allowed

SO $n \geq 0$ is Allowed.

So $f(n) = \Omega (n^2) \Rightarrow$ Quadratic Complexity

Ex3:

$$f(n) = 3n^2 + 5n + 9$$

$$3n^2 + 5n + 9 \geq 2n^2 \quad \text{for all } n \geq 7$$

$$3n^2 + 5n + 9 \geq 2n^2 \quad \text{for all } n \geq 7$$

Here $f(n) = 3n^2 + 5n + 9$

$$C = 2$$

$$g(n) = n^2$$

$$n_0 = 7$$

$$n=0 \quad \text{-----} \quad 9 \geq 0 \quad // \text{ Allowed}$$

$$n=1 \quad \text{-----} \quad 17 \geq 2 \quad // \text{ Allowed}$$

$$n=2 \quad \text{-----} \quad 31 \geq 8 \quad // \text{ Allowed}$$

SO $n \geq 0$ is Allowed.

So $f(n) = \Omega(n^2) \Rightarrow$ Quadratic Complexity

12. Theta Notation

- ♦ Theta Notation gives tight bound of the given function
- ♦ It is Represented as $f(n) = \Theta(g(n))$
i.e Exact bound of $f(n)$ is $g(n)$ at Larger Values of n .
- ♦ Average Running time of an Algorithm is always between lower bound and upper bound.

$$\text{Function } f(n) = \Theta(g(n))$$

if there exists +ve constants c_1, c_2 and n_0

such that $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n \geq n_0$

Ex1:

$$f(n) = 5n + 7$$

$$4n \leq 5n + 7 \leq 6n \quad \text{for all } n \geq 7$$

Here $f(n) = 5n + 7$

$$C1 = 4$$

$$C2 = 6$$

$$g(n) = n$$

$$n0 = 7$$

$n=0$ ---- $0 \leq 7 \leq 0$ // Not Allowed
 $n=1$ ---- $4 \leq 12 \leq 6$ // Not Allowed
 $n=2$ ---- $8 \leq 17 \leq 12$ // Not Allowed
 $n=3$ ---- $12 \leq 22 \leq 18$ // Not Allowed
 ..
 $n=6$ ---- $24 \leq 37 \leq 36$ // Not Allowed
 $n=7$ ---- $28 \leq 42 \leq 42$ // Not Allowed

SO $n \geq 7$ is Allowed.

So $f(n) = \Theta(n)$ => Linear Complexity

Ex2:

$$f(n) = 3n^2 + 5n + 9$$

$$4n^2 \leq 3n^2 + 5n + 9 \leq 2n^2 \quad \text{for all } n \geq (\text{You Find})$$

Here $f(n) = 3n^2 + 5n + 9$

$$C1 = 4$$

$$C2 = 2$$

$$g(n) = n^2$$

$$n0 = (\text{You Find})$$

$n=0$ --- $0 \leq 9 \leq 0$ // Not Allowed
 $n=1$ ---- $4 \leq 17 \leq 2$ // Not Allowed
 $n=2$ ---- $16 \leq 31 \leq 8$ // Not Allowed
 ...

You Calculate n value

So $f(n) = \Theta(n^2)$ => Quadratic Complexity



13. More Examples

Example 1:

```
void sum() {  
    int a= 10;  
    int b=20;  
    int sum = a + b;  
    System.out.println(sum);  
}
```

$f(n) = O(1 + 1 + 1 + 1)$
 $= O(4)$
 $= O(1) \Rightarrow \text{Constant Time Complexity}$

Example 2:

```
int sum(int n) {  
    System.out.println("Begin");  
    int sum= 0;  
  
    for(int i= 1; i<=n; i++){  
        sum = sum + i;  
    }  
  
    System.out.println("End");  
    return sum;  
}
```

$f(n) = O(1 + 1 + n + n + 1 + 1)$
 $= O(2n + 4)$
 $= O(n) \Rightarrow \text{Linear Time Complexity}$



Example 3:

```
void show(int n) {  
    System.out.println("Begin");  
  
    for(int i= 1; i<=n; i++){  
        System.out.println(i);  
    }  
  
    System.out.println("OK");  
  
    for(int i= 1; i<=n; i++){  
        System.out.println(i);  
    }  
  
    System.out.println("End");  
}
```

$$\begin{aligned}f(n) &= O(1 + n + 1 + n + 1) \\&= O(2n + 3) \\&= O(n) \Rightarrow \text{Linear Time Complexity}\end{aligned}$$



Example 4:

```
void show(int n) {  
    System.out.println("Begin");  
  
    System.out.println("OK");  
  
    for(int i= 1; i<=n; i++){  
        for(int j= 1; j<=n; j++){  
            System.out.println(i+"\t"+j);  
        }  
    }  
  
    System.out.println("OK");  
  
    System.out.println("End");  
}
```

$$f(n) = O(1 + 1 + (n * n) + 1 + 1)$$

$$= O(n^2 + 4)$$

$$= O(n^2) \Rightarrow \text{Quadratic Time Complexity}$$



Example 5:

```
void show(int n) {  
    System.out.println("Begin");  
  
    System.out.println("OK");  
  
    for(int i= 1; i<=n; i++){  
        System.out.println(i);  
    }  
  
    System.out.println("OK");  
  
    for(int i= 1; i<=n; i++){  
        for(int j= 1; j<=n; j++){  
            System.out.println(i+"\t"+j);  
        }  
    }  
    System.out.println("OK");  
  
    System.out.println("End");  
}
```

$$f(n) = O(1 + 1 + n + 1 + (n * n) + 1 + 1)$$

$$= O(n^2 + n + 5)$$

$$= O(n^2) \Rightarrow \text{Quadratic Time Complexity}$$



Example 6:

```
void show(int n) {  
    System.out.println("Begin");  
  
    System.out.println("OK");  
  
    for(int i= 1; i<=n; i++){  
        System.out.println(i);  
    }  
  
    System.out.println("OK");  
  
    for(int i= 1; i<=n; i++){  
        for(int j= 1; j<=n; j++){  
            for(int k= 1; k<=n; k++){  
  
                System.out.println(i+"\t"+j+"\t"+k);  
            }  
        }  
    }  
  
    System.out.println("OK");  
  
    System.out.println("End");  
}
```

$$\begin{aligned}f(n) &= O(1 + 1 + n + 1 + (n * n * n) + 1 + 1) \\&= O(n^3 + n + 5) \\&= O(n^3) \Rightarrow \text{Cubic Time Complexity}\end{aligned}$$



Example 7:

```
void show(int m,int n) {  
    System.out.println("Begin");  
  
    System.out.println("OK");  
  
    for(int i= 1; i<=n; i++){  
        System.out.println(i);  
    }  
  
    System.out.println("OK");  
  
    for(int i= 1; i<=m; i++){  
        System.out.println(i);  
    }  
  
    System.out.println("OK");  
  
    System.out.println("End");  
}
```

$f(n) = O(1 + 1 + n + 1 + m + 1 + 1)$

$= O(m + n + 5)$

$= O(m+n) \Rightarrow \text{Linear Time Complexity}$

14. Space Complexity

- ♦ Space complexity refers to the total amount of memory space used by an algorithm including the space of input values.
- ♦ Auxiliary space is simply extra or temporary space, and it is not the same as space complexity.

Space Complexity = Auxiliary space + space use by input values

- ♦ The best algorithm/program should have a low level of space complexity.
- ♦ The less space required, the faster it executes.

Example 1:

```
void sum(int a, int b) {  
    int sum = a + b;  
    System.out.println(sum);  
}
```

$f(n) = O(1 + 1 + 1)$

$= O(3)$

$= O(1) \Rightarrow \text{Constant Space Complexity}$



Example 2:

```
int sum(int []arr) {  
    int sum= 0;  
  
    for(int i= 0; i<arr.length; i++){  
        sum = sum + arr[i];  
    }  
    return sum;  
}
```

$f(n) = O(n + 1)$
 $= O(n + 1)$
 $= O(n) \Rightarrow \text{Linear Space Complexity}$

Example 3:

```
void show(int arr[]) {  
  
    int myarr [] = new int[arr.length];  
  
    for(int i= 0; i<n; i++){  
        myarr[i]= arr[i] * arr[i];  
    }  
}
```

$f(n) = O(n + n)$
 $= O(2n)$
 $= O(n) \Rightarrow \text{Linear Space Complexity}$



Java Learning Center

No.1 In Java Training & placement

DSA

Module 2

Numbers and Loops

Author

Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

2. Control Statements

- ♦ Control Statements are used to control the execution flow of Java Program.

There are three types of control statements:

1) Conditional Control Statements

- a. if statement
- b. switch statement

2) Looping Control Statements

- a. for statement
- b. Enhanced for Statement (Added in Java 5)
- c. while statement
- d. do-while statement

3) Unconditional Control Statements

- a. break
- b. continue

2.1. Conditional Control Statements

- ♦ Conditional Control Statements are also called as decision-making statements or selection statements.
- ♦ It allows the program to select between alternative actions during the program execution.
- ♦ The execution will be based on the result of conditional expression.

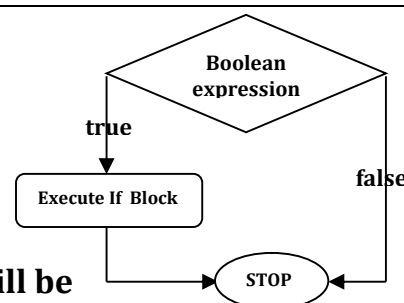
2.1.1. if Statement

1) Simple If

```
if(<condition>){  
    // Statements (IF BLOCK)  
}
```

```
// Other Statements
```

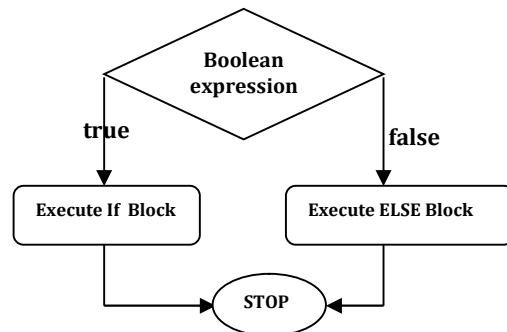
- ♦ It indicates whether IF BLOCK Statements will be executed or not, as per the condition.





2) If-else Statement

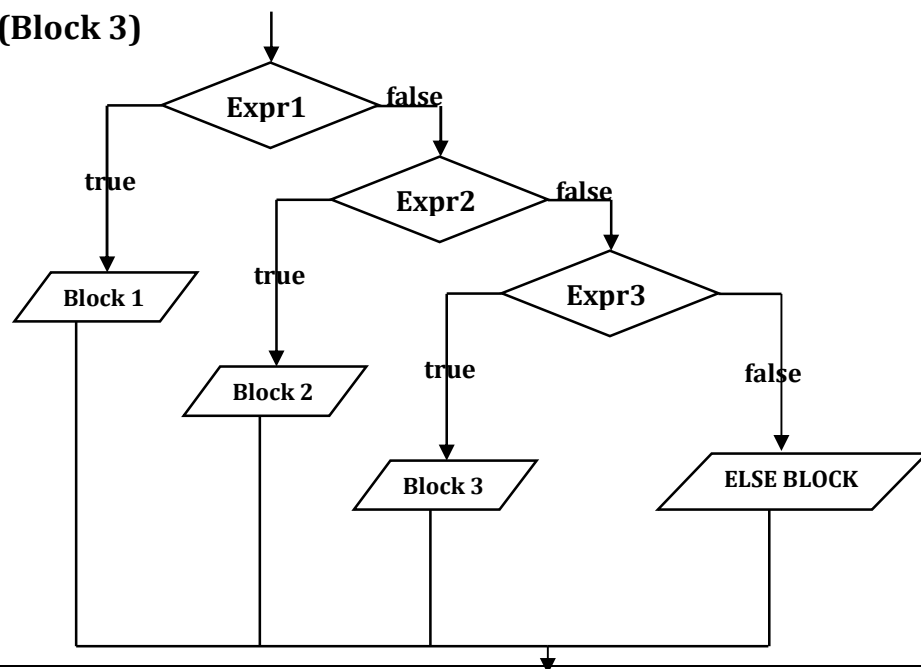
```
if(<condition>){  
    // Statements (IF BLOCK)  
}else{  
    // Statements (ELSE BLOCK)  
}
```



- ♦ It indicates which block will be executed, as per the condition.
- ♦ if condition is true IF BLOCK Statements will be executed otherwise ELSE BLOCK Statements will be executed.

3) If Else-if statement

```
if(<condition1>){  
    // Statements (Block 1)  
}else if(<condition2>){  
    // Statements (Block 2)  
}else if(<condition3>){  
    // Statements (Block 3)  
}  
...  
else{  
    // Statements  
}
```



- ♦ Condition can be any expression which returns boolean value.
- ♦ Block notation is mandatory when if block or else block have more than one statement.
- ♦ Block notation is optional when if block or else block have only one statement.
- ♦ When you declare any variable within if block or else block then that variable can be accessed within that block only. It can't be accessed from outside that block.

1) Demo1.java

```
package com.jlcindia;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo1{
    public static void main(String as[]) {
        int a=42;
        if(a % 2==0)
            System.out.println("Number is EVEN");
        else
            System.out.println("Number is ODD");
    }
}
```



2) Demo2.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo2{
public static void main(String as[]) {
int a=45;
if(a % 5==0)
    System.out.println("Number is Divisible by 5");
else
    System.out.println("Number is Not Divisible by 5");
}
}
```

3) Demo3.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo3{
public static void main(String as[]) {
int a=10;
int b=20;
int max=0;
if(a>b)
    max=a;
else
    max = b;

System.out.println("Max is "+max);

}
}
```




4) Demo4.java

```
package com.jlcindia;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo4{
    public static void main(String as[]) {
        int a=10;
        int b=20;
        int c=15;
        int max=0;
        if(a>b && a> c)
            max=a;
        else if(b> a && b>c)
            max = b;
        else
            max=c;
        System.out.println("Max is "+max);
    }
}
```

2.1.2 switch Statement

- ♦ Switch statement allows the program to select one action among multiple actions during the program execution.

Syntax:

```
switch(<variable/expression/value>){  
    case <val1>:  
        // Statements    [<break>];  
    case <val2>:  
        // Statements    [<break>];  
    ...  
    default:  
        // Statements  
}
```

- ♦ First switch expression will be evaluated and then tries to find case value matching with resultant value of the expression.
- ♦ If matching case is found then the statements related to that case will be executed otherwise statements related to default case will be executed.
- ♦ Following types of values can be passed in switch as an argument up to Java 2:
 - byte
 - short
 - char
 - int
- ♦ enum can be passed in switch as an argument from Java 5.
- ♦ String can also be passed in switch as an argument from Java 7.



5) Demo5.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo5{
public static void main(String as[]) {
int a=3;
switch(a){
case 0:
    System.out.println("Sunday");
    break;
case 1:
    System.out.println("Monday");
    break;
case 2:
    System.out.println("Tuesday");
    break;
case 3:
    System.out.println("Wednesday");
    break;
case 4:
    System.out.println("Thursday");
    break;
case 5:
    System.out.println("Friday");
    break;
case 6:
    System.out.println("Saturday");
    break;
default:
    System.out.println("Wrong Input");
}
}
}
```



6) Demo6.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo5{
public static void main(String as[]) {

char ch='A';
if(ch>=65 && ch<=90){
switch(ch){
case 'A':
case 'E':
case 'I':
case 'O':
case 'U':
    System.out.println("Character is Vowel");
    break;
default:
    System.out.println("Character is Consonant");
}
}else{
    System.out.println("Invalid Alphabet");
}

}
}
```



2.2. Looping Control Statements

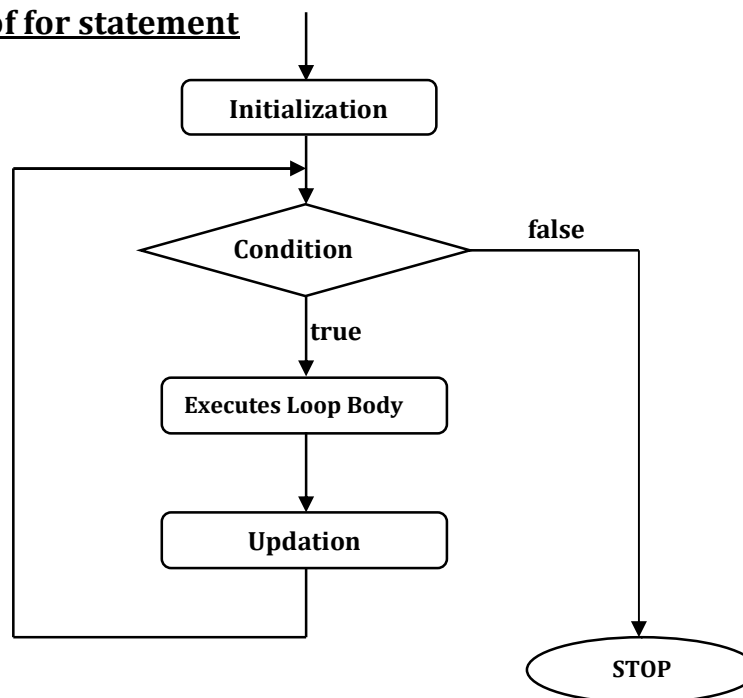
- ♦ Looping Control statements are also called as Iteration statements.
- ♦ These statements are used to execute a block of statements repeatedly.
- ♦ An expression is required for looping control statements which returns boolean value.
- ♦ Block notation is mandatory when looping block has more than one statement.
- ♦ Block notation is optional when looping block has only one statement.
- ♦ When you declare any variable within loop block then that variable can be accessed within that block only. It can't be accessed from outside that block.

2.2.1 for Statement

Syntax:

```
for(<initialization> ; <condition> ; <updation>)  
{  
  // Statements  
}
```

Processing Flow of for statement



- a) Initialization Statement will be executed only once first time.
- b) Conditional & Updation statement can be executed more than once.

7) Demo7.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo7{
public static void main(String as[]) {

int num=10;
for(int i=1;i<=num;i++){
System.out.println(i);
}

}
}
```

8) Demo8.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo8{
public static void main(String as[]) {

int num=25;
for(int i=1;i<=num;i++){
if(i % 2 == 0)
System.out.println(i);
}
}
}
```

2.2.2 Enhanced For Statement

- ♦ It is a new feature added in Java 5.
- ♦ It is also called as for-each loop.
- ♦ The enhanced for-loop is mainly used to access the elements from Collection/Array sequentially.

Syntax:

```
for(<data Type> varName : array | Collection) {  
    // Statements (BLOCK)  
}
```

9) Demo9.java

```
package com.jlcindia;  
  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
class Demo9{  
    public static void main(String as[]) {  
  
        int arr[]={99,88,77,66,55};  
  
        System.out.println("Length :"+arr.length);  
  
        for(int element []: arr){  
            System.out.println(element);  
        }  
    }  
}
```

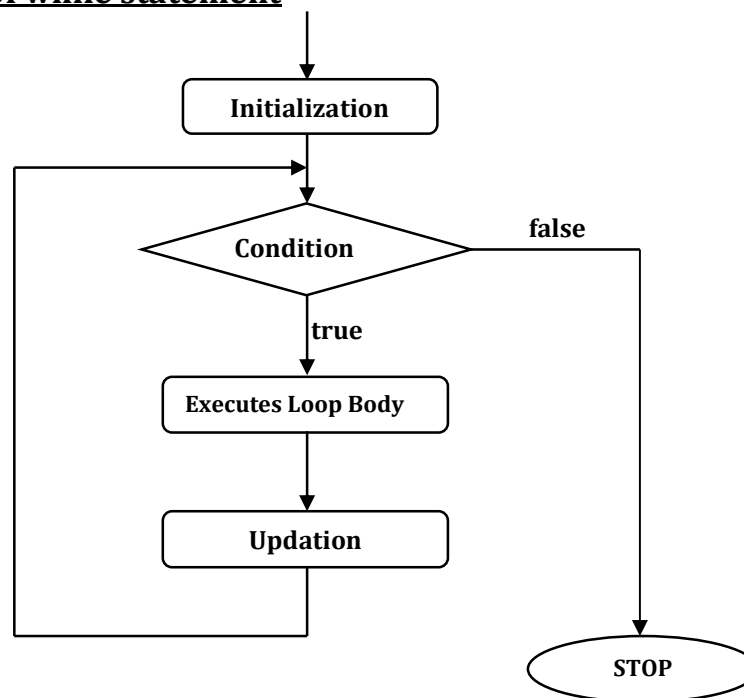
2.2.3 while Statement

Syntax:

```
<initialization>  
while( <condition>){  
    // Statements (LOOP BODY)  
    <updation>  
}
```

- ♦ condition of while statement is mandatory and must be boolean type.

Processing Flow of while statement





10) Demo10.java

```
package com.jlccindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo10{
public static void main(String as[]) {

int i=1;
while(i<=10){
System.out.println(i);
i++;
}

}
}
```

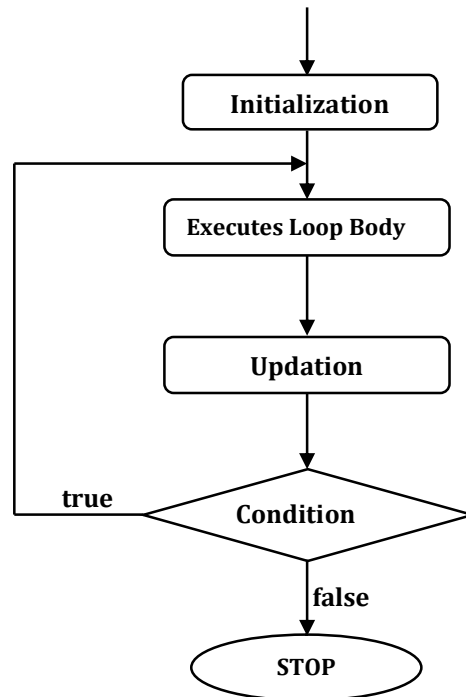
2.2.4 do-while Statement

- ♦ When you are using for statement or while statement then it verifies the condition before executing the block.
- ♦ So in the case of for statement and while statement, when first time condition is false then the block will not be executed.
- ♦ for and while statement are also called as Entry Controlled Loop.
- ♦ If you want to execute the block at least once then use do-while statement.

Using do-while loop

- ♦ In the case of do-while first block of statements will be executed and then condition will be verified.
- ♦ do-while statement is also called as Exit Controlled Loop.

Processing Flow of while statement



11) Demo11.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo11{
public static void main(String as[]) {

int i=1;
do{
System.out.println(i);
i++;
}while(i<=10);

}
}
```

2.3 Unconditional Control Statements

2.3.1 break Statement

- ♦ **break** is a keyword.
- ♦ It can be used within switch or any looping statements.
- ♦ It is used to terminate the execution of the current looping/switch statement.
- ♦ **break** can be used in two ways:
 - **break;**
 - **break <label>;**

12) Demo12.java

```
package com.jlccindia;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo12{
    public static void main(String as[]) {

        for(int i=0;i<10;i++){
            System.out.println(i);
            break;
            System.out.println("---Inside Loop---");
        }
        System.out.println("---Outside Loop---");

    }
}
```

2.3.2 continue Statement

- ♦ **continue** is a keyword.
- ♦ It can be used within any looping statements.
- ♦ It is used to continue the execution of the current looping statement with next iteration.
- ♦ **continue** can be used in two ways:
 - **continue;**
 - **continue <label>;**

13) Demo13.java

```
package com.jlccindia;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Demo13{
    public static void main(String as[]) {

        for(int i=0;i<10;i++){
            System.out.println(i);
            continue;
            System.out.println("---Inside Loop---");
        }
        System.out.println("---Outside Loop---");

    }
}
```

2.4. Programs on Numbers and Loops

1) Lab1.java

```
package com.jlcindia;

// 1. WAP to Swap Two Numbers without third Variable.

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Lab1 {
    public static void main(String as[]) {

        int a=10;
        int b=20;

        System.out.println("a = "+a+ " , "+ "b = "+b);

        a= a + b;
        b= a - b;
        a= a - b;

        System.out.println("a = "+a+ " , "+ "b = "+b);

    }
}
```



2) Lab2.java

```
package com.jlcindia;
```

```
// 2. WAP to find Max of Two Numbers.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab2 {
```

```
public static void main(String as[]) {
```

```
    int a=20;
```

```
    int b=10;
```

```
    int max = 0;
```

```
    if(a!=b) {
```

```
        if(a>b) {
```

```
            max= a;
```

```
        }
```

```
        else {
```

```
            max=b;
```

```
        }
```

```
        System.out.println("Max : "+max);
```

```
    }
```

```
    else {
```

```
        System.out.println("Both are Same ");
```

```
    }
```

```
}
```

```
}
```



3) Lab3.java

```
package com.jlcindia;
```

```
// 3. WAP to Print the Numbers from 1 to N.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab3 {
```

```
public static void main(String as[]) {
```

```
    int n = 10;
```

```
    for (int i = 1; i <= n; i++) {  
        System.out.println(i);  
    }
```

```
}
```

```
}
```



4) Lab4.java

```
package com.jlcindia;
```

```
// 4. WAP to Print the Even Numbers from 1 to N.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab4 {
```

```
public static void main(String as[]) {
```

```
    int n = 10;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        if (i % 2 == 0)
```

```
            System.out.println(i);
```

```
    }
```

```
}
```

```
}
```




5) Lab5.java

```
package com.jlcindia;
```

```
// 5. WAP to Print the Numbers Divisible by 5 from 1 to N.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab5 {
```

```
public static void main(String as[]) {
```

```
    int n = 25;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        if (i % 5 == 0)
```

```
            System.out.println(i);
```

```
    }
```

```
}
```

```
}
```



6) Lab6.java

```
package com.jlcindia;
```

```
// 6. WAP to Print Divisibles of the given Number.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab6 {
```

```
public static void main(String as[]) {
```

```
    int n = 10;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        if (n % i == 0)
```

```
            System.out.println(i);
```

```
    }
```

```
}
```

```
}
```



7) Lab7.java

```
package com.jlcindia;
```

```
// 7. WAP to Find the Number of Divisibles of the given Number.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab7 {
```

```
public static void main(String as[]) {
```

```
int n = 10;
```

```
    int count = 0;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        if (n % i == 0)
```

```
            count ++;
```

```
    }
```

```
    System.out.println("Count : "+ count);
```

```
}
```

```
}
```



8) Lab8.java

```
package com.jlcindia;
```

```
// 8. WAP to Find whether given Number is Prime or Not.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab8 {
```

```
public static void main(String as[]) {
```

```
    int n = 10;
```

```
    int count = 0;
```

```
    for (int i = 2; i <= n/2; i++) {
```

```
        if (n % i == 0) {
```

```
            count ++;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if(count==0) {
```

```
        System.out.println(n + " is Prime: ");
```

```
    }else {
```

```
        System.out.println(n + " is Not Prime: ");
```

```
    }
```

```
}
```

```
}
```



9) Lab9.java

```
package com.jlcindia;
```

```
// 9) WAP to print the prime numbers between M to N.
```

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Lab9 {
    public static void main(String as[]) {
        private static boolean isPrime(int num) {
            int count = 0;
            boolean primeFlag=false;
            for (int i = 2; i <= num / 2; i++) {
                if (num % i == 0) {
                    count++;
                    break;
                }
            }
            if (count == 0) {
                primeFlag = true;
            }
            return primeFlag;
        }

        public static void main(String[] args) {
            int m= 1;
            int n = 25;
            for (int num = m; num <= n; num++) {
                if (isPrime(num)) {
                    System.out.println(num);
                }
            }
        }
    }
}
```



9A) Lab9A.java

```
package com.jlcindia;
```

```
// 9A) WAP to print the prime numbers between M to N.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab9A {
```

```
public static void main(String as[]) {
```

```
    private static boolean isPrime(int num) {
```

```
        boolean primeFlag=true;
```

```
        for (int i = 2; i <= num / 2; i++) {
```

```
            if (num % i == 0) {
```

```
                primeFlag=false;
```

```
                break;
```

```
            }
```

```
        }
```

```
        return primeFlag;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int m= 10;
```

```
        int n = 25;
```

```
        for (int num = m; num <= n; num++) {
```

```
            if (isPrime(num)) {
```

```
                System.out.println(num);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



9B) Lab9B.java

```
package com.jlcindia;
```

```
// 9B) WAP to print the prime numbers between M to N.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
class Lab9B {
```

```
public static void main(String as[]) {
```

```
    private static boolean isPrime(int num) {
```

```
        for (int i = 2; i <= num / 2; i++) {
```

```
            if (num % i == 0) {
```

```
                return false;
```

```
            }
```

```
        }
```

```
        return true;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int m= 10;
```

```
        int n = 25;
```

```
        for (int num = m; num <= n; num++) {
```

```
            if (isPrime(num)) {
```

```
                System.out.println(num);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



10) Lab10.java

```
package com.jlcindia;

// 10) WAP to find sum of numbers from 1 to n.
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab10 {
    public static void main(String[] args) {
        int num = 5;
        int sum = 0;
        for (int i = 1; i <= num; i++) {
            sum = sum + i;
        }
        System.out.println("Sum : "+sum);
    }
}
```

10A) Lab10A.java

```
package com.jlcindia;

// 10) WAP to find sum of numbers from m to n.
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab10A {
    public static void main(String[] args) {
        int m= 1;
        int n = 5;
        int sum = (m + n) * n/2 ;
        System.out.println("Sum : "+sum);
    }
}
```




11) Lab11.java

```
package com.jlcindia;
```

```
// 11) WAP to find sum of squares of numbers from 1 to n.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab11 {
```

```
    public static void main(String[] args) {
```

```
        int num = 5;
```

```
        int sum = 0;
```

```
        for (int i = 1; i <= num; i++) {
```

```
            sum = sum + i * i; // i contains each Number
```

```
        }
```

```
        System.out.println("Sum : "+sum);
```

```
    }
```

```
}
```



12) Lab12.java

```
package com.jlcindia;
```

```
// 12) WAP to find sum of cubes of numbers from 1 to n.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab12 {
```

```
    public static void main(String[] args) {
```

```
        int num = 3;
```

```
        int sum = 0;
```

```
        for (int i = 1; i <= num; i++) {
```

```
            sum = sum + i * i * i; // i contains each Number
```

```
        }
```

```
        System.out.println("Sum : "+sum);
```

```
    }
```

```
}
```



13) Lab13.java

```
package com.jlcindia;
```

```
// 13) WAP to find sum of divisible of a given number
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab13 {
```

```
    public static void main(String[] args) {
```

```
        int n = 10; // Given Number
```

```
        int sum = 0;
```

```
        for (int i = 1; i <= n; i++) {
```

```
            if (n % i == 0) //i is having each number
```

```
                sum = sum + i;
```

```
        }
```

```
        System.out.println("Sum : "+ sum);
```

```
    }
```

```
}
```



14) Lab14.java

```
package com.jlcindia;
```

```
// 14) WAP to find whether given number is perfect or not.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab14 {
```

```
    public static void main(String[] args) {
```

```
        int num = 6; // Given Number
```

```
        int sum = 0;
```

```
        for (int i = 1; i <= num / 2 ; i++) {
```

```
            if (num % i == 0) // i is having each number
```

```
                sum = sum + i;
```

```
        }
```

```
        if (sum == num)
```

```
            System.out.println(num + " is Perfect Number");
```

```
        else
```

```
            System.out.println(num + " is Not Perfect Number");
```

```
    }
```

```
}
```



15) Lab15.java

```
package com.jlcindia;
```

```
// 15) WAP to display the perfect numbers from m to n.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab15 {
```

```
    private static boolean isPerfect(int num) {
```

```
        int sum = 1;
```

```
        for (int i = 2; i <= num / 2 ; i++) {
```

```
            if (num % i == 0) // i is having each number
```

```
                sum = sum + i;
```

```
        }
```

```
        if (sum == num)
```

```
            return true;
```

```
        else
```

```
            return false;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int m= 1;
```

```
        int n = 50;
```

```
        for(int k=m;k<=n;k++) {
```

```
            if(isPerfect(k)) {
```

```
                System.out.println(k);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```



16) Lab16.java

```
package com.jlcindia;
```

```
// 16) WAP to Print the digits of given number?
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab16 {
```

```
    public static void main(String[] args) {
```

```
        int num = 123;
```

```
        while(num!=0) {
```

```
            int rem = num %10;
```

```
            System.out.println(rem);
```

```
            num = num /10;
```

```
        }
```

```
    }
```

```
}
```



17) Lab17.java

```
package com.jlcindia;
```

```
// 17) WAP to find number of digits of given number?
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab17 {
```

```
    public static void main(String[] args) {
```

```
        int num = 123;
```

```
        int count = 0;
```

```
        while(num!=0) {
```

```
            //int rem = num %10;
```

```
            count++;
```

```
            num = num /10;
```

```
        }
```

```
        System.out.println("No of Digits : "+ count);
```

```
    }
```

```
}
```



18) Lab18.java

```
package com.jlcindia;
```

```
// 18) WAP to find sum of individual digits of given number?
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab18 {
```

```
    public static void main(String[] args) {
```

```
        int num = 123;
```

```
        int sum = 0;
```

```
        while(num!=0) {
```

```
            int digit = num %10;
```

```
            sum = sum +digit;
```

```
            num = num /10;
```

```
        }
```

```
        System.out.println("Sum : "+ sum);
```

```
    }
```

```
}
```




18A) Lab18A.java

```
package com.jlccindia;
```

```
// 18) WAP to find sum of individual digits of given number?
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab18A {
```

```
    public static void main(String[] args) {
```

```
        int num = 123;
```

```
        int sum = 0;
```

```
        for(;num!=0;num = num /10) {
```

```
            int rem = num %10;
```

```
            sum = sum +rem;
```

```
        }
```

```
        System.out.println("Sum : "+ sum);
```

```
    }
```

```
}
```



19) Lab19java

```
package com.jlcindia;
```

```
// 19) WAP to find sum of Squares of individual digits of given number?
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab19 {
```

```
    public static void main(String[] args) {
```

```
        int num = 123;
```

```
        int sum = 0;
```

```
        while(num!=0) {
```

```
            int digit = num %10;
```

```
            sum = sum + digit * digit ;
```

```
            num = num /10;
```

```
        }
```

```
        System.out.println("Sum : "+ sum);
```

```
    }
```

```
}
```



20) Lab20.java

```
package com.jlcindia;
```

```
// 20) WAP to find sum of cubes of individual digits of given number?
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab20{
```

```
    public static void main(String[] args) {
```

```
        int num = 123;
```

```
        int sum = 0;
```

```
        while(num!=0) {
```

```
            int digit = num %10;
```

```
            sum = sum + digit * digit * digit;
```

```
            num = num /10;
```

```
        }
```

```
        System.out.println("Sum : "+ sum);
```

```
    }
```

```
}
```

```
// Iterations : No of Digits times
```



21) Lab21.java

```
package com.jlcindia;
```

```
// 21) WAP to find the Factorial of given Number.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab21{
```

```
    public static void main(String[] args) {
```

```
        int num = 1;
```

```
        int fact = 1;
```

```
        for(int i=2; i<= num;i++) {
```

```
            fact = fact * i;
```

```
        }
```

```
        System.out.println("Factorial is : "+ fact);
```

```
    }
```

```
}
```



21A) Lab21A.java

```
package com.jlcindia;
```

```
import java.math.BigInteger;
```

```
// 21) WAP to find the Factorial of given Number.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab21A{
```

```
    public static void main(String[] args) {
```

```
        int num = 200;
```

```
        BigInteger fact = BigInteger.valueOf(1);
```

```
        for(int i=2; i<= num;i++) {
```

```
            fact = fact.multiply(BigInteger.valueOf(i)) ;
```

```
        }
```

```
        System.out.println("Factorial is : "+ fact);
```

```
    }
```

```
}
```

```
// For Large Numbers , use BigInteger class
```



22) Lab22.java

```
package com.jlcindia;
```

```
//22) WAP to find sum of Factorials of individual digits of given number?
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab22 {
```

```
    private static int factorial(int num) {
```

```
        int fact = 1;
```

```
        for (int i = 2; i <= num; i++) {
```

```
            fact = fact * i;
```

```
        }
```

```
        return fact;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int num = 1234;
```

```
        int sum = 0;
```

```
        while(num!=0) {
```

```
            int digit = num %10; //Each Digit
```

```
            sum = sum + factorial(digit);
```

```
            num = num /10;
```

```
        }
```

```
        System.out.println("Sum : "+ sum);
```

```
    }
```

```
}
```



23) Lab23.java

```
package com.jlcindia;
```

```
// 23) WAP to find whether given number is strong or not.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab23 {
```

```
    private static int factorial(int num) {
```

```
        int fact = 1;
```

```
        for (int i = 2; i <= num; i++) {
```

```
            fact = fact * i;
```

```
        }
```

```
        return fact;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int num = 145;
```

```
        int sum = 0;
```

```
        int numCopy = num;
```

```
        while (num != 0) {
```

```
            int digit = num % 10;
```

```
            sum = sum + factorial(digit);
```

```
            num = num / 10;
```

```
        }
```

```
        if (sum == numCopy)
```

```
            System.out.println(numCopy + " is Strong Number ");
```

```
        else
```

```
            System.out.println(numCopy + " is Not Strong Number ");
```

```
    }
```

```
}
```



24) Lab24.java

```
package com.jlcindia;
```

```
// 24) WAP to display the strong numbers from m to n.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab24 {
```

```
    private static int factorial(int num) {
```

```
        int fact = 1;
```

```
        for (int i = 2; i <= num; i++) {
```

```
            fact = fact * i;
```

```
        }
```

```
        return fact;
```

```
    }
```

```
    private static boolean isStrong(int num) {
```

```
        int sum = 0;
```

```
        int numCopy = num;
```

```
        while (num != 0) {
```

```
            int digit = num % 10;
```

```
            sum = sum + factorial(digit);
```

```
            num = num / 10;
```

```
        }
```

```
        if (sum == numCopy)
```

```
            return true;
```

```
        else
```

```
            return false;
```

```
    }
```




```
public static void main(String[] args) {  
  
    int m = 1;  
    int n = 150;  
  
    for (int i = m; i <= n; i++) {  
        if (isStrong(i)) {  
            System.out.println(i);  
        }  
    }  
}
```



25) Lab25.java

```
package com.jlcindia;
```

```
// 25) WAP to find whether given number is armstrong or not.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab25 {
```

```
    private static int cube(int num) {
```

```
        return num * num * num;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int num = 153;
```

```
        int sum = 0;
```

```
        int numCopy = num;
```

```
        while (num != 0) {
```

```
            int digit = num % 10;
```

```
            sum = sum + cube(digit);
```

```
            num = num / 10; // Changing the Number
```

```
        }
```

```
        if (sum == numCopy)
```

```
            System.out.println(numCopy + " is ArmStrong Number ");
```

```
        else
```

```
            System.out.println(numCopy + " is Not ArmStrong Number ");
```

```
    }
```

```
}
```



26) Lab26.java

```
package com.jlcindia;
```

```
//26) WAP to display the Armstrong numbers from 1 to n.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab26 {
```

```
    private static int cube(int num) {  
        return num * num * num;  
    }
```

```
    private static boolean isArmStrong(int num) {
```

```
        int sum = 0;  
        int numCopy = num;  
        while (num != 0) {  
            int digit = num % 10;  
            sum = sum + cube(digit);  
            num = num / 10;  
        }
```

```
        if (sum == numCopy)  
            return true;  
        else  
            return false;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int m = 1;  
        int n = 1000;
```

```
        for (int i = m; i <= n; i++) {  
            if (isArmStrong(i)) {  
                System.out.println(i);  
            }  
        }  
    }  
}
```

27) Lab27.java

```
package com.jlcindia;
```

```
// 27) WAP to print the Reverse of the given number.
```

```
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public class Lab27 {  
    public static void main(String[] args) {  
  
        int num = 1234;  
        int sum = 0;  
  
        while (num != 0) {  
            int digit = num % 10;  
            sum = sum * 10 + digit ;  
            num = num / 10; // Changing the Number  
        }  
  
        System.out.println(" Reverse Number is "+ sum);  
    }  
}
```



28) Lab28.java

```
package com.jlcindia;
```

```
// 28) WAP to find whether given number is Palindrome or not.
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab28 {
```

```
    public static void main(String[] args) {
```

```
        int num = 121;
```

```
        int numCopy = num;
```

```
        int rev = 0;
```

```
        while (num != 0) {
```

```
            int digit = num % 10;
```

```
            rev = rev * 10 + digit ;
```

```
            num = num / 10; // Changing the Number
```

```
        }
```

```
        if(numCopy == rev)
```

```
            System.out.println( numCopy + " is Palindrome");
```

```
        else
```

```
            System.out.println( numCopy + " is NOT Palindrome");
```

```
    }
```

```
}
```



29) Lab29.java

```
package com.jlcindia;

/*
29) WAP to print the fibonocii series.
0 1 1 2 3 5 8 13
*/

/*
* @Author : Srinivas Dande
* @Company: Java Learning Center
**/

public class Lab29 {
    public static void main(String[] args) {

        int n = 2; // Number of values Required in Series
        // Validation Code
        int firstNum = 0;
        int secondNum = 1;

        System.out.print(firstNum + "\t" + secondNum);

        int nextNum = 0;

        for (int i = 3; i <= n; i++) {
            nextNum = firstNum + secondNum;
            System.out.print("\t" + nextNum);
            firstNum = secondNum;
            secondNum = nextNum;
        }
    }
}
```



29A) Lab29A.java

```
package com.jlcindia;

/*
29) WAP to print the fibonocii series.
0 1 1 2 3 5 8 13
*/

/*
* @Author : Srinivas Dande
* @Company: Java Learning Center
**/
public class Lab29A {
    public static void main(String[] args) {

        int n = 5; // Number of values Required in Series
        if(n<2) {
            System.out.println("Invalid Input");
        }else {

            int firstNum = 0;
            int secondNum = 1;
            System.out.print(firstNum + "\t" + secondNum);

            int nextNum = 0;
            for (int i = 1; i <= n-2; i++) {
                nextNum = firstNum + secondNum;
                System.out.print("\t" + nextNum);
                firstNum = secondNum;
                secondNum = nextNum;
            }
        }
    }
}
```



30) Lab30.java

```
package com.jlcindia;
```

```
/*
```

```
30) WAP to print the following series.
```

```
1 1 6 4 11 9 16 16 21 25
```

```
*/
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab30 {
```

```
    public static void main(String[] args) {
```

```
        int n= 5; // Number of values Required in Series
```

```
        //Input - Only Even Numbers
```

```
        int a = 1;
```

```
        int b = 1;
```

```
        for (int i = 1; i <= n/2; i++) {
```

```
            System.out.print(a+"\t"+b+"\t");
```

```
            a = a+5;
```

```
            b = (i+1) * (i+1);
```

```
        }
```

```
    }
```

```
}
```




31) Lab31.java

```
package com.jlcindia;
```

```
/*
```

```
31) WAP to print the following series.
```

```
1 1 3 8 6 27 9 64 12 125
```

```
*/
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
**/
```

```
public class Lab31 {
```

```
    public static void main(String[] args) {
```

```
        int n= 5; // Number of values Required in Series
```

```
        //Input - Only Even Numbers
```

```
        int a = 1;
```

```
        int b = 1;
```

```
        for (int i = 1; i <= n/2; i++) {
```

```
            System.out.print(a+"\t"+b+"\t");
```

```
            a = 3 * i;
```

```
            b = i+1;
```

```
            b = b * b * b;
```

```
        }
```

```
    }
```

```
}
```



32) Lab32.java

```
package com.jlcindia;
```

```
/*
```

32) WAP to evaluate following series.

$1 + 2/x + 3/x^2 + 4/x^3 + 5/x^4 +$

$x = 1 \Rightarrow 1 + 2/x + 3/x^2 \Rightarrow 1 + 2 + 3 \Rightarrow 6$

$x = 2 \Rightarrow 1 + 2/x + 3/x^2 \Rightarrow 1 + 1 + 3/4 \Rightarrow 2.75$

```
*/
```

```
/*
```

*** @Author : Srinivas Dande**

*** @Company: Java Learning Center**

```
*/
```

```
public class Lab32 {
```

```
    private static double power( int n , int p) {
```

```
        double power = 1;
```

```
        for(int i=1;i<=p;i++) {
```

```
            power = power * n;
```

```
        }
```

```
        return power;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int n= 3; // Number of values Required in Series
```

```
        int x = 2;
```

```
        double sum= 0.0 ;
```

```
        for (int i = 1; i <= n; i++) {
```

```
            sum = sum + i / power(x,i-1);
```

```
        }
```

```
        System.out.println("Sum : "+sum);
```

```
    }
```

```
}
```



33) Lab33.java

```
package com.jlcindia;
```

```
/*
```

33) WAP to evaluate following series.

$1 - 2/x + 3/x^2 - 4/x^3 + 5/x^4 - 6/x^5$

$x = 1 \Rightarrow 1 - 2/x + 3/x^2 \Rightarrow 1 - 2 + 3 \Rightarrow 2$

$x = 2 \Rightarrow 1 - 2/x + 3/x^2 \Rightarrow 1 - 1 + 3/4 \Rightarrow 0.75$

```
*/
```

```
/*
```

*** @Author : Srinivas Dande**

*** @Company: Java Learning Center**

```
*/
```

```
public class Lab33 {
```

```
    private static double power( int n , int p) {
```

```
        double power = 1;
```

```
        for(int i=1;i<=p;i++) {
```

```
            power = power * n;
```

```
        }
```

```
        return power;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int n= 3; // Number of values Required in Series
```

```
        int x = 1;
```

```
        double sum= 0.0 ;
```

```
        int sign = 1;
```

```
        for (int i = 1; i <= n; i++) {
```

```
            sum = sum + sign * ( i / power(x,i-1) );
```

```
            sign = sign * -1;
```

```
        }
```

```
        System.out.println("Sum : "+sum);
```

```
    }
```

```
}
```



34) Lab34.java

```
package com.jlcindia;
```

```
/*
```

34) WAP to evaluate following series.

$1/(x-1)! + x/x! + x^2/(x+1)! + x^3/(x+2)! + x^4/(x+3)!$

$x = 1 \Rightarrow 1+1+1/2 \Rightarrow 2.5$

$x = 2 \Rightarrow 1+1+4/6 \Rightarrow 2.66$

```
*/
```

```
/*
```

*** @Author : Srinivas Dande**

*** @Company: Java Learning Center**

```
*/
```

```
public class Lab34 {
```

```
    private static double power( int n , int p) {  
        double power = 1;  
        for(int i=1;i<=p;i++) {  
            power = power * n;  
        }  
        return power;  
    }
```

```
    private static long factorial( int n ) {  
        long fact = 1;  
        for(int i=1;i<=n;i++) {  
            fact = fact * i;  
        }  
        return fact;  
    }
```

```
    public static void main(String[] args) {
```

```
        int n= 3; // Number of values Required in Series  
        int x = 2;  
        int k= -1;
```



```
double sum= 0.0 ;

for (int i = 1; i <= n; i++) {
    sum = sum + power(x,i-1) / factorial (x+k) ;
    k++;
}

System.out.println("Sum : "+sum);

}

}
```

35) Lab35.java

```
package com.jlcindia;
```

```
/*
```

35) WAP to evaluate following series.

$1/(x-1)! - x/x! + x^2/(x+1)! - x^3/(x+2)! + x^4/(x+3)!$

$x = 1 \Rightarrow 1-1+1/2 \Rightarrow 0.5$

$x = 2 \Rightarrow 1-1+4/6 \Rightarrow 0.66$

```
*/
```

```
/*
```

*** @Author : Srinivas Dande**

*** @Company: Java Learning Center**

```
**/
```

```
public class Lab35 {
    private static double power( int n , int p) {
        double power = 1;
        for(int i=1;i<=p;i++) {
            power = power * n;
        }
        return power;
    }
}
```



```
private static long factorial( int n ) {
    long fact = 1;
    for(int i=1;i<=n;i++) {
        fact = fact * i;
    }
    return fact;
}

public static void main(String[] args) {

    int n= 3; // Number of values Required in Series
    int x = 1;
    int k= -1;
    double sum= 0.0 ;
    int sign = 1;

    for (int i = 1; i <= n; i++) {
        sum = sum + sign * ( power(x,i-1) / factorial (x+k) );
        k = k + 1;
        sign = sign * -1;
    }

    System.out.println("Sum : "+sum);

}
```



36) Lab36.java

```
package com.jlcindia;
/*
36) Print the following Pattern
*****
*****
*****
*****
*****

*/

/*
* @Author : Srinivas Dande
* @Company: Java Learning Center
**/
public class Lab36{
    public static void main(String[] args) {

        int stars = 5;
        int rows= 5;

        for(int i=1;i<=rows;i++) {

            //Task 1: Print Stars
            for (int j = 1; j <= stars; j++) {
                System.out.print(" * ");
            }

            //Task 2: Print Newline
            System.out.println("");

        }
    }
}
```



37) Lab37.java

```
package com.jlcindia;
/*
37) Print the following Pattern
    *
   **
  ***
 ****
*****

*/

/*
* @Author : Srinivas Dande
* @Company: Java Learning Center
**/
public class Lab37{
    public static void main(String[] args) {

        int stars = 1;
        int rows= 5;

        for(int i=1;i<=rows;i++) {

            //Task 1: Print Stars
            for (int j = 1; j <= stars; j++) {
                System.out.print(" * ");
                num++;
            }

            //Task 2: Print Newline
            System.out.println("");

        }
    }
}
```




38) Lab38.java

```
package com.jlcindia;
```

```
/*
```

38) Print the following Pattern

```
 *
 ***
 *****
 *****
```

```
*/
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
**/
```

```
public class Lab38{
```

```
    public static void main(String[] args) {
```

```
        int rows= 6; //Even and Odd Numbers
```

```
        int stars = 1;
```

```
        for(int i=1;i<=rows;i++) {
```

```
            //Task 1: Print Stars
```

```
            for (int k = 1; k<= stars; k++) {
```

```
                System.out.print(" * ");
```

```
            }
```

```
            //Task 2: Print Newline
```

```
            System.out.println("");
```

```
            stars = stars + 2;
```

```
        }
```

```
    }
```

```
}
```



39) Lab39.java

```
package com.jlcindia;
```

```
/*
```

39) Print the following Pattern

```
*
* * *
* * * * *
* * * * * *
* * * * *
* * *
*
```

```
*/
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab39{
    public static void main(String[] args) {

        int stars = 1;
        int rows= 7; //Odd Numbers

        for(int i=1;i<=rows/2+1;i++) {

            //Task 1: Print Stars
            for (int j = 1; j <= stars; j++) {
                System.out.print(" * ");
            }

            //Task 2: Print Newline
            System.out.println("");

            stars++;
        }
    }
}
```

```

stars = rows/2;

for(int i=1;i<=rows/2;i++) {

    //Task 1: Print Stars
    for (int j = 1; j <= stars; j++) {
        System.out.print(" * ");
    }

    //Task 2: Print Newline
    System.out.println("");

    stars--;
}

}
}

```

40) Lab40.java

```

package com.jlcindia;
/*
40) Print the following Pattern
    *
   ***
  *****
 *****

*/

/*
* @Author : Srinivas Dande
* @Company: Java Learning Center
**/
public class Lab40{

```



```
public static void main(String[] args) {  
  
    int rows= 6; //Even and Odd Numbers  
    int stars = 1;  
    int spaces = 15;  
  
    for(int i=1;i<=rows;i++) {  
  
        //Task 1: Print Spaces  
        for (int j = 1; j <= spaces; j++) {  
            System.out.print(" ");  
        }  
  
        //Task 2: Print Stars  
        for (int k = 1; k<= stars; k++) {  
            System.out.print(" * ");  
        }  
  
        //Task 3: Print Newline  
        System.out.println("");  
  
        spaces = spaces - 2;  
        stars = stars + 2;  
  
    }  
  
}
```



41) Lab41.java

```
package com.jlcindia;
```

```
/*
```

41) Print the following Pattern

```
      *
     ***
    *****
   *********
  **********
 ***
  ***
   *
```

```
*/
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
**/
```

```
public class Lab41{
```

```
    public static void main(String[] args) {
```

```
        int rows= 6; //Even and Odd Numbers
```

```
        int stars = 1;
```

```
        int spaces = 25;
```

```
        for(int i=1;i<=rows/2+1;i++) {
```

```
            //Task 1: Print Spaces
```

```
                for (int j = 1; j <= spaces; j++) {
```

```
                    System.out.print(" ");
```

```
                }
```

```
            //Task 2: Print Stars
```

```
                for (int k = 1; k<= stars; k++) {
```

```
                    System.out.print(" * ");
```

```
                }
```



```
//Task 3: Print Newline
    System.out.println("");

    spaces = spaces - 2;
    stars = stars + 2;

}

for(int i=1;i<=rows;i++) {

    //Task 1: Print Spaces
        for (int j = 1; j <= spaces; j++) {
            System.out.print(" ");
        }

    //Task 2: Print Stars
        for (int k = 1; k<= stars; k++) {
            System.out.print(" * ");
        }

    //Task 3: Print Newline
        System.out.println("");

        spaces = spaces + 2;
        stars = stars - 2;

}

}
}
```

DSA

Module 3

Bit Manipulation

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

3. Bitwise Operators

- ◆ These Operators will be used to perform the operation on individual bits of the value.

Bitwise Operators	Description
~	Bitwise NOT
&	Bitwise AND
	Bitwise OR
^	Exclusive OR (XOR)
<<	Left Shift
>>	Right Shift
>>>	Unsigned Right Shift

- ◆ Operands for the following Bitwise Operators can be of boolean type or Integer type:
 - Bitwise AND
 - Bitwise OR
 - Exclusive OR (XOR)
- ◆ Operands for the following Bitwise Operators will be of Integer type only:
 - Bitwise NOT
 - Left Shift
 - Right Shift
 - Unsigned Right Shift

1's Complement

- ◆ 1's complement of a binary number is formed by changing 1's to 0's and 0's to 1's

0	0	1	0	1	1	0	1	← Actual Bits
1	1	0	1	0	0	1	0	← 1's Complement

2's Complement

- ♦ 2's complement of a binary number is formed by adding 1 to the least significant bit of 1's complement.

0	0	1	0	1	1	0	1	Actual Bits
1	1	0	1	0	0	1	0	1's Complement
							1	Adding 1 with value
1	1	0	1	0	0	1	1	2's Complement

3.1. Exploring Bitwise AND

- ♦ Bitwise AND returns 1 when both the bits are 1
- ♦ Bitwise AND returns 0 when any one bit is 0.
- ♦ **Truth Table of Bitwise AND**

A	B	A & B
1	1	1
1	0	0
0	1	0
0	0	0

Lab1.java

```

package com.jlccindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {
    public static void main(String[] args) {
        int a= 5;
        int b= 7;
        int result= a & b;
        System.out.println(result);
    }
}
    
```

3.2. Exploring Bitwise OR

- ◆ Bitwise OR returns 0 when both the bits are 0
- ◆ Bitwise OR returns 1 when any one bit is 1.
- ◆ **Truth Table of Bitwise OR**

A	B	A B
1	1	1
1	0	1
0	1	1
0	0	0

Lab2.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {
    public static void main(String[] args) {
        int a= 5;
        int b= 7;

        int result= a | b;

        System.out.println(result);
    }
}
```

3.3. Exploring Bitwise XOR

- ◆ Bitwise XOR returns 0 when both the bits are same
- ◆ Bitwise XOR returns 1 when both the bits are different.
- ◆ **Truth Table of Bitwise XOR**

A	B	A ^ B
1	1	0
1	0	1
0	1	1
0	0	0

- ◆ **Imp Points to Remember with Bitwise XOR**

1. $(a \wedge b) \wedge c = a \wedge (b \wedge c)$
2. $a \wedge 0 = a$
3. $a \wedge a = 0$
4. $a \wedge b = b \wedge a$

Lab3.java

```
package com.jlccindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab3 {
    public static void main(String[] args) {
        int a= 5;
        int b= 7;

        int result= a ^ b;

        System.out.println(result);
    }
}
```



Lab4.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab4 {
    public static void main(String[] args) {

        int p= 5;

        System.out.println(p ^ p); //0
        System.out.println(p ^ 0); //p
        System.out.println(0 ^ p); //p

        int a=9;
        int b= 13;
        int c= 25;

        int x= a^b;
        int y= b^a;

        System.out.println(x);
        System.out.println(y);

        x= (a^b)^c;
        y= a^(b^c);

        System.out.println(x);
        System.out.println(y);

    }
}
```

3.4. Exploring Bitwise NOT

- ◆ Bitwise NOT returns 0 when the Bit is 1
- ◆ Bitwise NOT returns 1 when the Bit is 0.
- ◆ **Truth Table of Bitwise NOT**

A	~A
1	0
0	1

Lab5.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5 {

    public static void main(String[] args) {

        int a=8;
        int b= -7;

        int result1= ~a;
        int result2= ~b;

        System.out.println(result1);
        System.out.println(result2);

    }
}
```

3.5. Exploring Bitwise LeftShift

- ◆ Bitwise LeftShift Operator shift the Bits towards leftside for Specified number of Times
- ◆ When Bits are Shifted to Left Once then It is Equals to multiplying the number by 2

Lab6.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab6 {

    public static void main(String[] args) {

        int a=5;

        System.out.println(a<<1);
        System.out.println(a<<2);
        System.out.println(a<<3);
        System.out.println(a<<4);

    }

}
```

3.6. Exploring Bitwise Rightshift

- ◆ Bitwise RightShift Operator shift the Bits towards rightside for Specified number of Times
- ◆ When Bits are Shifted to Right Once then It is Equals to dividing the number by 2
- ◆ RightShift also called as Signed RightShift
- ◆ Signed RightShift holds the Sign i.e
 - If the Number is Positive then fills with **Zero**
 - If the Number is Negative then fills with **One**

Lab7.java

```
package com.jlccindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab7 {

    public static void main(String[] args) {

        int a=25;

        System.out.println(a>>1);
        System.out.println(a>>2);

        int b=-25;

        System.out.println(b>>1);
        System.out.println(b>>2);

    }

}
```


3.7. Exploring Bitwise Unsigned Rightshift

- ◆ Bitwise Unsigned RightShift Operator shift the Bits towards rightside for Specified number of Times
- ◆ When Bits are Shifted to Right Once then It is Equals to dividing the number by 2
- ◆ Unsigned Signed RightShift Ignores the Sign i.e
 - If the Number is Positive then fills with **Zero**
 - If the Number is Negative then also fills with **Zero**

Lab8.java

```
package com.jlccindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab8 {

    public static void main(String[] args) {

        int a=5;

        System.out.println(a>>>1);
        System.out.println(a>>>2);

        int b=-5;

        System.out.println(b>>>1);
        System.out.println(b>>>2);

    }

}
```



3.8. Problems on Bitwise

Lab9.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Check whether Last bit of given number is Set or Not
public class Lab9 {
    public static void main(String[] args) {
        int n=4;
        if((n & 1) != 0)
            System.out.println("Last Bit is Set");
        else
            System.out.println("Last Bit is Not Set");
    }
}
```

Lab10.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Check whether given number is Odd or Even
//If Last Bit is Set then It is Odd
//If Last Bit is Un-Set then It is Even
public class Lab10 {
    public static void main(String[] args) {
        int n=6;
        if((n&1) !=0)
            System.out.println("Odd");
        else
            System.out.println("Even");
    }
}
```



Lab11.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Check whether Kth bit of given number is Set or Not

public class Lab11 {

    public static void main(String[] args) {

        int n=15;
        int k=5;

        for(int i=1;i<=k-1;i++)
            n= n >> 1;  //Shifting One time

        if((n & 1) != 0)
            System.out.println("Kth Bit is Set");
        else
            System.out.println("Kth Bit is Not Set");

    }

}

// Time Complexity => O(n)
```



Lab11A.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Check whether Kth bit of given number is Set or Not

public class Lab11A {

    public static void main(String[] args) {

        int n=15;
        int k=5;

        n = n >> k-1; //Shifting k-1 times

        if((n & 1) != 0)
            System.out.println("Kth Bit is Set");
        else
            System.out.println("Kth Bit is Not Set");

    }

}

// Time Complexity=> O(1)
```



Lab12.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Count the Number of Set Bits in given number
// Visiting All the Bits and Counting only Set Bits

public class Lab12 {
    public static void main(String[] args) {

        int n = 1024;
        int count = 0;
        int loopCount = 0;

        while (n > 0) {
            loopCount++;
            if ((n & 1) != 0)
                count++;

            n = n >> 1; // n = n/2;
        }

        System.out.println("Loop Count : " + loopCount);
        System.out.println("Set Bit Count : " + count);
    }
}

// Time Complexity => O(100) => O(1)
```



Lab12A.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Count the Number of Set Bits in given number
//Brain Kerningam's Algorithm
//Visit and Count only Set Bits

public class Lab12A {
    public static void main(String[] args) {

        int n = 40;
        int count = 0;

        while (n > 0) {

            n = n & (n-1);
            count++;

        }

        System.out.println("Set Bit Count : " + count);
    }
}

// Time Complexity => O(1)
```



Lab13.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Find whether Given Number is Power of 2 or Not
// Number of Set Bits are 1

public class Lab13 {
    public static void main(String[] args) {

        int n = 63;
        int count = 0;

        while (n > 0) {
            n = n & (n-1);
            count++;
        }

        System.out.println("Set Bit Count : " + count);

        if(count==1)
            System.out.println("2 Power ");
        else
            System.out.println("Not 2 Power ");

    }
}

// Time Complexity => O(n) /O(1)
```



Lab13A.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Find whether Given Number is Power of 2 or Not
// Number of Set Bits are 1

public class Lab13A {
    public static void main(String[] args) {

        int n = 64;

        if( ( n & (n-1) ) == 0)
            System.out.println("2 Power ");
        else
            System.out.println("Not 2 Power ");

    }
}

// Time Complexity => O(1)
```




Lab14.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Reverse the Bits of given Number

public class Lab14 {
    public static void main(String[] args) {

        int n = 15;
        int rev = 0;

        while (n > 0) {
            rev = rev << 1;
            if( (n & 1) == 1){
                rev = rev ^ 1;
            }
            n = n >> 1;
        }

        System.out.println(rev);
    }
}

// Time Complexity => O(1)
```



Lab15.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Add two Numbers without + Operator

public class Lab15 {
    public static void main(String[] args) {

        int a = 71;
        int b = 51;

        while (b > 0) {
            int temp = (a & b) <<1;
            a = a^b;
            b = temp;
        }

        System.out.println(a);
    }
}

// Time Complexity => O(1)
```



Lab16.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Find the One Odd Occurring Number in Array

public class Lab16 {
    public static void main(String[] args) {

        int arr [] = {3,5,7,3,7,5,9,9,5};
        int length = arr.length; //9

        for(int i=0; i<length; i++) {
            int count = 0;
            for(int j=0;j<length;j++) {
                if(arr[i]==arr[j]) {
                    count++;
                }
            } //Inner For loop

            if(count%2!=0) {
                System.out.println(arr[i]);
                break;
            }
        } //Outer For loop
    }

    // Time Complexity => O(n2)
}
```



Lab16A.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Find the One Odd Occurring Number in Array

public class Lab16A {
    public static void main(String[] args) {

        int arr [] = {3,5,7,3,7,5,9,9,5};
        int length = arr.length; //9

        int result = 0;
        for(int i=0; i<length; i++) {
            result = result ^ arr[i];
        }

        System.out.println(result);

    }
}

// Time Complexity => O(n)
```



Lab17.java

```
package com.jlccindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Find Two Odd Occurring Numbers in Array

public class Lab17 {
    public static void main(String[] args) {
        int arr[] = { 3, 5, 3, 7, 9, 7 };
        int length = arr.length; // 9

        int result = 0;
        for (int i = 0; i < length; i++) {
            result = result ^ arr[i];
        }

        int k = result & ~(result - 1);

        int first = 0;
        int second = 0;

        for (int i = 0; i < length; i++) {
            if ((arr[i] & k) == 0)
                first = first ^ arr[i];
            else
                second = second ^ arr[i];
        }
        System.out.println(first);
        System.out.println(second);
    }
}
// Time Complexity => O(n)
```



Lab18.java

```
package com.jlccindia;

import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Find the Occurrence of each element in the array
public class Lab18 {
    public static void main(String[] args) {

        int arr[] = { 3, 5, 6, 3, 7, 9, 7, 7, 6, 9 };
        int length = arr.length;

        Map<Integer, Integer> mymap = new HashMap<>();

        for (int i = 0; i < length; i++) {
            int count = 0;
            for (int j = 0; j < length; j++) {
                if (arr[i] == arr[j]) {
                    count++;
                }
            } // Inner For loop

            mymap.put(arr[i], count);

        } // Outer For loop

        System.out.println(mymap);
    }

    // Time Complexity => O(n2)
}
```

DSA

Module 4

Recursion

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

4. Recursion

- ♦ Calling the Method from itself is called as Recursion.
- ♦ When You call the Method Recursively, You must specify the Condition to Terminate the Recursive Calls. That Condition is Called as Base Consition or Base Case.

Recursion is the process of calling one method from itself until the given base case is met.

- ♦ You can solve every problem is two approaches
 - Iterative Approach
 - Recursive Approach
- ♦ Iterative Approach is best suited in some use-cases and Recursive Approach will be good in some use-cases.

Q) When We have to Recursion?

Ans:

- ♦ When You are able to divide the Problem into Sub-Problems and You are able to find the solution for Sub-Problem then Go for Recursion

Structure of Recursive Methods.

```
void show(){  
  //1 Base Case  
  //2. SubTask Logic  
  //3. Recursive Call  
}
```

```
void show(){  
  //1. Base Case  
  //2. Recursive Call  
  //3. SubTask Logic  
}
```

- ◆ Types of Recursion
 - Direct Recursion
 - InDirect Recursion

//Direct Recursion // In the below example, show() is calling itself directly. //That why Direct Recursion <pre> class Hello{ public static void main(String as[]){ show(); } static void show(){ // Some Code show(); } } </pre>	//Indirect Recursion // In the below example, m1() is calling m2() and m2() is calling m1(). //That why Indirect Recursion <pre> class Hello{ public static void main(String as[]){ m1(); } static void m1(){ // Some Code m2(); } static void m2(){ // Some Code m1(); } } </pre>
--	--

4.1. Understanding Method Calls

- ◆ When You have method calls then One Stack will be Created by the JVM internally.
- ◆ When You call the method then that Method call will be pushed into Stack.
- ◆ When the method execution is completed then that Method call will be popped from Stack
- ◆ See the below Example on Method Calls without Recursion.



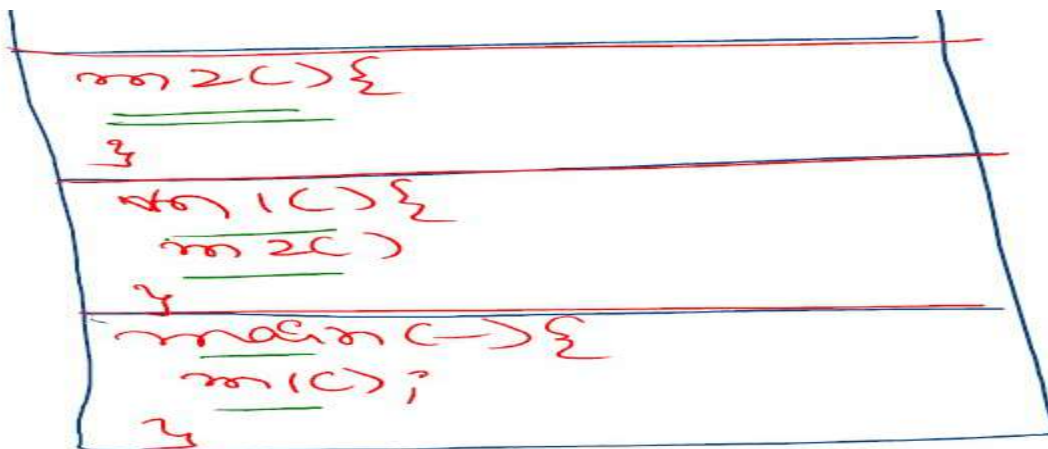
Lab1.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {

    public static void main(String[] args) {
        System.out.println("main - begin");
        m1();
        System.out.println("main - end");
    }

    static void m1() {
        System.out.println("m1 - begin");
        m2();
        System.out.println("m1 - end");
    }

    static void m2() {
        System.out.println("I am m2");
    }
}
```



4.2. Understanding Recursive Method Calls

- ◆ Every time the Recursive call is happened then that Method call will be pushed into Stack.
- ◆ When the method execution is completed then that Method call will be popped from Stack
- ◆ See the below Example on Method Calls with Recursion **without Basecase**.
- ◆ When Base case is not specified then **StackOverflowError** will be thrown at Runtime.

Lab2.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab2 {

    static void show() {
        System.out.println("Hello Guys");
        show();
    }

    public static void main(String[] args) {
        show();
    }
}
```

4.3. Understanding Recursive Method Calls with Basecase

- ◆ See the below Example on Method Calls with Recursion **with Basecase**.
- ◆ When Base case is specified then **Recursive Calls will be Terminated when the Basecase is met**.
- ◆ You can write the Basecase anywhere in the Recursive Method.
- ◆ You can write the Recursive Call anywhere in the Recursive Method.
- ◆ See Lab3, Lab4, Lab5



Lab3.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    static void show(int n) {
        if(n==0) return;
        System.out.println("Hello Guys");
        show(n-1);
    }
    public static void main(String[] args) {
        show(5);
    }
}
```

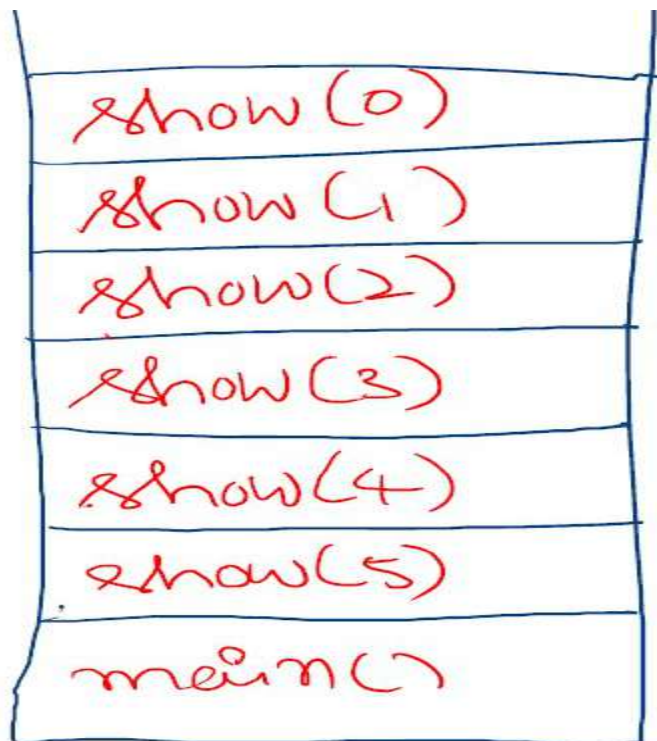
Lab4.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    static void show(int n) {
        if(n==0) return;
        show(n-1);
        System.out.println("Hai Guys");
    }
    public static void main(String[] args) {
        show(5);
    }
}
```



Lab5.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab5 {
    static void show(int n) {
        System.out.println("Welcome to JLC");
        if(n==0) return;
        show(n-1);
    }
    public static void main(String[] args) {
        show(5);
    }
}
```



4.4. Types of Recursion

- ◆ There are Two types of Recursion.
 - Tail Recursion
 - Non-Tail Recursion

Tail Recursion:

- ◆ When the Recursive Call is the Last Statement then It is Called as Tail Recursion
- ◆ Nothing to do after the Recursive Call.

Ex:

```
void show(){  
    //1 Base Case  
    //2. SubTask Logic  
    //3. Recursive Call => Last Statement  
}
```

Lab6.java

```
package com.jlcindia;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
//Print Numbers from N to 1  
public class Lab6 {  
    static void printNumbers(int n) {  
        if(n==0) return;  
        System.out.println(n);  
        printNumbers(n-1); // Recursive Call  
    }  
    public static void main(String[] args) {  
        int n=5;  
        printNumbers(n);  
    }  
}
```


Non-Tail Recursion:

- ♦ When the Recursive Call is Not the Last Statement then It is Called as Non-Tail Recursion
- ♦ You have Somthing to do after the Recursive Call

Ex:

```
void show(){  
    //1. Base Case  
    //2. Recursive Call => Not a Last Statement  
    //3. SubTask Logic  
}
```

Lab7.java

```
package com.jlcindia;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
//Print Numbers from 1 to N  
public class Lab7 {  
    static void printNumbers(int n) {  
        if(n==0) return;  
        printNumbers(n-1);  
        System.out.println(n);  
    }  
    public static void main(String[] args) {  
        int n=5;  
        printNumbers(n);  
    }  
}
```


Q) Which is Better – Tail or Non- Tail Recursion?

Ans:

- ♦ Choose Tail Recursion if Possible
- ♦ In the case of Tail Recursion, Code will be Optimized by the Compiler
- ♦ So Tail Recursion faster than Non-Tail Recursion

Q) Can I Use Tail Recursion Always?

Ans:

- ♦ Not Possible ,
- ♦ Because that Depends on the Problem You are solving

Ex :

- Quick Sort can use Tail Recursion
- Merge Sort needs Non-Tail Recursion only

Q) How the Compiler Optimizing the Code with Tail Recursion?

Ans:

// Your Code	// Optimized Code
<pre>static void printNumbers(int n) { if(n==0) return; System.out.println(n); printNumbers(n-1); }</pre>	<pre>static void printNumbers(int n) { JLC: if(n==0) return; System.out.println(n); n = n-1; goto JLC; }</pre> Now No Recursion in the Optimized Code



4.5. Problems on Bitwise

Lab8.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Find the Sum of Numbers from 1 to N

public class Lab8 {
    static int sum(int n) {

        if(n==0)
            return 0;
        else
            return n + sum(n-1);
    }
    public static void main(String[] args) {
        int n=5;
        int sum = sum(n);
        System.out.println(sum);
    }
}

//Non-Tail Recursive
//Time Complexity : O(n)
//Aux Space Complexity : O(1)
```



Lab9.java

```
package com.jlccindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

/Find the Factorial of given number

public class Lab9 {

    static int fact(int n) {

        if(n==0 || n==1)
            return 1;
        else
            return n * fact(n-1);
    }

    public static void main(String[] args) {
        int n=1;
        int f = fact(n);
        System.out.println(f);
    }
}

// Non-Tail Recursive
// Time Complexity : O(n)
//Aux Space : O(1)
```



Lab10.java

```
package com.jlcindia;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Find the Sum of Individual Digits of given number
// n= 135 => 1+ 3 + 5 => 9

public class Lab10 {

    static int digitSum(int n) {

        if(n==0)
            return 0;
        else
            return n%10 + digitSum(n/10);
    }

    public static void main(String[] args) {
        int n=135;
        int sum = digitSum(n);
        System.out.println(sum);
    }
}

// Non-Tail Recursive
// Time Complexity : O(n)
//Aux Space : O(1)
```



Java Learning Center

No.1 In Java Training & placement

DSA

Module 5

Arrays

Author

Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

5. Arrays

- ◆ Array is a collection of elements which are of same data type.
- ◆ Array is also called as Homogeneous data structure.
- ◆ Elements of an array will be stored in contiguous memory locations.
- ◆ Arrays are objects in Java.
- ◆ Three tasks to remember while you are working with arrays:
 - Array Declaration
 - Array Construction
 - Array Initialization
- ◆ Arrays can be constructed with multiple dimensions i.e 1-D Array, 2-D Array etc.

5.1. Single Dimensional Arrays

- ◆ Single Dimensional array can be called as 1-D Array.

5.1.1. Array Declaration and Construction

- ◆ You can declare and construct an array in one statement also.

Syntax:

```
<dataType> <refVarName>[] = new <dataType>[<size>];
<dataType> []<refVarName> = new <dataType>[<size>];
```

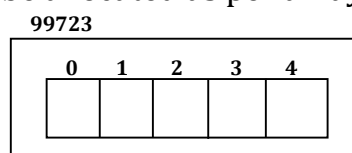
Ex:

```
int arr[]=new int[5];
int []arr=new int[5];
```

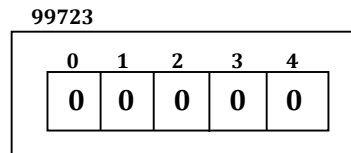
- 1) Allocates 8 bytes of memory for the reference variable and initializes with null value.



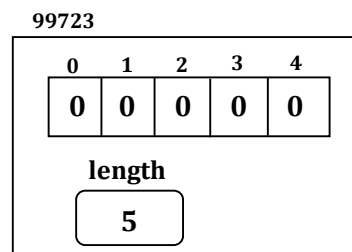
- 2) Memory blocks will be allocated as per array size.



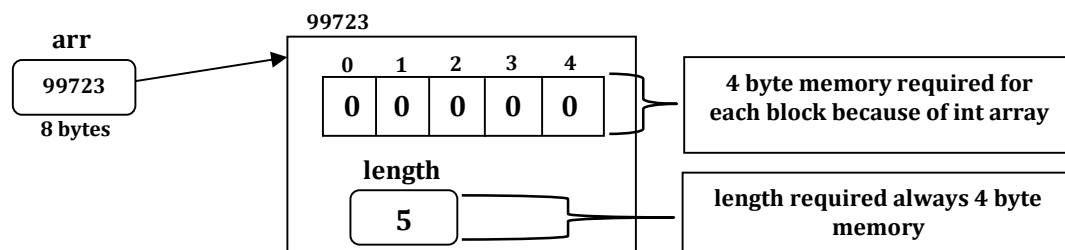
3) Memory blocks will be initialized with default value as per array data type.



4) Memory will be allocated for length variable and will be initialized with size of an array.

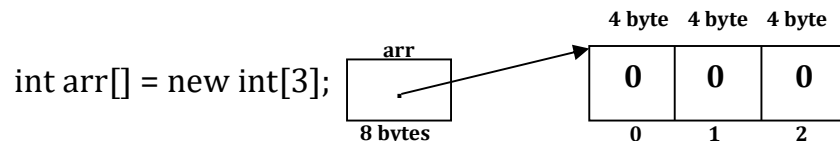


5) Array object address will be assigned to reference variable.



5.1.2. Array Initialization

- Once array is constructed then array elements will be initialized with default values as per array data type.
- You can initialize array elements with your own value using index representation.



- Here we are creating an array with size 3, so only three elements will be stored in this array.

`arr[0]` -> Represents the first element which contains value 0.

`arr[1]` -> Represents the second element which contains value 0.

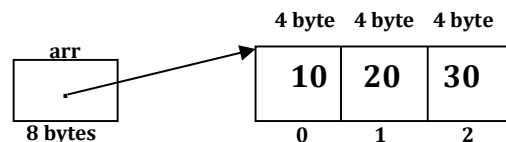
`arr[2]` -> Represents the third element which contains value 0.

- You can initialize the array elements as follows:

`arr[0] = 10;`

`arr[1] = 20;`

`arr[2] = 30;`



- Accessing the elements after initialization

`arr[0]` -> Represents the first element which contains value 10.

`arr[1]` -> Represents the second element which contains value 20.

`arr[2]` -> Represents the third element which contains value 30.



Lab1.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static void main(String[] args) {

        // 1. Create Array
        int arr[] = new int[5];

        // 2.find the Length of Array
        int n = arr.length;
        System.out.println(n);

        // 3.Access Elements of Array
        for (int i = 0; i < n; i++)
            System.out.println(arr[i]);

        // 4. Initialize the Array
        arr[0] = 11;
        arr[2] = 33;
        arr[4] = 55;
        System.out.println("-----");

        // Access Elements of Array
        for (int i = 0; i < n; i++)
            System.out.println(arr[i]);
    }
}
```

5.1.3 Array Declaration, Construction and Initialization

- ♦ You can declare, construct and initialize an array in one statement also.

Syntax:

```
<dataType> <refVarName>[] = {<val1>,<val2>,<val3>,...,<valN>};
```

Ex:

```
int arr[]={10,20,30};  
String names[]{"Hello","Srinivas","Dande"};
```

Lab2.java

```
package com.jlcindia.arrays;  
  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
  
public class Lab2{  
    public static void main(String[] args) {  
  
        int arr[] = {10,20,30,40,50};  
  
        int n= arr.length;  
  
        // Access Elements of Array  
        for (int i = 0; i < n; i++)  
            System.out.println(arr[i]);  
    }  
}
```

5.1.4. Why and Why Not Arrays?

- ♦ **Limitations with array:**

- ✓ Array is a homogeneous data structure i.e only one type of elements can be stored in the Array.
- ✓ Array is static in nature i.e size of an array cannot be modified.
- ✓ Insert and delete operations require more shifting of elements.
- ✓ You need to write the Code for doing any Operations on Arrays. No Built-In Funcatioality provided in Arrays.

- ♦ **Advantages with array:**

- ✓ Arrays are faster in Accessing the Elements.
- ✓ Arrays are Cache-Friendly



5.1.5 Dynamic Size Array - ArrayList

- ♦ ArrayList is implemented as a resizable array.
- ♦ ArrayList elements will be stored internally using indexing notation.
- ♦ This is one of the most widely used concrete class.
- ♦ It is fast to access the elements, but slow to insert and delete the elements.
- ♦ Size of ArrayList will be growable dynamically.
- ♦ Default Capacity is 10. i.e When You create ArrayList, It allocates the Space for 10 Elements in advance.
- ♦ When you add add elements then size will be increased.
- ♦ Following the Formula for Capacity Calculation.

In Java 6

$$\text{newCapacity} = (\text{oldCapacity} * 3) / 2 + 1;$$

In Java 7

$$\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} >> 1)$$

- ♦ ArrayList comes with many built-methods as follows.
 - `int size()`
 - `boolean isEmpty()`
 - `Object get(int idx)`
 - `boolean add(Object obj)`
 - `void add(int idx, Object obj)`
 - `boolean remove(Object obj)`
 - `Object remove(int idx)`
 - `Object set(int idx, Object obj)`
 - `boolean contains(Object obj)`



Lab3.java

```
package com.jlcindia.arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.ArrayList;
import java.util.List;

public class Lab3{
    public static void main(String[] args) {

        List<Integer> mylist = new ArrayList<>();

        mylist.add(10);
        mylist.add(20);
        mylist.add(30);
        mylist.add(40);

        System.out.println(mylist);
        mylist.add(50);
        System.out.println(mylist);
        mylist.remove(Integer.valueOf(40));
        System.out.println(mylist);
        //ArrayList has many Built-In methods to Use
    }
}
```



5.2. Operations on Arrays

Lab4.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Search for given Element in the Unsorted Array

public class Lab4 {
    static int linearSearch(int arr[], int element) {
        int n = arr.length;

        for (int i = 0; i < n; i++) {
            if (arr[i] == element) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int arr[] = { 25, 10, 5, 15, 30, 20, 50 };
        int element = 25;

        int index = linearSearch(arr, element);

        System.out.println(index);
        if (index == -1)
            System.out.println("Element Not Found");
        else
            System.out.println("Element Found at Index : " + index);
    }
}

// Time Complexity - - L.S O(n)
// Aux Space - O(1)
```



Lab5A.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Insert the Element at given position in an Array
// If Capacity is not there Dont Insert

public class Lab5A {

    static int insert(int arr[], int element,int position) {

        int n = arr.length;
        int index = position - 1;

        for (int i = n-1; i > index; i--) {
            arr[i] = arr[i-1];
        }

        arr[index] = element;

        return n+1;
    }

    public static void main(String[] args) {

        int arr[] = new int[5];
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;

        int element = 99;
        int position = 3;
```




```
        for(int i=0;i<arr.length;i++) {
            System.out.print(arr[i]+" ");
        }

        insert(arr, element,position);

        System.out.println("----");
        for(int i=0;i<arr.length;i++) {
            System.out.print(arr[i]+" ");
        }
    }
}
```

// Time Complexity - - O(n)

// Aux Space - O(1)

Lab5B.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```

//Insert the Element at given position in an Array
// Create New Array and Insert

```
public class Lab5B {

    static int[] insert(int arr[], int element, int position) {

        int index = position - 1;
        int tempArr[] = new int[arr.length + 1];
        int n = tempArr.length;
```



```
        for (int i = n - 1; i > index; i--) {
            tempArr[i] = arr[i - 1];
        }

        // Insert
        tempArr[index] = element;

        for (int i = 0; i < index; i++) {
            tempArr[i] = arr[i];
        }
        return tempArr;
    }

    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50};
        int element = 99;
        int position = 3;

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

        System.out.println();

        int newArr[] = insert(arr, element, position);

        System.out.println("----");
        for (int i = 0; i < newArr.length; i++) {
            System.out.print(newArr[i] + " ");
        }
    }
}

// Time Complexity - - O(n)
// Aux Space - O(n)
```



Lab6.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Delete the Element at given position in an Array

public class Lab6 {

    static void deleteAt(int arr[], int position) {

        int index = position - 1;
        int n = arr.length;

        if(position>n) {
            return ;
        }

        for (int i = index; i < n-1; i++) {
            arr[i] = arr[i + 1];
        }

        arr[n-1] = 0;
    }

    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50};
        int position = 5;

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
}
```



```
deleteAt(arr, position);
```

```
System.out.println("----");  
for (int i = 0; i < arr.length; i++) {  
    System.out.print(arr[i] + " ");  
}  
  
}  
}
```

```
// Time Complexity - - O(n)
```

```
// Aux Space - O(1)
```



Lab7.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Delete the given Element from an Array

public class Lab7 {

    static void deleteElement(int arr[], int element) {

        int n = arr.length;
        int index=-1;

        for(int i=0;i<n;i++) {
            if(arr[i]==element) {
                index=i;
                break;
            }
        }

        if(index!=-1) {
            return ;
        }

        for (int i = index; i < n-1; i++) {
            arr[i] = arr[i + 1];
        }

        arr[n-1] = 0;

    }
}
```



```
public static void main(String[] args) {  
  
    int arr[] = {10,20,30,40,50};  
    int element = 55;  
  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i] + " ");  
    }  
  
    System.out.println();  
  
    deleteElement(arr, element);  
  
    System.out.println("----");  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i] + " ");  
    }  
}  
}
```

// Time Complexity - - $O(n)$

// Aux Space - $O(1)$



Lab8.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Access the Elements of an Array

public class Lab8 {

    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50};

        System.out.println(arr[0]);
        System.out.println(arr[1]);
        System.out.println(arr[2]);
        System.out.println(arr[3]);

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
            //Do something with Element
        }

    }
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```

5.3. Time Complexity of Array Operations

Operation	Time Complexity
Access One Element	$O(1)$
Access All Elements	$O(n)$
Update One Element	$O(1)$
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Insert Element at nth postion	$O(n)$
Insert Element at First postion	$O(n)$
Insert Element at Last postion	$O(1)$
Delete Element at nth postion	$O(n)$
Delete Element at First postion	$O(n)$
Delete Element at Last postion	$O(1)$



5.4. Problems on Arrays

Problem1A.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Reverse the given Array

public class Problem1A {
    static void reverse(int arr[]) {

        int n = arr.length;
        int start = 0;
        int end = n - 1;

        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;

            start++;
            end--;
        }
    }

    public static void main(String[] args) {

        int arr[] = { 10, 20, 30, 40, 50 };

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println("\n ");

        reverse(arr);
    }
}
```



```
        for (int i = 0; i < arr.length; i++) {  
            System.out.print(arr[i] + " ");  
        }  
    }  
}
```

// Time Complexity - - $O(n/2)$ / $O(n)$

// Aux Space - $O(1)$

Problem1B.java

```
package com.jlcindia.arrays;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
//Reverse the given Array  
  
public class Problem1B {  
  
    static void reverse(int arr[]) {  
        int n = arr.length;  
  
        for (int i=0,j=n-1;i < j;i++,j--) {  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
    public static void main(String[] args) {  
        //Same as Problem1 main() method  
    }  
}
```

// Time Complexity - - $O(n/2)$

// Aux Space - $O(1)$



Problem2.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Remove the Duplicates from Sorted Array

public class Problem2 {

    static int removeDuplicates(int arr[]) {
        int n = arr.length;
        int temp = 1;

        for (int i = 1; i < n; i++) {

            if (arr[i] != arr[temp - 1]) {
                arr[temp] = arr[i];
                temp++;
            }
            arr[i]=0;
        }
        return temp;
    }

    public static void main(String[] args) {

        int arr[] = { 0,0,10, 10, 20, 20, 20, 30, 40, 40, 40 };

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

        int tempCount = removeDuplicates(arr);
    }
}
```



```
System.out.println(" \n ");

for (int i = 0; i < arr.length; i++) {
    System.out.print(arr[i] + " ");
}

System.out.println(" \n ");

for (int i = 0; i < tempCount; i++) {
    System.out.print(arr[i] + " ");
}
}
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem3.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Move All Zeros to End

public class Problem3 {

    static int moveZeros(int arr[]) {

        int n = arr.length;
        int temp=0;

        for(int i=0;i<n;i++) {
            if(arr[i]!=0) { //Non-Zero values

                int x = arr[i];
                arr[i]=arr[temp];
                arr[temp]=x;

                temp++;
            }
        }
        return temp;
    }

    public static void main(String[] args) {

        int arr[] = { 10,15,0,0,25,0,20 };
        //int arr[] = { 10,15,25,20,0,0,0 };

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```



```
int nonZeroCount = moveZeros(arr);
```

```
System.out.println("\n ");
```

```
//Print All
```

```
for (int i = 0; i < arr.length; i++) {  
    System.out.print(arr[i] + " ");  
}
```

```
System.out.println("\n ");
```

```
//Print Non-Zeors
```

```
for (int i = 0; i < nonZeroCount; i++) {  
    System.out.print(arr[i] + " ");  
}
```

```
}
```

```
}
```

```
// Time Complexity - - O(n)
```

```
// Aux Space - O(1)
```



Problem4.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Find whether the Array is Sorted in ASC or Not

public class Problem4 {
    static boolean isSorted(int arr[]) {

        int n = arr.length;

        for(int i=1;i<n;i++) {
            if(arr[i-1]>arr[i]) {
                return false;
            }
        }
        return true;
    }
    public static void main(String[] args) {

        int arr[] = { 10,20,30,40,50 };
        // int arr[] = { 10,20,30,50,40 };

        boolean flag = isSorted(arr);

        if(flag)
            System.out.println("Yes Sorted");
        else
            System.out.println("Not Sorted");
    }
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem5.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Rotate the Elements Left side 1 time.
public class Problem5 {
    static void leftRotateOne(int arr[]) {

        int n = arr.length;
        int temp = arr[0];

        for (int i = 1; i < n; i++) {
            arr[i - 1] = arr[i];
        }
        arr[n - 1] = temp;
    }
    public static void main(String[] args) {
        int arr[] = { 1, 2, 3, 4, 5 };

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

        leftRotateOne(arr);

        System.out.println("\n ");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```




Problem6A.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Rotate the Elements Left side K times.

public class Problem6A {
    static void leftRotateOne(int arr[]) {

        int n = arr.length;
        int temp = arr[0];

        for (int i = 1; i < n; i++) {
            arr[i - 1] = arr[i];
        }

        arr[n - 1] = temp;
    }

    static void leftRotate(int arr[], int k) {

        for (int i = 1; i <= k; i++) {
            leftRotateOne(arr);
        }
    }

    public static void main(String[] args) {

        int arr[] = { 1, 2, 3, 4, 5 };
        int k = 3; //

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```



```
leftRotate(arr,k);
```

```
System.out.println("\n ");
```

```
for (int i = 0; i < arr.length; i++) {  
    System.out.print(arr[i] + " ");  
}
```

```
}
```

```
}
```

```
// Time Complexity - -  $O(kn) \Rightarrow O(n)$ 
```

```
// Aux Space -  $O(1)$ 
```

Problem6B.java

```
package com.jlcindia.arrays;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
// Rotate the Elements Left side K times.
```

```
public class Problem6B {
```

```
    static void reverse(int arr[],int start,int end) {
```

```
        while (start < end) {
```

```
            int temp = arr[start];
```

```
            arr[start] = arr[end];
```

```
            arr[end] = temp;
```

```
            start++;
```

```
            end--;
```

```
        }
```

```
    }
```



```
static void leftRotate(int arr[], int k) {  
  
    int n= arr.length;  
  
    reverse(arr,0,k-1);  
    reverse(arr,k,n-1);  
    reverse(arr,0,n-1);  
  
}  
  
public static void main(String[] args) {  
  
    int arr[] = { 1, 2, 3, 4, 5};  
    int k = 5; // k<=n  
  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i] + " ");  
    }  
  
    leftRotate(arr,k);  
  
    System.out.println("\n ");  
  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i] + " ");  
    }  
  
}  
}  
  
// Time Complexity - - O(kn) => O(n)  
// Aux Space - O(1)
```

Note: Remaining Array Problems will be discussed in LeetCode Sessions



Java Learning Center

No.1 In Java Training & placement

DSA

Module 6

Matrix

Author

Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

6. Multi Dimensional Arrays

- ♦ Multi Dimensional Array is an Array of Arrays.

Single Dimensional Array	Collection of Actual value
Two Dimensional Array	Collection of Single Dimensional Arrays
Three Dimensional Array	Collection of 2-D Arrays

6.1. Two Dimensional Arrays

- ♦ Two Dimensional Arrays can be called as 2-D array.
- ♦ 2-D array is Array of 1-D Arrays.
- ♦ 2-D Array can be called as Matrix.
- ♦ **A matrix is a collection of numbers arranged into a fixed number of rows and columns.**

6.2. 2-D Array Declaration and Construction

Syntax:

```
<dataType> <refVarName>[][] = new <dataType>[<size1>][<size2>];  
<dataType>[][] <refVarName> = new <dataType>[<size1>][<size2>];
```

Here

<size1> represents number of arrays to construct.

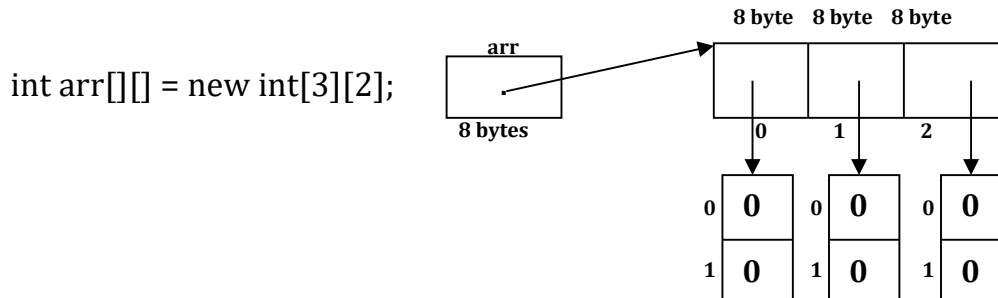
<size2> represents number of elements required for each array.

Ex:

```
int arr[][] = new int[3][2];  
int[][] arr = new int[3][2];
```

6.3. 2-D Array Initialization

- You can initialize array elements with your own value using index representation.



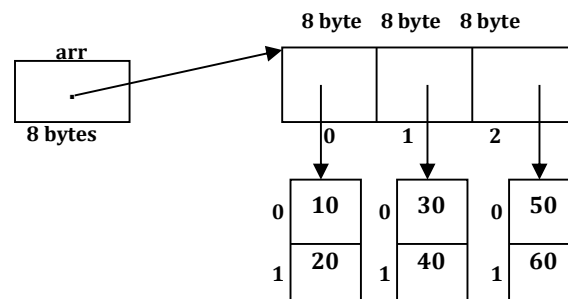
- Here we are creating 3 arrays with size 2, and these three arrays addresses will be stored in main array.

`arr[0][0]` -> Represents the 1st element of 1st array which contains value 0.
`arr[0][1]` -> Represents the 2nd element of 1st array which contains value 0.
`arr[1][0]` -> Represents the 1st element of 2nd array which contains value 0.
`arr[1][1]` -> Represents the 2nd element of 2nd array which contains value 0.
`arr[2][0]` -> Represents the 1st element of 3rd array which contains value 0.
`arr[2][1]` -> Represents the 2nd element of 3rd array which contains value 0.

- You can initialize the array elements as follows:

```

arr[0][0]    =10;
arr[0][1]    =20;
arr[1][0]    =30;
arr[1][1]    =40;
arr[2][0]    =50;
arr[2][1]    =60;
  
```



- ♦ **Accessing the elements after initialization**

arr[0][0] -> Represents the 1st element of 1st array which contains value 10.
arr[0][1] -> Represents the 2nd element of 1st array which contains value 20.
arr[1][0] -> Represents the 1st element of 2nd array which contains value 30.
arr[1][1] -> Represents the 2nd element of 2nd array which contains value 40.
arr[2][0] -> Represents the 1st element of 3rd array which contains value 50.
arr[2][1] -> Represents the 2nd element of 3rd array which contains value 60.

6.4. Jagged Arrays

Ex:

```
int arr[][] = new int[3][]; //Second size is Optional
```

```
arr[0]= new int[3];
```

```
arr[1]= new int[4];
```

```
arr[2]= new int[5];
```

- ♦ Now You can 12 Elements in the Array
- ♦ This Array is called as **Jagged Array**.

6.5. Array Declaration, Construction and Initialization

Syntax:

```
<dataType> <refVarName>[][] = {{v1,v2,...},{v1,v2,...},{v1,v2,...},...};
```

Ex:

```
int arr[][]={{10,20},{30,40},{50,60}};
```

```
int arr[][]={{10,20,30},{40},{50,60,70,80},{90,100}};
```


6.6. Examples on 2-D Arrays

Lab1.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static void main(String[] args) {

        int arr[][] = new int[3][4];

        int rows = arr.length;
        System.out.println("Rows : "+rows);

        int r1size = arr[0].length;
        System.out.println("Row 1 Size : "+ r1size);

        int r2size = arr[1].length;
        System.out.println("Row 2 Size : "+ r2size);

        int r3size = arr[2].length;
        System.out.println("Row 3 Size : "+ r3size);

        for(int i=0;i<rows;i++) {
            System.out.println(arr[i]);
        }

    }
}
```



Lab2.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {
    public static void main(String[] args) {

        //1. Creating 2-D Array
        int arr[][] = new int[3][3];

        //2. Accessing Elements of 2-D
        for(int i=0;i<arr.length;i++) {
            for(int j=0;j<arr[i].length;j++) {
                System.out.print(arr[i][j]+"\\t");
            }
            System.out.println("");
        }

        //3.Initializing 1st Array/1st Row
        arr[0][0]=11;
        arr[0][1]=12;
        arr[0][2]=13;

        //4.Initializing 2nd Array/2nd Row
        arr[1][0]=21;
        arr[1][1]=22;
        arr[1][2]=23;

        //5.Initializing 3rd Array/3rd Row
        arr[2][0]=31;
        arr[2][1]=32;
        arr[2][2]=33;
```



```
//6. Accessing Elements of 2-D
for(int i=0;i<arr.length;i++) {
    for(int j=0;j<arr[i].length;j++) {
        System.out.print(arr[i][j]+"\\t");
    }
    System.out.println("");
}

}
```

Lab3.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    public static void main(String[] args) {
        // 1. Creating 2-D Jagged Array
        int arr[][] = new int[3][];

        // 2. Creating 1-D Arrays
        arr[0] = new int[3];
        arr[1] = new int[4];
        arr[2] = new int[5];

        // 3. Accessing Elements of 2-D
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                System.out.print(arr[i][j] + "\\t");
            }
            System.out.println("");
        }
        // 4.Initializing 2-D Array
        arr[0][0] = 11;
        arr[1][1] = 22;
```



```
arr[2][2] = 33;

arr[1][3] = 1;
arr[2][3] = 1;
arr[2][4] = 1;

// 5. Accessing Elements of 2-D
for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr[i].length; j++) {
        System.out.print(arr[i][j] + "\t");
    }
    System.out.println("");
}
}
```

Lab4.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    public static void main(String[] args) {
        // 1. Creating and Initializing 2-D Array
        int arr[][] = { {1,2,3},{4,5,6},{7,8,9} };

        // 2. Accessing Elements of 2-D
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                System.out.print(arr[i][j] + "\t");
            }
            System.out.println("");
        }
    }
}
```



Lab5.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5 {
    public static void main(String[] args) {

        // 1. Creating and Initializing 2-D Array
        int arr[][] = { { 1, 2, 3 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12, 13 }, { 14, 15, 16, 17, 18, 19 } };

        // 2. Accessing Elements of 2-D
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                System.out.print(arr[i][j] + "\t");
            }
            System.out.println("");
        }
    }
}
```



Lab6.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab6 {
    public static void main(String[] args) {

        // 1. Creating and Initializing 2-D Array
        int arr[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 },
                        { 13, 14, 15, 16 } };

        System.out.println("1st Array --->");

        // 2. Accessing 1st Array Elements
        for (int j = 0; j < arr[0].length; j++) {
            System.out.print(arr[0][j] + "\t");
        }

        System.out.println("\n \n 2nd Array --->");

        // 3. Accessing 2nd Array Elements
        for (int j = 0; j < arr[1].length; j++) {
            System.out.print(arr[1][j] + "\t");
        }

        System.out.println("\n \n 3rd Array --->");

        // 4. Accessing 3rd Array Elements
        for (int j = 0; j < arr[2].length; j++) {
            System.out.print(arr[2][j] + "\t");
        }

    }
}
```



6.7. Problems on Matrices

Problem1A.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem1- Print the Main Diagonal of n * n Matrix

public class Problem1A {
    public static void main(String[] args) {

        // int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
        int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        //Main Diagonal
        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                if (i == j)
                    System.out.print(mat[i][j] + "\t");
            }
        }
    }
}

// Time Complexity - -  $O(n^2)$ 
// Aux Space -  $O(1)$ 
```



Problem1B.java

```
package com.jlcindia.matrices;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */
```

//Problem1- Print the Main Diagonal of n * n Matrix

```
public class Problem1B {  
    public static void main(String[] args) {  
  
        //    int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };  
        int mat[][] = { { 1, 2, 3,4 }, { 5, 6,7,8 }, { 9,10,11,12 },{13,14,15,16} };  
  
        //Main Diagonal  
        for (int i = 0; i < mat.length; i++) {  
            System.out.print(mat[i][i] + "\t");  
        }  
    }  
}
```

// Time Complexity - - O(n)

// Aux Space - O(1)



Problem2.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```

//Problem2- Print the Secondary Diagonal of n * n Matrix

```
public class Problem2 {
    public static void main(String[] args) {

        // int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
        int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        int n = mat.length;

        //Main Diagonal
        for (int i = 0, j = n - 1; i < n; i++, j--) {
            System.out.print(mat[i][j] + "\t");
        }

    }
}
```

// Time Complexity - - O(n)

// Aux Space - O(1)



Problem3A.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem3- Sum of Diagonal elements of n * n Matrix

public class Problem3A {
    public static void main(String[] args) {

        // int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
        int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        int n = mat.length;
        int sum = 0;

        // Main Diagonal Sum
        for (int i = 0; i < n; i++) {
            sum = sum + mat[i][i];
        }

        // Secondary Diagonal Sum
        for (int i = 0, j = n - 1; i < n; i++, j--) {
            if (i != j)
                sum = sum + mat[i][j];
        }

        System.out.println("Sum : " + sum);
    }
}

// Time Complexity - - O(2n) /O(n)
// Aux Space - O(1)
```



Problem3B.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem3- Sum of Diagonal elements of n * n Matrix

public class Problem3B {
    public static void main(String[] args) {

        // int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
        int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        int n = mat.length;
        int sum = 0;

        for (int i = 0, j = n - 1; i < n; i++, j--) {
            if (i != j)
                sum = sum + mat[i][i] + mat[i][j];
            else
                sum = sum + mat[i][i];
        }

        System.out.println("Sum : " + sum);
    }
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem4.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem4- Reverse the Columns of Matrix

public class Problem4 {

    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }

    static void reverseColumns(int mat[][]) {
        int n = mat.length;

        for (int i = 0; i < n; i++) {
            int start = 0;
            int end = n - 1;
            while (start < end) {
                int temp = mat[start][i];
                mat[start][i] = mat[end][i];
                mat[end][i] = temp;

                start++;
                end--;
            }
        }
    }
}
```



```
public static void main(String[] args) {

    //int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        printMatrix(mat);
        System.out.println("-----");
        reverseColumns(mat);
        printMatrix(mat);
    }
}

// Time Complexity - - O(n2)
// Aux Space - O(1)
```

Problem5.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem5- Reverse the Rows of Matrix

public class Problem5 {

    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }
}
```



```
static void reverseRows(int mat[][]) {
    int n = mat.length;

    for (int i = 0; i < n; i++) {
        int start = 0;
        int end = n - 1;
        while (start < end) {
            int temp = mat[i][start];
            mat[i][start] = mat[i][end];
            mat[i][end] = temp;

            start++;
            end--;
        }
    }
}

public static void main(String[] args) {

    //int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

    printMatrix(mat);
    System.out.println("-----");
    reverseRows(mat);
    printMatrix(mat);
}

// Time Complexity - -  $O(n^2)$ 
// Aux Space -  $O(1)$ 
```



Problem6.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem6- Transpose of Matrix
// Swap the Rows to Cols and Cols to Rows

public class Problem6 {

    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }

    static void transpose(int mat[][]) {

        int n = mat.length;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int temp = mat[i][j];
                mat[i][j] = mat[j][i];
                mat[j][i] = temp;
            }
        }
    }
}
```



```
public static void main(String[] args) {

    // int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        printMatrix(mat);
        System.out.println("-----");
        transpose(mat);
        printMatrix(mat);
    }
}
```

// Time Complexity - - $O(n^2)$
// Aux Space - $O(1)$

Problem7.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```

//Problem7- Rotate of Matrix to Anti-Clockwise by 90 degrees.

```
public class Problem7 {

    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }
}
```




```
static void transpose(int mat[][]) {
    int n = mat.length;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {

            int temp = mat[i][j];
            mat[i][j] = mat[j][i];
            mat[j][i] = temp;
        }
    }
}

static void reverseColumns(int mat[][]) {
    int n = mat.length;

    for (int i = 0; i < n; i++) {
        int start = 0;
        int end = n - 1;
        while (start < end) {
            int temp = mat[start][i];
            mat[start][i] = mat[end][i];
            mat[end][i] = temp;

            start++;
            end--;
        }
    }
}

static void rotateLeft90(int mat[][]) {
    transpose(mat);
    reverseColumns(mat);
}
```



```
public static void main(String[] args) {

    // int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        printMatrix(mat);
        System.out.println("-----");
        rotateLeft90(mat);
        //rotateLeft90(mat);
        //rotateLeft90(mat);
        printMatrix(mat);
    }
}

// Time Complexity - -  $O(n^2)$ 
// Aux Space -  $O(1)$ 
```

Problem8.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem8- Rotate of Matrix to Clockwise by 90 degrees.

public class Problem8 {
    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }
}
```



```
static void transpose(int mat[][]) {
    int n = mat.length;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int temp = mat[i][j];
            mat[i][j] = mat[j][i];
            mat[j][i] = temp;
        }
    }
}

static void reverseRows(int mat[][]) {
    int n = mat.length;

    for (int i = 0; i < n; i++) {
        int start = 0;
        int end = n - 1;
        while (start < end) {
            int temp = mat[i][start];
            mat[i][start] = mat[i][end];
            mat[i][end] = temp;

            start++;
            end--;
        }
    }
}

static void rotateRight90(int mat[][]) {
    transpose(mat);
    reverseRows(mat);
}
```



```
public static void main(String[] args) {

    // int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        printMatrix(mat);
        System.out.println("-----");
        rotateRight90(mat);
        //rotateRight90(mat);
        //rotateRight90(mat);
        printMatrix(mat);

    }
}
```

// Time Complexity - - $O(n^2)$
// Aux Space - $O(1)$

DSA

Module 7

Strings

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

7. Strings

- ◆ String is a Sequence of Characters.
- ◆ String can be represented in java in the following ways
 - 1) String class
 - 2) StringBuffer
 - 3) StringBuilder
 - 4) Array of Characters
 - 5) ArrayList of Characters

7.1. String Class

- ◆ String is a built-in class in java.lang package.
- ◆ String is final class, so you can't define the subclass for String class.
- ◆ String class implements the following interfaces:
 - ◆ java.io.Serializable
 - ◆ java.lang.Comparable
 - ◆ java.lang.CharSequence
- ◆ String class has following variable to hold data.

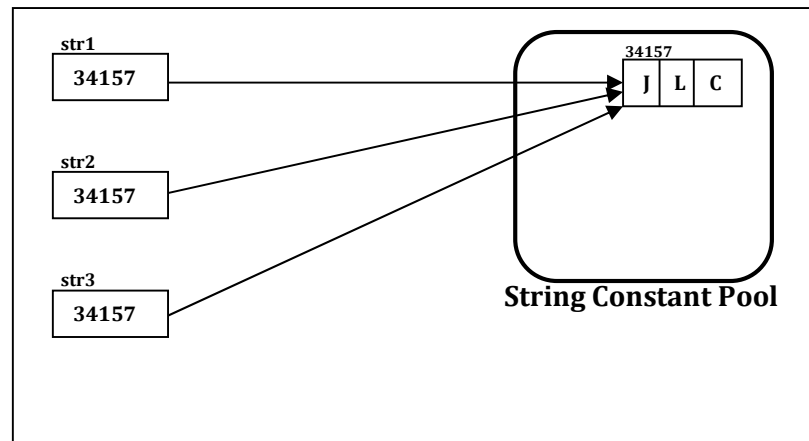
```
private final char value[];
```
- ◆ String objects are immutable objects. It means once the object is created then the content or data of the object can't be modified.
- ◆ When you try to modify the contents of object then new object will be created as a result.
- ◆ You can create the object of String in two ways.
 - ◆ Without using new operator.
 - ◆ Using new operator.

7.1.1. Creating String object without new operator

- 1) JVM allocates the memory for the String reference variable.
- 2) JVM verifies the String literal in the String Constant Pool.
- 3) If String literal is not available in the String Constant Pool then it creates new String object inside the String Constant pool and newly created String object address will be assigned to String reference variable.
- 4) If String literal is available then existing String object address will be assigned to String reference variable.

Demo1.java

```
class Lab457{
public static void main(String as[]){
String str1="JLC";
String str2="JLC";
String str3="JLC";
System.out.println(str1==str2);
System.out.println(str1==str3);
System.out.println(str2==str3);
}
}
```

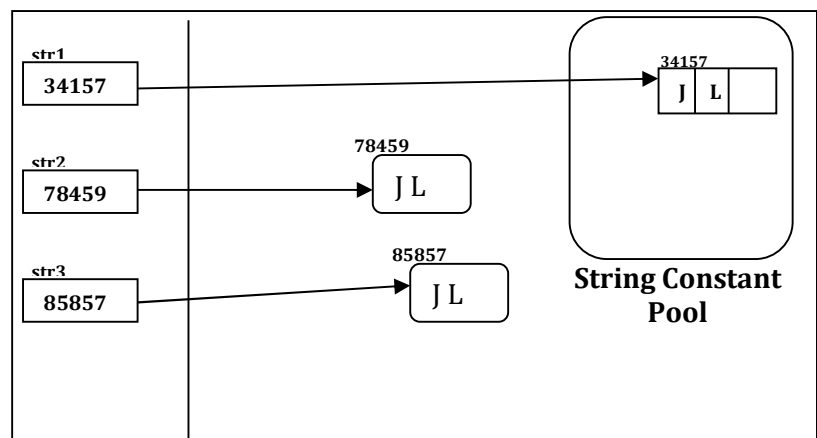


7.1.2 Creating String object with new operator

- 1) JVM allocates the memory for the String reference variable.
- 2) JVM verifies the String literal in the String Constant Pool.
- 3) If String literal is not available in the String Constant Pool then it creates new String object inside the String Constant pool.
- 4) If String literal is available in the String Constant Pool then ignores that.
- 5) It creates another new String object outside the constant pool and assigns address of the newly created String object outside the pool to String reference variable.

Demo2.java

```
class Lab458{
public static void main(..... as[]){
String str1="JLC";
String str2=new String("JLC");
String str3=new String("JLC");
System.out.println(str1==str2);
System.out.println(str1==str3);
System.out.println(str2==str3);
}
}
```



7.2. StringBuffer Class

- ◆ StringBuffer is a final class in java.lang package.
- ◆ It is used to store the sequence of characters.
- ◆ This is used when there is a necessity to make a lot of modifications to characters of String.
- ◆ StringBuffer is a mutable sequence of characters, It means the contents of the StringBuffer can be modified after creation.
- ◆ Every StringBuffer object has a capacity associated with it.
- ◆ The capacity of the StringBuffer is the number of characters it can hold.
- ◆ The capacity increases automatically as more contents added to it.
- ◆ The methods available in the StringBuffer class are synchronized.
- ◆ StringBuffer class is thread-safe i.e. multiple threads cannot access it simultaneously.

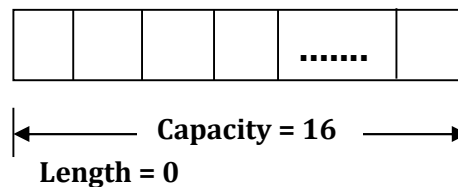
7.3. StringBuilder Class

- ◆ StringBuilder is a final class in java.lang package.
- ◆ StringBuilder is a newly added class from Java 5.
- ◆ StringBuilder class functionality is same as StringBuffer only except the methods available in the StringBuilder class are non synchronized.
- ◆ StringBuilder class is not thread-safe i.e. multiple threads can access it simultaneously.

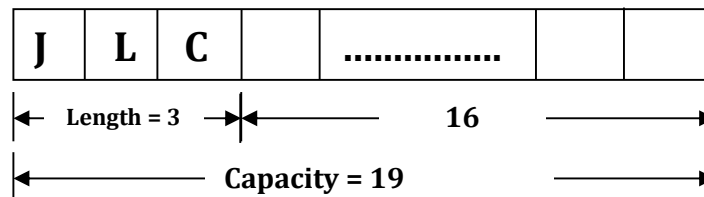
7.4. About StringBuffer and StringBuilder

- ♦ equals() is not overridden in StringBuilder and StringBuffer class.
- ♦ When you want to compare contents of StringBuilder or StringBuffer object then you need to convert the object of StringBuilder or StringBuffer into String.
- ♦ hashCode() is not overridden in the StringBuilder and StringBuffer class. It uses the default implementation of Object class.
- ♦ StringBuilder and StringBuffer are not implementing java.lang.Comparable interface, So You can't use compareTo() with StringBuilder and StringBuffer.

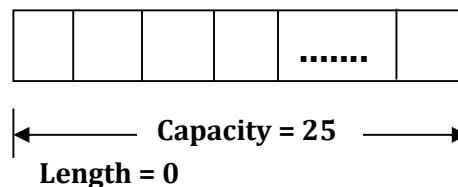
StringBuilder sb=new StringBuilder();



StringBuilder sb=new StringBuilder("JLC");



StringBuilder sb=new StringBuilder(25);





7.5. Examples on Strings

Lab1.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static void main(String[] args) {

        String str1="JLC";
        String str2="JLC";

        System.out.println(str1==str2);

        String str3=new String("JLC");
        System.out.println(str1==str3);

    }
}
```



Lab2.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {
    public static void main(String[] args) {

        String str1="Hello";
        String str2=" Guys";

        String str3="Hello Guys";
        String str4= str1.concat(str2);
        String str5="Hello".concat(" Guys");

        System.out.println(str3==str4); //F
        System.out.println(str3==str5); //F
        System.out.println(str4==str5); //F

        String str6=str1+str2;
        String str7="Hello" + " Guys"; //IMP- Literal + Literal

        System.out.println(str3==str6);
        System.out.println(str3==str7);

    }
}
```



Lab3.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab3 {
    public static void main(String[] args) {

        String str1="JLCD";
        String str2="JLCD";
        String str3="JLCd";

        System.out.println(str1.equals(str2));

        System.out.println(str3.equals(str2));
        System.out.println(str3.equalsIgnoreCase(str2));

        System.out.println(str1.compareTo(str2)); //Same
        System.out.println(str2.compareTo(str3)); //str2 is smaller
        System.out.println(str3.compareTo(str2)); //str3 is Bigger

    }
}
```



Lab4.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    public static void main(String[] args) {

        String str="srinivasd";

        System.out.println(str);
        System.out.println(str.length());

        char chArr[] = str.toCharArray();

        for(int i=0;i<chArr.length;i++)
            System.out.print(chArr[i]+" ");

        System.out.println("\n");

        chArr[0]='S';
        chArr[8]='D';

        for(int i=0;i<chArr.length;i++)
            System.out.print(chArr[i]+" ");

        System.out.println("\n");

        String str1=new String(chArr);
        System.out.println(str1);
        System.out.println(str);
    }
}
```



Lab5.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5 {
    public static void main(String[] args) {

        String str="srinivasd";

        System.out.println(str.length());
        System.out.println(str.charAt(0));
        System.out.println(str.charAt(8));

        System.out.println(str.substring(5));
        System.out.println(str.substring(3,8));
        System.out.println(str.substring(3,9));

        System.out.println(str.indexOf("nivas"));
        System.out.println(str.indexOf("i"));
        System.out.println(str.lastIndexOf("i"));

        System.out.println(str.contains("vas"));
        System.out.println(str.contains("nivas"));
        System.out.println(str.contains("hello"));

    }
}
```



Lab6.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab6 {
    public static void main(String[] args) {

        StringBuffer sb1=new StringBuffer("JLC");
        StringBuffer sb2=new StringBuffer("JLC");

        System.out.println(sb1==sb2);

        System.out.println(sb1.equals(sb2));

        boolean b1= sb1.toString().contentEquals(sb2.toString());
        boolean b2= sb1.toString().equals(sb2.toString());

        System.out.println("b1 : "+b1);
        System.out.println("b2 : "+b2);

        sb1.append(" Guys");
        System.out.println(sb1);

    }
}
```




Lab7.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab7 {
    public static void main(String[] args) {

        StringBuilder sb1=new StringBuilder("JLC");
        StringBuilder sb2=new StringBuilder("JLC");

        System.out.println(sb1==sb2);

        System.out.println(sb1.equals(sb2));

        boolean b1= sb1.toString().contentEquals(sb2.toString());
        boolean b2= sb1.toString().equals(sb2.toString());

        System.out.println("b1 : "+b1);
        System.out.println("b2 : "+b2);

        sb1.append(" Guys");
        System.out.println(sb1);

        sb1.reverse();
        System.out.println(sb1);
    }
}
```

7.6. Problems on Strings

Problem1A.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem1- Reverse the given String

public class Problem1A {
    public static void main(String[] args) {

        String str = "Hello";
        System.out.println(str);

        StringBuffer sb=new StringBuffer(str);

        sb.reverse();

        String revStr = sb.toString();
        System.out.println(revStr);

    }
}

// Time Complexity - - O(1)
// Aux Space - O(1)
```



Problem1B.java

```
package com.jlccindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem1- Reverse the given String
public class Problem1B {
    static String reverseString(String str) {

        char chArr[] = str.toCharArray();
        int n = chArr.length;
        int start=0;
        int end=n-1;
        while(start<end) {
            char temp = chArr[start];
            chArr[start] = chArr[end];
            chArr[end]=temp;

            start++;
            end--;
        }
        String revStr=new String(chArr);
        return revStr;
    }
    public static void main(String[] args) {
        String str = "Hello";
        System.out.println(str);

        String revStr = reverseString(str);
        System.out.println(revStr);
    }
}

// Time Complexity - - O(n/2)/O(n)
// Aux Space - O(n)
```



Problem2.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem2- Reverse Words of given Sentence

public class Problem2 {

    static void reverse(char chArr[], int start, int end) {
        while (start < end) {
            char temp = chArr[start];
            chArr[start] = chArr[end];
            chArr[end] = temp;

            start++;
            end--;
        }
    }

    static void reverseWords(char chArr[]) {
        int n = chArr.length;
        int start = 0;
        int end = 0;
        while (end < n) {
            if (chArr[end] == ' ') {
                reverse(chArr, start, end - 1);
                start = end + 1;
            }
            end++;
        }
        reverse(chArr, start, end-1); // Reverse Last Word
        reverse(chArr, 0, n-1); //Reverse the Entire String
    }
}
```



```
public static void main(String[] args) {  
  
    String str = "SD OK";  
    System.out.println(str);  
  
    char chArr[] = str.toCharArray();  
    reverseWords(chArr);  
  
    String revStr= new String(chArr);  
    System.out.println(revStr);  
  
    }  
}
```

//Time Complexity - $O(3n)$ / $O(n)$

//Aux Space --- $O(n)$



Problem3.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem3- Frequency of each Character of given String with lower
// Alphabets in Sorted Order
public class Problem3 {

    static void charFrequency(String str) {
        int n= str.length();

        int count[] = new int[26];

        for(int i=0;i<n;i++) {
            char ch=str.charAt(i);
            count[ch-97]++;
        }

        for(int i=0;i<count.length;i++) {
            if(count[i]>0) {
                char ch = (char) (97+i);
                System.out.println(ch + "\t"+count[i]);
            }
        }
    }

    public static void main(String[] args) {
        String str = "srinivas";
        charFrequency(str);
    }
}

//Time Complexity - O(n)
//Aux Space --- O(26)/O(1)
```



Problem4A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P4- Valid Palindrome
//Check whether given string is Palindrome or Not.

public class Problem4A {

    static boolean isPalindrome(String str) {

        StringBuilder sb=new StringBuilder(str); //O(1)
        sb.reverse(); // O(n)
        String revStr=sb.toString(); //O(1)

        return str.equals(revStr);// O(n)
    }

    public static void main(String[] args) {

        String str1 = "abcba";
        boolean b1 = isPalindrome(str1);
        System.out.println(b1);

        String str2 = "abc";
        boolean b2 = isPalindrome(str2);
        System.out.println(b2);

    }
}

//Time Complexity - O(n)
//Aux Space Complexity - O(n)
```



Problem4B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P4- Valid Palindrome
//Check whether given string is Palindrome or Not.

public class Problem4B {
    static boolean isPalindrome(String str) {
        int n = str.length();
        int start = 0;
        int end = n - 1;

        while (start < end) {
            if (str.charAt(start) != str.charAt(end)) {
                return false;
            }

            start++;
            end--;
        }
        return true;
    }
    public static void main(String[] args) {

        String str1 = "abccba";
        boolean b1 = isPalindrome(str1);
        System.out.println(b1);
    }
}

//Time Complexity -  $O(n/2) \Rightarrow O(n)$ 
//Aux Space Complexity -  $O(1)$ 
```




Problem5A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.Arrays;

//P5- Valid Anagrams
//Check whether Two strings are anagrams are or not
public class Problem5A {
    static boolean isAnagram(String str1,String str2) {

        if(str1.length()!=str2.length())
            return false;

        char charr1[] = str1.toCharArray();
        Arrays.sort(charr1);
        str1 = new String(charr1);

        char charr2[] = str2.toCharArray();
        Arrays.sort(charr2);
        str2 = new String(charr2);

        return str1.equals(str2);
    }
    public static void main(String[] args) {
        String str1 = "sir";
        String str2 = "sri";

        boolean b1 = isAnagram(str1,str2);
        System.out.println(b1);
    }
}

//Time Complexity - O(n log(n))
//Aux Space Complexity - O(m+n) // O(n)
```



Problem5B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P5- Valid Anagrams
//Check whether Two strings are anagrams are or not
public class Problem5B {
    static boolean isAnagram(String str1,String str2) {
        if(str1.length()!=str2.length())
            return false;

        int counter[] = new int[26];

        for(int i=0;i<str1.length();i++) {
            counter[str1.charAt(i) - 97]++;
            counter[str2.charAt(i) - 'a']--;
        }

        for(int i=0;i<counter.length;i++) {
            if(counter[i]!=0) {
                return false;
            }
        }
        return true;
    }
    public static void main(String[] args) {
        String str1 = "abc";
        String str2 = "cba";
        boolean b1 = isAnagram(str1,str2);
        System.out.println(b1);
    }
}
//Time Complexity - O(n+26) / O(n)
//Aux Space Complexity - O(26) // O(1)
```



Problem6A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P6- First Repeating Character in the given String
//Consider the String with Lower Alphabets

public class Problem6A {

    static int firstRepeat(String str) {
        int n = str.length();
        for(int i=0;i<n;i++) {
            for(int j=i+1;j<n;j++) {
                if(str.charAt(i) == str.charAt(j)) {
                    return i;
                }
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        String str = "abcdcdef";
        int index = firstRepeat(str);
        System.out.println(index);
    }
}

//Time Complexity - O(n^2)
//Aux Space Complexity - O(1)
```



Problem6B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P6- First Repeating Character in the given String
//Consider the String with Lower Alphabets

public class Problem6B {

    static int firstRepeat(String str) {
        int counter[] = new int[26];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - 'a']++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-97]>1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        String str = "abcdcdef";
        int index = firstRepeat(str);
        System.out.println(index);
    }
}

//Time Complexity -  $O(2n) \Rightarrow O(n)$ 
//Aux Space Complexity -  $O(26) \Rightarrow O(1)$ 
```



Problem7A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7- First Unique Character in the given String
//Consider the String with Lower Alphabets

public class Problem7{

    static int firstUnique(String str) {

        int counter[] = new int[26];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - 'a']++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-97]==1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        String str = "abcdabc";
        int index = firstUnique(str);
        System.out.println(index);
    }
}

//Time Complexity - O(2n) => O(n)
//Aux Space Complexity - O(26) => O(1)
```



Problem7B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7- First Unique Character in the given String
//Consider the String with AlphaNumeric (A-Z,a-z,0-9):

public class Problem7B{

    static int firstUnique(String str) {

        int counter[] = new int[75];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - '0']++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-48]==1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        String str = "1Ab2D1cd5Ab";
        int index = firstUnique(str);
        System.out.println(index);
    }
}
//Time Complexity - O(2n) => O(n)
//Aux Space Complexity - O(75) => O(1)
```



Problem7C.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7- First Non-Repeating Character in the given String
//Consider the String with Alphabets (A-Z,a-z):

public class Problem7C{

    static int firstUnique(String str) {

        int counter[] = new int[58];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - 'A']++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-65]==1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        String str = "AbDcdAbD";
        int index = firstUnique(str);
        System.out.println(index);
    }
}
//Time Complexity - O(2n) => O(n)
//Aux Space Complexity - O(58) => O(1)
```



Problem7D.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7- First Non-Repeating Character in the given String
//Consider the String with Special Symbols, Alphanumerics

public class Problem7D{

    static int firstUnique(String str) {

        int counter[] = new int[95];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - ' ' ]++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-32]==1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        String str = "#A1*2$BcdF#A1    *2BcdF#A1*2BcdF#A1*2BcdF";
        int index = firstUnique(str);
        System.out.println(index);
    }
}

//Time Complexity - O(2n) => O(n)
//Aux Space Complexity - O(95) => O(1)
```




Problem8.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P8- IsSubSequence

public class Problem8 {
    static boolean isSubsequence(String main,String sub) {
        int m= main.length();
        int n= sub.length();

        if(m<n)
            return false;

        int i= 0;
        int j=0;

        while(i<m && j<n) {
            if(main.charAt(i)==sub.charAt(j)) {
                j++;
            }
            i++;
        }
        return (j==n);
    }
    public static void main(String[] args) {
        String main = "srinivas";
        String sub = "snvs";
        boolean b= isSubsequence(main,sub);
        System.out.println(b);
    }
}
//Time Complexity - O(n)
// Space Complexity - O(1)
```



Problem9.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P9- Remove Duplicate Characters
public class Problem9 {
    static String removeDuplicates(String str) {

        int n= str.length();
        int count[] = new int[26];

        for(int i=0;i<n;i++) {
            count[str.charAt(i)-97]++;
        }

        StringBuffer sb= new StringBuffer();
        for(int i=0;i<count.length;i++) {
            if(count[i]!=0) {
                char ch= (char)(i+97);
                sb.append(ch);
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        String str = "bacacdbac";
        String mystr= removeDuplicates(str);
        System.out.println(mystr);
    }
}

//Time Complexity => O(n)
// Space Complexity => O(1)
```



Problem10.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//P10- Reverse the Vowels of String

public class Problem10 {

    public boolean isVowel(char ch) {
        boolean b = ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U';
        return b;
    }

    public String reverseVowels(String str) {
        char chArr[] = str.toCharArray();

        int n= chArr.length;
        int start = 0;
        int end = n-1;

        while(start<end) {

            while(start<n && !isVowel(chArr[start])) {
                start++;
            }

            while(end>=0 && !isVowel(chArr[end])) {
                end--;
            }


```



```
        if(start<end) {
            char temp = chArr[start];
            chArr[start] = chArr[end];
            chArr[end] = temp;
        }

        start++;
        end--;

    }

    String resStr= new String(chArr);
    return resStr;
}

public static void main(String[] args) {

    String str = "srinivaas";
    String mystr = reverseVowels(str);
    System.out.println(mystr);

}
}
```

//Time Complexity - O(n)
// Space Complexity - O(n)



Java Learning Center

No.1 In Java Training & placement

DSA

Module 8

Hashing

Author

Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

8. Hashing

- ◆ **Hashing** is a technique used to store and retrieve data as quickly as possible.
- ◆ Hashing is used to perform Optimal Searches
- ◆ Consider the following two Use-Cases.

Use-Case 1:

- ◆ Consider we have an array of integer numbers.
- ◆ I want to perform the following 3 Operations on the Array

Operation	Time Complexity
Search the number in the Array	$O(n)$
Insert the number in the Array	$O(n)$
Delete the number from the Array	$O(n)$

Use-Case 1:

- ◆ Consider we have an Binary Search Tree(BST).
- ◆ I want to perform the following 3 Operations on BST

Operation	Time Complexity
Search the number in the BST	$O(\log n)$
Insert the number in the BST	$O(\log n)$
Delete the number from the BST	$O(\log n)$

- ◆ Hashing works **best for these 3 Operations** and beats all remaining Data Structures.
- ◆ Hashing **provides $O(1)$ average time complexity** for 3 Important Operations - Search , Insert , Delete
- ◆ Hashing does exact key search
- ◆ Hashing is not useful when you want to find closet values or prefix searching etc.

8.1. Applications of Hashing

- 1) Dictionaries/Map
- 2) Sets
- 3) Database Indexing
- 4) Symbol Tables in Compilers
- 5) rypthography
- 6) Password Verification (MD5,SHA256)
- 7) Robin-Karp Algorithm
- Etc

8.2. Direct Address Table

- ◆ Consider we have 25 keys with the values ranging from 0 to 24.
- ◆ How can we implement the following 3 operations in $O(1)$ time complexity?
 - a) Search
 - b) Insert
 - c) Delete

Lab1.java

```
package com.jlccindia.hashing1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyTable {
    int mytable[];
    int capacity;

    public MyTable(int capacity) {
        this.capacity = capacity;
        mytable = new int[capacity];
    }

    public void insert(int key) {
        mytable[key]++;
    }
}
```




```
public void delete(int key) {
    mytable[key]--;
}
public int search(int key) {
    return mytable[key];
}
public void show() {
    for(int i=0;i<capacity;i++) {
        if(mytable[i]!=0) {
            System.out.println(i);
        }
    }
}
}
public class Lab1 {
    public static void main(String[] args) {
        MyTable mytable = new MyTable(25);
        mytable.insert(5);
        mytable.insert(2);
        mytable.insert(7);
        mytable.insert(9);
        mytable.insert(0);

        mytable.show();

        mytable.delete(0);
        mytable.delete(2);

        System.out.println("After Delete--");
        mytable.show();
        System.out.println("-----");

        System.out.println(mytable.search(5));
        System.out.println(mytable.search(2));
    }
}
```

Problems with above solution:

- ♦ It does not handle the following
 - a) Large Numbers
 - b) Negative Numbers
 - c) Floating Point numbers
 - d) Strings
 - e) Addresses (Any Object Address)
- ♦ We can use Hashing to solve the above problems

8.3. Exploring Hashing

- ♦ Hashing is a technique used to store and retrieve keys as quickly as possible
- ♦ Keys can be any of the following
 - a) Large Numbers
 - b) Negative Numbers
 - c) Floating Point numbers
 - d) Strings
 - e) Addresses (Any Object Address)
- ♦ You can have Universe of Keys -
- ♦ You can store the keys in Hashtable by converting the keys to small values.

8.3.1. Hash Functions

- ♦ Hash function is used to transform the key into the index.
- ♦ Ideally, **Hash Function** should map each possible key to a unique slot index, but it is difficult to achieve in practice.
- ♦ Given a collection of elements, a hash function that maps each item into a unique slot is referred to as a perfect hash function

8.3.2. Hash Function Examples

1) for numbers

- $\text{hash}(\text{key}) = \text{key} \% m$
- where m is table size (consider prime number)

2) for Strings

- $\text{str} = \text{"abc"}$
- $\text{hash}(\text{key}) = (\text{str}[0] * x^0 + \text{str}[1] * x^1 + \text{str}[2] * x^2 + \dots) \% m$

3) Universal Hashing:

- Group of Hash Functions and Pick one Randomly.

Example-1

keys = {53,51,55,52,50,54,49}

$\text{hash}(\text{key}) = \text{key} \% 7$

Example-2

keys = { 53,51,55,52,50,54,49}

$\text{hash}(\text{key}) = \text{key} \% 11$

8.3.3. Load Factor

Load Factor = Number of items stored in the table / Size of the table
--

Ex:

Load Factor = $7/11 \Rightarrow 0.66$

- ♦ This is the decision parameter used when we want to rehash or expand the existing hash table entries.
- ♦ This also helps us in determining the efficiency of the hashing function.
- ♦ That means, it tells whether the hash function is distributing the keys uniformly or not.

8.3.4. Characteristics of Good Hash Functions

- ♦ A good hash function should have the following characteristics:
 - 1) Should generate same value every time for the given key.
 - 2) Should generate the values from 0 to m-1
 - 3) Should generate fast - $O(1)$ for Integers and $O(\text{len})$ for Strings.
 - 4) Should uniformly distribute Large keys into Hashtable slots.
 - 5) Minimize collision
 - 6) Have a Low load factor for a given set of keys

8.4. Collisions

- ♦ Hash functions are used to map each key to a different Hashtable slots, but practically it is not possible to create such a hash function and the problem is called collision.
- ♦ Collision is the condition where two or more keys are stored in the Same Hashtable slots.

Collision Handling:

- ♦ If you know the keys in Advance then we can do the hashing perfectly (called Perfect Hashing).
- ♦ Collision is bound to happen if you dont know the keys in advance.
- ♦ **The process of finding an Alternative Hashtable slot is called collision resolution.**
- ♦ Even though hash tables have collision problems, they are more efficient in many cases compared to all other data structures, like search trees.
- ♦ There are a number of collision resolution techniques, and the most popular are **direct chaining** and **open addressing**.

A) Direct Chaining:

- Separate Chaining or Chaining

B) Open Addressing:

- Linear probing
- Quadratic probing
- Double hashing

8.5. Chaining

- ◆ When two or more Keys with same hash goes to same hashtable slot, these Keys are stored into a Separate Data Structure called a chain.
- ◆ Data Structures for Storing Chains:
 - a) Linked List
 - b) ArrayList
 - c) Self Balanced BST

a) Linked List

✓ Not Cache Friendly	Insert	->	$O(1)$
✓ Extra Space for Node Representation	delete	->	$O(1)$
	Search	->	$O(1)$

b) ArrayList

✓ Cache Friendly	Insert	->	$O(1)$
✓ No Extra Space	delete	->	$O(1)$
	Search	->	$O(1)$

c) Self Balanced BST

✓ Not Cache Friendly	Insert	->	$O(\log l)$
✓ Used from Java8	delete	->	$O(\log l)$
	Search	->	$O(\log l)$

Example-1

keys = { 50, 21, 58, 17, 15, 49, 56, 22, 23, 25}

hash(key) = key % 7

Lab2.java

```
package com.jlcindia.hashing2;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

class MyHashSet {

    int bucket;
    List<LinkedList<Integer>> mytable;

    public MyHashSet(int bucket) {
        this.bucket = bucket;
        this.mytable = new ArrayList<LinkedList<Integer>>(bucket);

        for(int i=0;i<bucket;i++) {
            mytable.add(new LinkedList<Integer>());
        }
    }

    public void insert(int key) {
        int index = key%bucket;
        if( !search(key))
            mytable.get(index).add(key);
    }
}
```



```
public void delete(int key) {
    int index = key%bucket;
    mytable.get(index).remove((Integer)key);
}

public boolean search(int key) {
    int index = key%bucket;
    return mytable.get(index).contains(key);
}

public String toString() {
    return mytable.toString();
}
}

public class Lab2 {
    public static void main(String[] args) {

        MyHashSet myset = new MyHashSet(7);

        myset.insert(50);
        myset.insert(21);
        myset.insert(58);
        myset.insert(17);
        myset.insert(15);
        myset.insert(49);
        myset.insert(56);
        myset.insert(22);
        myset.insert(23);
        myset.insert(25);
        myset.insert(23);
        myset.insert(25);

        System.out.println(myset);
    }
}
```




```
        myset.delete(23);
        myset.delete(25);
        myset.delete(50);

        System.out.println(myset);

        System.out.println(myset.search(50));
        System.out.println(myset.search(49));
    }
}
```

Lab3.java

```
package com.jlcindia.hashing3;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

//Implement HashSet using Chaining
// To Store Integers
// Updated

class MyHashSet {

    private int bucketSize;
    private int currentSize;
    List<LinkedList<Integer>> mytable;

    public MyHashSet(int bucketSize) {
        this.bucketSize = bucketSize;
        this.mytable = new ArrayList<LinkedList<Integer>>(bucketSize);
    }
}
```




```
        for(int i=0;i<bucketSize;i++) {
            mytable.add(new LinkedList<Integer>());
        }
    }

    public int size() {
        return this.currentSize;
    }

    public boolean isEmpty() {
        return currentSize==0;
    }

    public void clear() {

        this.mytable.clear();

        for(int i=0;i<bucketSize;i++) {
            mytable.add(new LinkedList<Integer>());
        }
        this.currentSize=0;
    }

    public void insert(Integer key) {
        int index = myhash(key);
        if( !search(key)) {
            mytable.get(index).add(key);
            currentSize++;
        }
    }

    public void delete(Integer key) {
        int index = myhash(key);
        mytable.get(index).remove((Integer)key);
        currentSize--;
    }
}
```



```
public boolean search(Integer key) {
    int index = myhash(key);
    return mytable.get(index).contains(key);
}

public int myhash(Integer key) {
    int hash = key.hashCode() % this.bucketSize;
    return hash;
}

public String toString() {
    return mytable.toString();
}
}

public class Lab3 {
    public static void main(String[] args) {

        MyHashSet myset = new MyHashSet(7);

        System.out.println("-----1-----");
        System.out.println(myset.size());
        System.out.println(myset.isEmpty());
        System.out.println(myset);

        myset.insert(50);
        myset.insert(21);
        myset.insert(58);
        myset.insert(17);
        myset.insert(15);
        myset.insert(49);
        myset.insert(56);
        myset.insert(22);
        myset.insert(23);
        myset.insert(25);
        myset.insert(23);
    }
}
```



```
myset.insert(25);

System.out.println("-----2-----");

System.out.println(myset.size());
System.out.println(myset.isEmpty());
System.out.println(myset);

myset.delete(23);
myset.delete(25);
myset.delete(50);

System.out.println("-----3-----");

System.out.println(myset.size());
System.out.println(myset.isEmpty());
System.out.println(myset);

System.out.println("-----4-----");
System.out.println(myset.search(50));
System.out.println(myset.search(49));

System.out.println("-----5-----");
myset.clear();
System.out.println(myset.size());
System.out.println(myset.isEmpty());
System.out.println(myset);
}
}
```



Lab4.java

```
package com.jlcindia.hashing4;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

//Implement HashSet using Chaining
//To Store Strings

class MyHashSet {

    private int bucketSize;
    private int currentSize;
    List<LinkedList<String>> mytable;

    public MyHashSet(int bucketSize) {
        this.bucketSize = bucketSize;
        this.mytable = new ArrayList<LinkedList<String>>(bucketSize);

        for(int i=0;i<bucketSize;i++) {
            mytable.add(new LinkedList<String>());
        }
    }

    public int size() {
        return this.currentSize;
    }

    public boolean isEmpty() {
        return currentSize==0;
    }
}
```



```
public void clear() {  
  
    this.mytable.clear();  
  
    for(int i=0;i<bucketSize;i++) {  
        mytable.add(new LinkedList<String>());  
    }  
    this.currentSize=0;  
}  
  
public void insert(String key) {  
    int index = myhash(key);  
    if( !search(key)) {  
        mytable.get(index).add(key);  
        currentSize++;  
    }  
}  
  
public void delete(String key) {  
    int index = myhash(key);  
    mytable.get(index).remove(key);  
    currentSize--;  
}  
  
public boolean search(String key) {  
    int index = myhash(key);  
    return mytable.get(index).contains(key);  
}  
  
public int myhash(String key) {  
    int hash = key.hashCode() % this.bucketSize;  
    return hash;  
}
```



```
        public String toString() {
            return mytable.toString();
        }
    }

    public class Lab4 {
        public static void main(String[] args) {

            MyHashSet myset = new MyHashSet(7);

            System.out.println("-----1-----");
            System.out.println(myset.size());
            System.out.println(myset.isEmpty());
            System.out.println(myset);

            myset.insert("aa");
            myset.insert("bb");
            myset.insert("cc");
            myset.insert("dd");
            myset.insert("ee");

            myset.insert("ab");
            myset.insert("bc");
            myset.insert("cd");
            myset.insert("de");
            myset.insert("ef");

            System.out.println("-----2-----");

            System.out.println(myset.size());
            System.out.println(myset.isEmpty());
            System.out.println(myset);
        }
    }
```



```
        myset.delete("ab");
        myset.delete("de");
        myset.delete("dd");

        System.out.println("-----3-----");

        System.out.println(myset.size());
        System.out.println(myset.isEmpty());
        System.out.println(myset);

        System.out.println("-----4-----");
        System.out.println(myset.search("aa"));
        System.out.println(myset.search("dd"));

        System.out.println("-----5-----");
        myset.clear();
        System.out.println(myset.size());
        System.out.println(myset.isEmpty());
        System.out.println(myset);
    }
}
```

8.6. Open Addressing

- ♦ In open addressing, all keys are stored in the hash table itself.
- ♦ This approach is also known as closed hashing.
- ♦ This procedure is based on probing.
- ♦ Collision is resolved by probing.

- ♦ Open Addressing can be implemented in 3 ways
 - 1) Linear probing
 - 2) Quadratic probing
 - 3) Double hashing ***

8.7. Linear Probing

- ◆ Interval between probes is fixed at 1.
- ◆ In linear probing, we search the hash table sequentially, starting from the original hash location.
- ◆ If a location is occupied, we check the next location.
- ◆ We wrap around from the last table location to the first table location if necessary.

Ex:

keys = {50, 51, 49, 16, 56, 15, 19}

hash(key) = key % 7

- ◆ Problem with linear probing is that table items form clusters together in the hash table.
- ◆ This means that the table contains groups of consecutively occupied locations that are called clustering.
- ◆ Clustering causes long probe searches and therefore decreases the overall efficiency.

8.8. Quadratic Probing

- ◆ Interval between probes increases proportionally to the hash value
- ◆ Problem of Clustering can be reduced if we use the quadratic probing method.
- ◆ In quadratic probing, we start from the original hash location.
- ◆ If a location is occupied, we check the locations $i + 1^2$, $i + 2^2$, $i + 3^2$, $i + 4^2$...

Ex:

keys = {31, 19, 2, 13, 25, 24, 21, 9}

hash(key) = key % 11

- ◆ Even though clustering is avoided by quadratic probing, still there are chances of thin clustering.

8.9. Double Hashing

- ◆ Interval between probes is computed by another hash function.
- ◆ Double hashing reduces clustering in a better way.
- ◆ Increments for the probing sequence are computed by using a second hash function.
- ◆ In Double hashing, We first probe the location $h1(key)$.
- ◆ If the location is occupied, we probe the location $(h1(key) + i * h2(key)) \% m$

For Example, as follows

$$(h1(key) + i * h2(key)) \% m$$

$$(h1(key) + 1 * h2(key)) \% m$$

$$(h1(key) + 2 * h2(key)) \% m$$

$$(h1(key) + 3 * h2(key)) \% m$$

etc

Example-1

keys = {49, 63, 56, 52, 54, 48}

$m = 7$ (0 to 6)

$h1(key) = key \% 7$

$h2(key) = 6 - (key \% 6)$



Lab5.java

```
package com.jlcindia.hashing5;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Implement HashSet using Double Hashing

class MyHashSet {
    private int bucketSize;
    private int currentSize;
    private Integer[] mytable;

    public MyHashSet(int bucketSize) {
        this.bucketSize = bucketSize;
        this.mytable = new Integer[bucketSize];
        this.currentSize = 0;
    }

    public int size() {
        return this.currentSize;
    }

    public boolean isEmpty() {
        return currentSize == 0;
    }

    public void clear() {
        this.currentSize = 0;
        for (int i = 0; i < bucketSize; i++) {
            mytable[i] = null;
        }
    }
}
```



```
public void add(Integer key) {
    int index = myhash1(key);
    if (mytable[index] != null) {
        index = myhash(key,"add");
    }

    mytable[index] = key;
    this.currentSize++;
}

public void remove(Integer key) {

    int index = myhash1(key);
    if (mytable[index] != key) {
        index = myhash(key,"remove");
    }

    if (mytable[index] == key) {
        mytable[index] = null;
        this.currentSize--;
    }
}

public boolean contains(Integer key) {
    int index = myhash1(key);
    if (mytable[index] != key) {
        index = myhash(key,"search");
    }

    return mytable[index] == key;
}

public int myhash1(Integer key) {
    int hash1 = key.hashCode() % this.bucketSize;
    return hash1;
}
```



```
public int myhash2(Integer key) {
    int hash2 = (this.bucketSize - 1) - key.hashCode() % (this.bucketSize - 1);
    return hash2;
}

public int myhash(Integer key, String ops) {
    int hash1 = myhash1(key);
    int hash2 = myhash2(key);

    int myhash = 0;
    for (int i = 1; i <= this.bucketSize; i++) {
        myhash = (hash1 + i * hash2) % this.bucketSize;
        if (ops.equals("add")) {
            if (mytable[myhash] == null) {
                return myhash;
            }
        } else {
            if (mytable[myhash] == key) {
                return myhash;
            }
        }
    }
    return myhash;
}

public String toString() {
    String str = "[";
    for (int i = 0; i < this.bucketSize; i++) {
        if (mytable[i] != null) {
            str += mytable[i] + "\t";
        }
    }
    str = str + "]";
    return str;
}
}
```



```
public class Lab5 {  
    public static void main(String[] args) {  
        MyHashSet myset = new MyHashSet(7);  
  
        System.out.println("-----1-----");  
        System.out.println(myset.size());  
        System.out.println(myset.isEmpty());  
        System.out.println(myset);  
  
        myset.add(49);          myset.add(63);  
        myset.add(56);          myset.add(52);  
        myset.add(54);          myset.add(48);  
  
        System.out.println("-----2-----");  
        System.out.println(myset.size());  
        System.out.println(myset.isEmpty());  
        System.out.println(myset);  
  
        myset.remove(49);      myset.remove(48);  
        myset.remove(99);  
  
        System.out.println("-----3-----");  
        System.out.println(myset.size());  
        System.out.println(myset.isEmpty());  
        System.out.println(myset);  
  
        System.out.println("-----4-----");  
        System.out.println(myset.contains(49));  
        System.out.println(myset.contains(52));  
  
        myset.clear();  
        System.out.println("-----5-----");  
        System.out.println(myset.size());  
        System.out.println(myset.isEmpty());  
        System.out.println(myset);  
    }  
}
```



Lab6.java

```
package com.jlcindia.hashing6;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Implement HashMap using Chaining To Store Integers
// 706. Design HashMap

class MyEntry<K, V> {

    public Integer key;
    public Integer value;

    public MyEntry(Integer key, Integer value) {
        this.key = key;
        this.value = value;
    }

    public String toString() {
        return "{" + key + ":" + value + "}";
    }
}

class MyHashMap {

    private int bucketSize;
    private int currentSize;
    List<LinkedList<MyEntry<Integer, Integer>>> mytable;
```



```
public MyHashMap(int bucketSize) {

    this.bucketSize = bucketSize;
    this.mytable =
    new ArrayList<LinkedList<MyEntry<Integer, Integer>>>(bucketSize);

    for (int i = 0; i < bucketSize; i++) {
        mytable.add(new LinkedList<MyEntry<Integer, Integer>>());
    }
}

public int size() {
    return this.currentSize;
}

public boolean isEmpty() {
    return currentSize == 0;
}

public void clear() {
    this.mytable.clear();

    for (int i = 0; i < bucketSize; i++) {
        mytable.add(new LinkedList<MyEntry<Integer, Integer>>());
    }
    this.currentSize = 0;
}

public Integer get(Integer mykey) { // 50
    int index = myhash(mykey); // 1
    LinkedList<MyEntry<Integer, Integer>> mylinkedlist = mytable.get(index);

    for (MyEntry<Integer, Integer> myentry : mylinkedlist) {
        if (myentry.key.equals(mykey)) {
            return myentry.value;
        }
    }
    return -1;
}
```



```
public void put(Integer mykey, Integer myvalue) {
    int index = myhash(mykey);

    LinkedList<MyEntry<Integer, Integer>> mylinkedlist = mytable.get(index);

    boolean found=false;
    for (MyEntry<Integer, Integer> myentry : mylinkedlist) {
        if (myentry.key.equals(mykey)) {
            myentry.value=myvalue;
            found=true;
        }
    }

    if(!found) {
        mylinkedlist.add(new MyEntry<Integer,Integer>(mykey,myvalue));
        currentSize++;
    }

}

public void remove(Integer mykey) {
    int index = myhash(mykey);

    LinkedList<MyEntry<Integer, Integer>> mylinkedlist = mytable.get(index);

    for (MyEntry<Integer, Integer> myentry : mylinkedlist) {
        if (myentry.key.equals(mykey)) {
            mylinkedlist.remove(myentry);
            currentSize--;
            break;
        }
    }

}
```




```
public int myhash(Integer key) {
    int hash = key.hashCode() % this.bucketSize;
    return hash;
}

public String toString() {
    return mytable.toString();
}
}

public class Lab6 {
    public static void main(String[] args) {

        MyHashMap mymap = new MyHashMap(7);

        System.out.println("-----1-----");
        System.out.println(mymap.size());
        System.out.println(mymap.isEmpty());
        System.out.println(mymap);

        mymap.put(50, 5050);
        mymap.put(21, 2121);
        mymap.put(58, 5858);
        mymap.put(17, 1717);
        mymap.put(15, 1515);
        mymap.put(49, 4949);
        mymap.put(56, 5656);
        mymap.put(22, 2222);
        mymap.put(23, 2323);
        mymap.put(25, 2525);

        mymap.put(23, 8888);
        mymap.put(25, 9999);
    }
}
```



```
        System.out.println("-----2-----");

        System.out.println(mymap.size());
        System.out.println(mymap.isEmpty());
        System.out.println(mymap);

        mymap.remove(23);
        mymap.remove(25);
        mymap.remove(50);

        System.out.println("-----3-----");

        System.out.println(mymap.size());
        System.out.println(mymap.isEmpty());
        System.out.println(mymap);

        System.out.println("-----4-----");
        System.out.println(mymap.get(50));
        System.out.println(mymap.get(49));

        System.out.println("-----5-----");
        mymap.clear();
        System.out.println(mymap.size());
        System.out.println(mymap.isEmpty());
        System.out.println(mymap);
    }
}
```

DSA

Module 8

Hashing

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

8. Hashing

- ◆ **Hashing** is a technique used to store and retrieve data as quickly as possible.
- ◆ Hashing is used to perform Optimal Searches
- ◆ Consider the following two Use-Cases.

Use-Case 1:

- ◆ Consider we have an array of integer numbers.
- ◆ I want to perform the following 3 Operations on the Array

Operation	Time Complexity
Search the number in the Array	$O(n)$
Insert the number in the Array	$O(n)$
Delete the number from the Array	$O(n)$

Use-Case 2:

- ◆ Consider we have an Binary Search Tree(BST).
- ◆ I want to perform the following 3 Operations on BST

Operation	Time Complexity
Search the number in the BST	$O(\log n)$
Insert the number in the BST	$O(\log n)$
Delete the number from the BST	$O(\log n)$

- ◆ Hashing works **best for these 3 Operations** and beats all remaining Data Structures.
- ◆ Hashing **provides $O(1)$ average time complexity** for 3 Important Operations - Search , Insert , Delete
- ◆ Hashing does exact key search
- ◆ Hashing is not useful when you want to find closet values or prefix searching etc.

8.1. Applications of Hashing

- 1) Dictionaries/Map
- 2) Sets
- 3) Database Indexing
- 4) Symbol Tables in Compilers
- 5) rypthography
- 6) Password Verification (MD5,SHA256)
- 7) Robin-Karp Algorithm
- Etc

8.2. Direct Address Table

- ◆ Consider we have 25 keys with the values ranging from 0 to 24.
- ◆ How can we implement the following 3 operations in $O(1)$ time complexity?
 - a) Search
 - b) Insert
 - c) Delete

Lab1.java

```
package com.jlccindia.hashing1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyTable {
    int mytable[];
    int capacity;

    public MyTable(int capacity) {
        this.capacity = capacity;
        mytable = new int[capacity];
    }

    public void insert(int key) {
        mytable[key]++;
    }
}
```



```
public void delete(int key) {
    mytable[key]--;
}
public int search(int key) {
    return mytable[key];
}
public void show() {
    for(int i=0;i<capacity;i++) {
        if(mytable[i]!=0) {
            System.out.println(i);
        }
    }
}
}
public class Lab1 {
    public static void main(String[] args) {
        MyTable mytable = new MyTable(25);
        mytable.insert(5);
        mytable.insert(2);
        mytable.insert(7);
        mytable.insert(9);
        mytable.insert(0);

        mytable.show();

        mytable.delete(0);
        mytable.delete(2);

        System.out.println("After Delete--");
        mytable.show();
        System.out.println("-----");

        System.out.println(mytable.search(5));
        System.out.println(mytable.search(2));
    }
}
```

Problems with above solution:

- ♦ It does not handle the following
 - a) Large Numbers
 - b) Negative Numbers
 - c) Floating Point numbers
 - d) Strings
 - e) Addresses (Any Object Address)
- ♦ We can use Hashing to solve the above problems

8.3. Exploring Hashing

- ♦ Hashing is a technique used to store and retrieve keys as quickly as possible
- ♦ Keys can be any of the following
 - a) Large Numbers
 - b) Negative Numbers
 - c) Floating Point numbers
 - d) Strings
 - e) Addresses (Any Object Address)
- ♦ You can have Universe of Keys -
- ♦ You can store the keys in Hashtable by converting the keys to small values.

8.3.1. Hash Functions

- ♦ Hash function is used to transform the key into the index.
- ♦ Ideally, **Hash Function** should map each possible key to a unique slot index, but it is difficult to achieve in practice.
- ♦ Given a collection of elements, a hash function that maps each item into a unique slot is referred to as a perfect hash function

8.3.2. Hash Function Examples

1) for numbers

- $\text{hash}(\text{key}) = \text{key} \% m$
- where m is table size (consider prime number)

2) for Strings

- $\text{str} = \text{"abc"}$
- $\text{hash}(\text{key}) = (\text{str}[0] * x^0 + \text{str}[1] * x^1 + \text{str}[2] * x^2 + \dots) \% m$

3) Universal Hashing:

- Group of Hash Functions and Pick one Randomly.

Example-1

keys = {53,51,55,52,50,54,49}

$\text{hash}(\text{key}) = \text{key} \% 7$

Example-2

keys = { 53,51,55,52,50,54,49}

$\text{hash}(\text{key}) = \text{key} \% 11$

8.3.3. Load Factor

Load Factor = Number of items stored in the table / Size of the table
--

Ex:

Load Factor = $7/11 \Rightarrow 0.66$

- ♦ This is the decision parameter used when we want to rehash or expand the existing hash table entries.
- ♦ This also helps us in determining the efficiency of the hashing function.
- ♦ That means, it tells whether the hash function is distributing the keys uniformly or not.

8.3.4. Characteristics of Good Hash Functions

- ♦ A good hash function should have the following characteristics:
 - 1) Should generate same value every time for the given key.
 - 2) Should generate the values from 0 to m-1
 - 3) Should generate fast - $O(1)$ for Integers and $O(\text{len})$ for Strings.
 - 4) Should uniformly distribute Large keys into Hashtable slots.
 - 5) Minimize collision
 - 6) Have a Low load factor for a given set of keys

8.4. Collisions

- ♦ Hash functions are used to map each key to a different Hashtable slots, but practically it is not possible to create such a hash function and the problem is called collision.
- ♦ Collision is the condition where two or more keys are stored in the Same Hashtable slots.

Collision Handling:

- ♦ If you know the keys in Advance then we can do the hashing perfectly (called Perfect Hashing).
- ♦ Collision is bound to happen if you dont know the keys in advance.
- ♦ **The process of finding an Alternative Hashtable slot is called collision resolution.**
- ♦ Even though hash tables have collision problems, they are more efficient in many cases compared to all other data structures, like search trees.
- ♦ There are a number of collision resolution techniques, and the most popular are **direct chaining** and **open addressing**.

A) Direct Chaining:

- Separate Chaining or Chaining

B) Open Addressing:

- Linear probing
- Quadratic probing
- Double hashing

8.5. Chaining

- ◆ When two or more Keys with same hash goes to same hashtable slot, these Keys are stored into a Separate Data Structure called a chain.
- ◆ Data Structures for Storing Chains:
 - a) Linked List
 - b) ArrayList
 - c) Self Balanced BST

a) Linked List

✓ Not Cache Friendly	Insert	->	$O(1)$
✓ Extra Space for Node Representation	delete	->	$O(1)$
	Search	->	$O(1)$

b) ArrayList

✓ Cache Friendly	Insert	->	$O(1)$
✓ No Extra Space	delete	->	$O(1)$
	Search	->	$O(1)$

c) Self Balanced BST

✓ Not Cache Friendly	Insert	->	$O(\log l)$
✓ Used from Java8	delete	->	$O(\log l)$
	Search	->	$O(\log l)$

Example-1

keys = { 50, 21, 58, 17, 15, 49, 56, 22, 23, 25}

hash(key) = key % 7

Lab2.java

```
package com.jlcindia.hashing2;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

class MyHashSet {

    int bucket;
    List<LinkedList<Integer>> mytable;

    public MyHashSet(int bucket) {
        this.bucket = bucket;
        this.mytable = new ArrayList<LinkedList<Integer>>(bucket);

        for(int i=0;i<bucket;i++) {
            mytable.add(new LinkedList<Integer>());
        }
    }

    public void insert(int key) {
        int index = key%bucket;
        if( !search(key))
            mytable.get(index).add(key);
    }
}
```



```
public void delete(int key) {
    int index = key%bucket;
    mytable.get(index).remove((Integer)key);
}

public boolean search(int key) {
    int index = key%bucket;
    return mytable.get(index).contains(key);
}

public String toString() {
    return mytable.toString();
}
}

public class Lab2 {
    public static void main(String[] args) {

        MyHashSet myset = new MyHashSet(7);

        myset.insert(50);
        myset.insert(21);
        myset.insert(58);
        myset.insert(17);
        myset.insert(15);
        myset.insert(49);
        myset.insert(56);
        myset.insert(22);
        myset.insert(23);
        myset.insert(25);
        myset.insert(23);
        myset.insert(25);

        System.out.println(myset);
    }
}
```



```
        myset.delete(23);
        myset.delete(25);
        myset.delete(50);

        System.out.println(myset);

        System.out.println(myset.search(50));
        System.out.println(myset.search(49));
    }
}
```

Lab3.java

```
package com.jlcindia.hashing3;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

//Implement HashSet using Chaining
// To Store Integers
// Updated

class MyHashSet {

    private int bucketSize;
    private int currentSize;
    List<LinkedList<Integer>> mytable;

    public MyHashSet(int bucketSize) {
        this.bucketSize = bucketSize;
        this.mytable = new ArrayList<LinkedList<Integer>>(bucketSize);
    }
}
```



```
        for(int i=0;i<bucketSize;i++) {
            mytable.add(new LinkedList<Integer>());
        }
    }

    public int size() {
        return this.currentSize;
    }

    public boolean isEmpty() {
        return currentSize==0;
    }

    public void clear() {

        this.mytable.clear();

        for(int i=0;i<bucketSize;i++) {
            mytable.add(new LinkedList<Integer>());
        }
        this.currentSize=0;
    }

    public void insert(Integer key) {
        int index = myhash(key);
        if( !search(key)) {
            mytable.get(index).add(key);
            currentSize++;
        }
    }

    public void delete(Integer key) {
        int index = myhash(key);
        mytable.get(index).remove((Integer)key);
        currentSize--;
    }
}
```




```
public boolean search(Integer key) {
    int index = myhash(key);
    return mytable.get(index).contains(key);
}

public int myhash(Integer key) {
    int hash = key.hashCode() % this.bucketSize;
    return hash;
}

public String toString() {
    return mytable.toString();
}
}

public class Lab3 {
    public static void main(String[] args) {

        MyHashSet myset = new MyHashSet(7);

        System.out.println("-----1-----");
        System.out.println(myset.size());
        System.out.println(myset.isEmpty());
        System.out.println(myset);

        myset.insert(50);
        myset.insert(21);
        myset.insert(58);
        myset.insert(17);
        myset.insert(15);
        myset.insert(49);
        myset.insert(56);
        myset.insert(22);
        myset.insert(23);
        myset.insert(25);
        myset.insert(23);
    }
}
```




```
myset.insert(25);

System.out.println("-----2-----");

System.out.println(myset.size());
System.out.println(myset.isEmpty());
System.out.println(myset);

myset.delete(23);
myset.delete(25);
myset.delete(50);

System.out.println("-----3-----");

System.out.println(myset.size());
System.out.println(myset.isEmpty());
System.out.println(myset);

System.out.println("-----4-----");
System.out.println(myset.search(50));
System.out.println(myset.search(49));

System.out.println("-----5-----");
myset.clear();
System.out.println(myset.size());
System.out.println(myset.isEmpty());
System.out.println(myset);
}
}
```



Lab4.java

```
package com.jlcindia.hashing4;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

//Implement HashSet using Chaining
//To Store Strings

class MyHashSet {

    private int bucketSize;
    private int currentSize;
    List<LinkedList<String>> mytable;

    public MyHashSet(int bucketSize) {
        this.bucketSize = bucketSize;
        this.mytable = new ArrayList<LinkedList<String>>(bucketSize);

        for(int i=0;i<bucketSize;i++) {
            mytable.add(new LinkedList<String>());
        }
    }

    public int size() {
        return this.currentSize;
    }

    public boolean isEmpty() {
        return currentSize==0;
    }
}
```



```
public void clear() {  
  
    this.mytable.clear();  
  
    for(int i=0;i<bucketSize;i++) {  
        mytable.add(new LinkedList<String>());  
    }  
    this.currentSize=0;  
}  
  
public void insert(String key) {  
    int index = myhash(key);  
    if( !search(key)) {  
        mytable.get(index).add(key);  
        currentSize++;  
    }  
}  
  
public void delete(String key) {  
    int index = myhash(key);  
    mytable.get(index).remove(key);  
    currentSize--;  
}  
  
public boolean search(String key) {  
    int index = myhash(key);  
    return mytable.get(index).contains(key);  
}  
  
public int myhash(String key) {  
    int hash = key.hashCode() % this.bucketSize;  
    return hash;  
}
```



```
        public String toString() {
            return mytable.toString();
        }
    }

    public class Lab4 {
        public static void main(String[] args) {

            MyHashSet myset = new MyHashSet(7);

            System.out.println("-----1-----");
            System.out.println(myset.size());
            System.out.println(myset.isEmpty());
            System.out.println(myset);

            myset.insert("aa");
            myset.insert("bb");
            myset.insert("cc");
            myset.insert("dd");
            myset.insert("ee");

            myset.insert("ab");
            myset.insert("bc");
            myset.insert("cd");
            myset.insert("de");
            myset.insert("ef");

            System.out.println("-----2-----");

            System.out.println(myset.size());
            System.out.println(myset.isEmpty());
            System.out.println(myset);
        }
    }
}
```



```
myset.delete("ab");
myset.delete("de");
myset.delete("dd");

System.out.println("-----3-----");

System.out.println(myset.size());
System.out.println(myset.isEmpty());
System.out.println(myset);

System.out.println("-----4-----");
System.out.println(myset.search("aa"));
System.out.println(myset.search("dd"));

System.out.println("-----5-----");
myset.clear();
System.out.println(myset.size());
System.out.println(myset.isEmpty());
System.out.println(myset);
}
}
```

8.6. Open Addressing

- ♦ In open addressing, all keys are stored in the hash table itself.
- ♦ This approach is also known as closed hashing.
- ♦ This procedure is based on probing.
- ♦ Collision is resolved by probing.

- ♦ Open Addressing can be implemented in 3 ways
 - 1) Linear probing
 - 2) Quadratic probing
 - 3) Double hashing ***

8.7. Linear Probing

- ◆ Interval between probes is fixed at 1.
- ◆ In linear probing, we search the hash table sequentially, starting from the original hash location.
- ◆ If a location is occupied, we check the next location.
- ◆ We wrap around from the last table location to the first table location if necessary.

Ex:

keys = {50, 51, 49, 16, 56, 15, 19}

hash(key) = key % 7

- ◆ Problem with linear probing is that table items form clusters together in the hash table.
- ◆ This means that the table contains groups of consecutively occupied locations that are called clustering.
- ◆ Clustering causes long probe searches and therefore decreases the overall efficiency.

8.8. Quadratic Probing

- ◆ Interval between probes increases proportionally to the hash value
- ◆ Problem of Clustering can be reduced if we use the quadratic probing method.
- ◆ In quadratic probing, we start from the original hash location.
- ◆ If a location is occupied, we check the locations $i + 1^2$, $i + 2^2$, $i + 3^2$, $i + 4^2$...

Ex:

keys = {31, 19, 2, 13, 25, 24, 21, 9}

hash(key) = key % 11

- ◆ Even though clustering is avoided by quadratic probing, still there are chances of thin clustering.

8.9. Double Hashing

- ◆ Interval between probes is computed by another hash function.
- ◆ Double hashing reduces clustering in a better way.
- ◆ Increments for the probing sequence are computed by using a second hash function.
- ◆ In Double hashing, We first probe the location $h1(key)$.
- ◆ If the location is occupied, we probe the location $(h1(key) + i * h2(key)) \% m$

For Example, as follows

$$(h1(key) + i * h2(key)) \% m$$

$$(h1(key) + 1 * h2(key)) \% m$$

$$(h1(key) + 2 * h2(key)) \% m$$

$$(h1(key) + 3 * h2(key)) \% m$$

etc

Example-1

keys = {49, 63, 56, 52, 54, 48}

$m = 7$ (0 to 6)

$h1(key) = key \% 7$

$h2(key) = 6 - (key \% 6)$

Lab5.java

```
package com.jlcindia.hashing5;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Implement HashSet using Double Hashing
```



```
class MyHashSet {  
  
    private int bucketSize;  
    private int currentSize;  
    private Integer[] mytable;  
  
    public MyHashSet(int bucketSize) {  
        this.bucketSize = bucketSize;  
        this.mytable = new Integer[bucketSize];  
        this.currentSize = 0;  
    }  
  
    public int size() {  
        return this.currentSize;  
    }  
  
    public boolean isEmpty() {  
        return currentSize == 0;  
    }  
  
    public void clear() {  
        this.currentSize = 0;  
        for (int i = 0; i < bucketSize; i++) {  
            mytable[i] = null;  
        }  
    }  
  
    public void add(Integer key) {  
        int index = myhash1(key);  
        if (mytable[index] != null) {  
            index = myhash(key);  
        }  
  
        mytable[index] = key;  
        this.currentSize++;  
    }  
}
```




```
public void remove(Integer key) {

    int index = myhash1(key);
    if (mytable[index] != key) {
        index = myhash99(key);
    }

    if (mytable[index] == key) {
        mytable[index] = null;
        this.currentSize--;
    }

}

public boolean contains(Integer key) {
    int index = myhash1(key);
    if (mytable[index] != key) {
        index = myhash99(key);
    }

    return mytable[index] == key;
}

public int myhash1(Integer key) {
    int hash1 = key.hashCode() % this.bucketSize;
    return hash1;
}

public int myhash2(Integer key) {
    int hash2 = (this.bucketSize - 1) - key.hashCode() % (this.bucketSize - 1);
    return hash2;
}
```



```
public int myhash(Integer key) {
    int hash1 = myhash1(key);
    int hash2 = myhash2(key);

    int myhash = 0;
    for (int i = 1; i <= this.bucketSize; i++) {
        myhash = (hash1 + i * hash2) % this.bucketSize;
        if (mytable[myhash] == null) {
            return myhash;
        }
    }
    return myhash;
}

public int myhash99(Integer key) {
    int hash1 = myhash1(key);
    int hash2 = myhash2(key);

    int myhash = 0;
    for (int i = 1; i <= this.bucketSize; i++) {
        myhash = (hash1 + i * hash2) % this.bucketSize;
        if (mytable[myhash] == key) {
            return myhash;
        }
    }
    return myhash;
}

public String toString() {
    String str = "[";
    for (int i = 0; i < this.bucketSize; i++) {
        if (mytable[i] != null) {
            str += mytable[i] + "\t";
        }
    }
    str = str + "]";
    return str;
}
}
```



```
public class Lab5 {  
    public static void main(String[] args) {  
  
        MyHashSet myset = new MyHashSet(7);  
  
        System.out.println("-----1-----");  
        System.out.println(myset.size());  
        System.out.println(myset.isEmpty());  
        System.out.println(myset);  
  
        myset.add(49);  
        myset.add(63);  
        myset.add(56);  
        myset.add(52);  
        myset.add(54);  
        myset.add(48);  
  
        System.out.println("-----2-----");  
        System.out.println(myset.size());  
        System.out.println(myset.isEmpty());  
        System.out.println(myset);  
  
        myset.remove(49);  
        myset.remove(48);  
        myset.remove(99);  
  
        System.out.println("-----3-----");  
        System.out.println(myset.size());  
        System.out.println(myset.isEmpty());  
        System.out.println(myset);  
  
        System.out.println("-----4-----");  
        System.out.println(myset.contains(49));  
        System.out.println(myset.contains(52));  
    }  
}
```



```
        myset.clear();
        System.out.println("-----5-----");
        System.out.println(myset.size());
        System.out.println(myset.isEmpty());
        System.out.println(myset);
    }
}
```

DSA

Module 9

LinkedLists

Author
Srinivas Dande





Java Learning Center

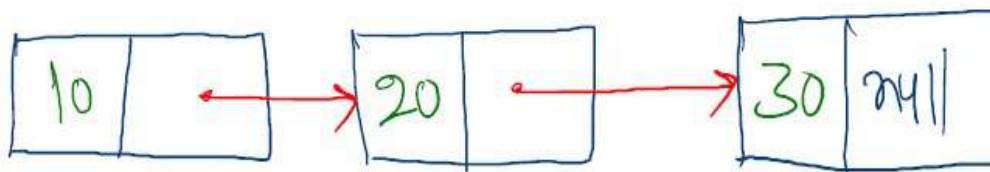
No.1 In Java Training & placement



9. Introduction to LinkedLists

- ♦ Linked List is Linear Data Structure.
- ♦ Linked List is Used to store collection of elements.
- ♦ LinkedLists elements can be accessed in sequential order only.
- ♦ LinkedLists elements will be stored with Node representation.

Ex:



- ♦ Linked list has the following properties.
 - ✓ Successive elements are connected by pointers
 - ✓ The last element points to NULL
 - ✓ Can grow or shrink in size during execution of a program
 - ✓ Does not waste memory space. It allocates memory as list grows but takes some extra memory for pointers.
- ♦ Types of Linked Lists
 - ✓ Singly Linked Lists
 - ✓ Doubly Linked Lists
 - ✓ Circular Linked Lists

9.1 Why LinkedLists

- ♦ There are many other data structures that do the same thing as linked lists.
- ♦ Before discussing linked lists it is important to understand the difference between linked lists and arrays.
- ♦ Both linked lists and arrays are used to store collections of data, and since both are used for the same purpose, we need to differentiate their usage.
- ♦ That means in which cases arrays are suitable and in which cases linked lists are suitable.

Advantages of Arrays:

- ♦ Arrays are Used to store collection of elements of Same type
- ♦ Arrays elements are stored in contiguous locations
- ♦ Arrays elements will be stored with index representaiton.
- ♦ Arrays allow Random access to elements.
- ♦ Search Operation is faster because we can apply Binary Search.
- ♦ Arrays are Cache Friendly.

DisAdvantages of Arrays:

- ♦ Arrays are fixed size.
It means Once you create the array, you can not increse the size.
- ♦ Momory waste when you allocate more space and store less elements.
- ♦ Momory shortage when you allocate less space and store more elements.
You need to resize the array when you want to store more elements.
Resizing is an expensive again.
- ♦ Insert Operation is expensive because it takes time to shift elements
- ♦ Delete Operation is expensive because it takes time to shift elements

Advantages of LinkedLists:

- ♦ LinkedLists are Used to store collection of elements
- ♦ LinkedLists elements will not be stored in contiguous locations
- ♦ LinkedLists elements will be stored with Node representaiton.
- ♦ LinkedLists allow Only sequential acess to elements, No Random access.
- ♦ Insert Operation is faster
- ♦ Delete Operation is faster
- ♦ LinkedLists are dynamic. i.e No Node Creation inadvance

DisAdvantages of LinkedLists:

- ♦ No Random access.
- ♦ Search Operation is expensive
we can not apply Binary Search on LinkedList because of Sequential Access.
- ♦ Momoeiy wastage for Node addresses.
- ♦ LinkedLists are Not Cache Friendly.

Comparison of Linked Lists with Arrays & Dynamic Arrays

<u>Parameter</u>	<u>Arrays</u>	<u>ArrayList</u>	<u>LinkedList</u>
Get Element	$O(1)$	$O(1)$	$O(n)$
Insert at Beginning	$O(n)$	$O(n)$	$O(1)$
Delete at Beginning	$O(n)$	$O(n)$	$O(1)$
Insert at End	$O(1)$	$O(1)$	$O(n)$
Delete at End	$O(1)$	$O(1)$	$O(n)$
Insert at Middle	$O(n)$	$O(n)$	$O(n)$
Delete at Middle	$O(n)$	$O(n)$	$O(n)$

Use-cases:

1. I am writing some programs which has to store very large number of elements. I may not get the contiguous memory because memory may be available as chunks. Which one to choose to store large number of elements.

Ans: LinkedList

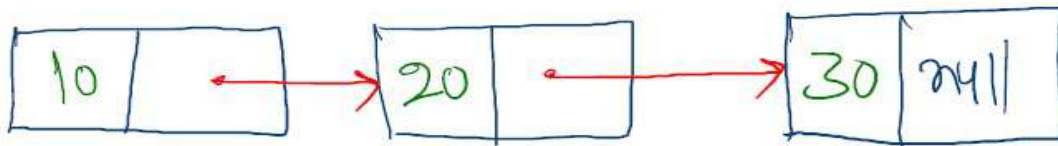
2. Stacks and Queues can be implemented with Arrays and LinkedLists.
Ex: I want to Implement Round Robin Scheduling.

Ans: Use Queue with LinkedLists

9.2. Singly Linked List

- ◆ Singly Linked List consists of a number of nodes
- ◆ Each Node has will have 2 parts
 - a) Data
 - b) Address of Next Node
- ◆ Address of the last node in the list is NULL, which indicates the end of the list.

Ex:



- ◆ Following is a type declaration for a linked list:

```

public class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
  
```

- ◆ Main Linked Lists Operations
 - ✓ Traverse: Access the elements in the list
 - ✓ Count: returns the number of elements in the list
 - ✓ Find nth node from the end of the list
 - ✓ Insert: inserts an element into the list
 - ✓ Delete: removes the specified position element from the list

9.2.1. Traverse the LinkedList using Iterative Style

Lab1.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {

    static void displayList(Node headNode) {

        Node currentNode = headNode;

        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        displayList(head);

    }
}
```

9.2.2. Traverse the LinkedList using Recursive Style

Lab2.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab2 {

    static void displayList(Node currentNode) {

        //Base Condition
        if(currentNode==null)
            return ;

        System.out.print(currentNode.data+"\t");
        displayList(currentNode.next); //Recursive Call

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        displayList(head);
    }
}
```



9.2.3. Count the No. of Nodes in the LinkedList

Lab3.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    static int length(Node headNode) {

        int length = 0;
        Node currentNode = headNode;
        while (currentNode != null) {
            length++;
            currentNode = currentNode.next;
        }
        return length;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        int len = length(head);
        System.out.println(len);

        Node myhead = null;
        int mylen = length(myhead);
        System.out.println(mylen);
    }
}
```



9.2.4. Insert the Node at the begining of LinkedList

Lab4.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    static Node insertFirst(Node headNode,int data) {

        Node temp = new Node(data);
        temp.next = headNode;
        return temp;
    }
    static void displayList(Node headNode) {
        Node currentNode = headNode;
        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }
        System.out.println("\n");
    }
    public static void main(String[] args) {

        Node head = null;
        head = insertFirst(head,10);
        head = insertFirst(head,20);
        head = insertFirst(head,30);
        head = insertFirst(head,40);
        head = insertFirst(head,50);

        displayList(head);
    }
}
```



9.2.5. Insert the Node at the end of LinkedList

Lab5.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab5 {
    static Node insertLast(Node headNode,int data) {

        Node temp = new Node(data);
        if(headNode==null){
            return temp;
        }

        //To Reach the Last Node of LL
        Node currentNode = headNode;
        while(currentNode.next != null) {
            currentNode=currentNode.next;
        }

        currentNode.next = temp;

        return headNode;
    }

    static void displayList(Node headNode) {
        Node currentNode = headNode;
        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }

        System.out.println("\n");
    }
}
```




```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
  
    displayList(head);  
}
```

9.2.6. Insert the Node at given position of LinkedList

Lab6.java

```
package com.jlcindia.linkedlist;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public class Lab6 {  
    static Node insert(Node headNode,int position, int data) {  
  
        Node temp = new Node(data);  
  
        if(position==1){  
            temp.next = headNode;  
            return temp;  
        }  
  
        Node currentNode = headNode;  
        for(int i=1;i<=position-2 && currentNode != null;i++) {  
            currentNode=currentNode.next;  
        }  
    }  
}
```




```
        if(currentNode==null) {
            return headNode;
        }

        temp.next=currentNode.next;
        currentNode.next = temp;

        return headNode;
    }
    static void displayList(Node headNode) {
        //Copy from Lab5
    }
    static Node insertLast(Node headNode,int data) {
        //Copy from Lab5
    }
    public static void main(String[] args) {
        Node head = null;
        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);

        displayList(head);
        System.out.println("-----");

        head = insert(head,3,99);
        displayList(head);
        System.out.println("-----");
        head = insert(head,4,88);
        displayList(head);
    }
}
```



9.2.7. Delete the First Node of LinkedList

Lab7.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab7 {

    static Node deleteFirst(Node headNode) {

        if(headNode==null)
            return null;

        Node newHead = headNode.next;
        headNode.next = null;
        return newHead;
    }

    static Node insertLast(Node headNode,int data) {
        //Copy from Lab5
    }

    static void displayList(Node headNode) {
        //Copy from Lab5
    }

    public static void main(String[] args) {

        Node head = null;
        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);
    }
}
```



```
        displayList(head);
        System.out.println("-----");

        head= deleteFirst(head);
        displayList(head);
        System.out.println("-----");
        head= deleteFirst(head);
        displayList(head);
    }
}
```

9.2.8. Delete the Last Node of LinkedList

Lab8.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab8 {

    static Node deleteLast(Node headNode) {

        if(headNode==null)
            return null;

        Node newHead = headNode.next;
        headNode.next = null;
        return newHead;
    }

    static Node insertLast(Node headNode,int data) {
        //Copy from Lab5
    }
}
```



```
static void displayList(Node headNode) {  
    //Copy from Lab5  
}  
  
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
  
    displayList(head);  
    System.out.println("-----");  
  
    head= deleteLast(head);  
    displayList(head);  
    System.out.println("-----");  
    head= deleteLast(head);  
    displayList(head);  
}  
}
```

DSA

Module 9

LinkedLists

Author
Srinivas Dande





Java Learning Center

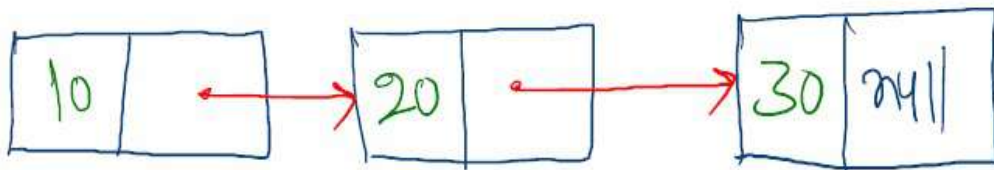
No.1 In Java Training & placement



9. Introduction to LinkedLists

- ♦ Linked List is Linear Data Structure.
- ♦ Linked List is Used to store collection of elements.
- ♦ LinkedLists elements can be accessed in sequential order only.
- ♦ LinkedLists elements will be stored with Node representation.

Ex:



- ♦ Linked list has the following properties.
 - ✓ Successive elements are connected by pointers
 - ✓ The last element points to NULL
 - ✓ Can grow or shrink in size during execution of a program
 - ✓ Does not waste memory space. It allocates memory as list grows but takes some extra memory for pointers.
- ♦ Types of Linked Lists
 - ✓ Singly Linked Lists
 - ✓ Doubly Linked Lists
 - ✓ Circular Linked Lists

9.1 Why LinkedLists

- ♦ There are many other data structures that do the same thing as linked lists.
- ♦ Before discussing linked lists it is important to understand the difference between linked lists and arrays.
- ♦ Both linked lists and arrays are used to store collections of data, and since both are used for the same purpose, we need to differentiate their usage.
- ♦ That means in which cases arrays are suitable and in which cases linked lists are suitable.

Advantages of Arrays:

- ♦ Arrays are Used to store collection of elements of Same type
- ♦ Arrays elements are stored in contiguous locations
- ♦ Arrays elements will be stored with index representaiton.
- ♦ Arrays allow Random access to elements.
- ♦ Search Operation is faster because we can apply Binary Search.
- ♦ Arrays are Cache Friendly.

DisAdvantages of Arrays:

- ♦ Arrays are fixed size.
It means Once you create the array, you can not increse the size.
- ♦ Momory waste when you allocate more space and store less elements.
- ♦ Momory shortage when you allocate less space and store more elements.
You need to resize the array when you want to store more elements.
Resizing is an expensive again.
- ♦ Insert Operation is expensive because it takes time to shift elements
- ♦ Delete Operation is expensive because it takes time to shift elements

Advantages of LinkedLists:

- ♦ LinkedLists are Used to store collection of elements
- ♦ LinkedLists elements will not be stored in contiguous locations
- ♦ LinkedLists elements will be stored with Node representaiton.
- ♦ LinkedLists allow Only sequential acess to elements, No Random access.
- ♦ Insert Operation is faster
- ♦ Delete Operation is faster
- ♦ LinkedLists are dynamic. i.e No Node Creation inadvance

DisAdvantages of LinkedLists:

- ♦ No Random access.
- ♦ Search Operation is expensive
we can not apply Binary Search on LinkedList because of Sequential Access.
- ♦ Momoeiy wastage for Node addresses.
- ♦ LinkedLists are Not Cache Friendly.

Comparison of Linked Lists with Arrays & Dynamic Arrays

<u>Parameter</u>	<u>Arrays</u>	<u>ArrayList</u>	<u>LinkedList</u>
Get Element	$O(1)$	$O(1)$	$O(n)$
Insert at Beginning	$O(n)$	$O(n)$	$O(1)$
Delete at Beginning	$O(n)$	$O(n)$	$O(1)$
Insert at End	$O(1)$	$O(1)$	$O(n)$
Delete at End	$O(1)$	$O(1)$	$O(n)$
Insert at Middle	$O(n)$	$O(n)$	$O(n)$
Delete at Middle	$O(n)$	$O(n)$	$O(n)$

Use-cases:

1. I am writing some programs which has to store very large number of elements. I may not get the contiguous memory because memory may be available as chunks. Which one to choose to store large number of elements.

Ans: LinkedList

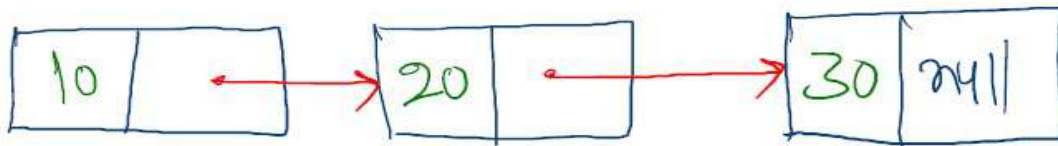
2. Stacks and Queues can be implemented with Arrays and LinkedLists.
Ex: I want to Implement Round Robin Scheduling.

Ans: Use Queue with LinkedLists

9.2. Singly Linked List

- ◆ Singly Linked List consists of a number of nodes
- ◆ Each Node has will have 2 parts
 - a) Data
 - b) Address of Next Node
- ◆ Address of the last node in the list is NULL, which indicates the end of the list.

Ex:



- ◆ Following is a type declaration for a linked list:

```

public class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
  
```

- ◆ Main Linked Lists Operations
 - ✓ Traverse: Access the elements in the list
 - ✓ Count: returns the number of elements in the list
 - ✓ Find nth node from the end of the list
 - ✓ Insert: inserts an element into the list
 - ✓ Delete: removes the specified position element from the list

9.2.1. Traverse the LinkedList using Iterative Style

Lab1.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {

    static void displayList(Node headNode) {

        Node currentNode = headNode;

        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        displayList(head);

    }
}
```

9.2.2. Traverse the LinkedList using Recursive Style

Lab2.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab2 {

    static void displayList(Node currentNode) {

        //Base Condition
        if(currentNode==null)
            return ;

        System.out.print(currentNode.data+"\t");
        displayList(currentNode.next); //Recursive Call

        System.out.println("\n");
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        displayList(head);

    }
}
```



9.2.3. Count the No. of Nodes in the LinkedList

Lab3.java

```
package com.jlccindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    static int length(Node headNode) {

        int length = 0;
        Node currentNode = headNode;
        while (currentNode != null) {
            length++;
            currentNode = currentNode.next;
        }
        return length;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        int len = length(head);
        System.out.println(len);

        Node myhead = null;
        int mylen = length(myhead);
        System.out.println(mylen);
    }
}
```



9.2.4. Insert the Node at the begining of LinkedList

Lab4.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    static Node insertFirst(Node headNode,int data) {

        Node temp = new Node(data);
        temp.next = headNode;
        return temp;
    }
    static void displayList(Node headNode) {
        Node currentNode = headNode;
        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }
        System.out.println("\n");
    }
    public static void main(String[] args) {

        Node head = null;
        head = insertFirst(head,10);
        head = insertFirst(head,20);
        head = insertFirst(head,30);
        head = insertFirst(head,40);
        head = insertFirst(head,50);

        displayList(head);
    }
}
```

9.2.5. Insert the Node at the end of LinkedList

Lab5.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab5 {
    static Node insertLast(Node headNode,int data) {

        Node temp = new Node(data);
        if(headNode==null){
            return temp;
        }

        //To Reach the Last Node of LL
        Node currentNode = headNode;
        while(currentNode.next != null) {
            currentNode=currentNode.next;
        }

        currentNode.next = temp;

        return headNode;
    }

    static void displayList(Node headNode) {
        Node currentNode = headNode;
        while(currentNode != null) {
            System.out.print(currentNode.data+"\t");
            currentNode=currentNode.next;
        }

        System.out.println("\n");
    }
}
```




```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
  
    displayList(head);  
}
```

9.2.6. Insert the Node at given position of LinkedList

Lab6.java

```
package com.jlcindia.linkedlist;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public class Lab6 {  
    static Node insert(Node headNode,int position, int data) {  
  
        Node temp = new Node(data);  
  
        if(position==1){  
            temp.next = headNode;  
            return temp;  
        }  
  
        Node currentNode = headNode;  
        for(int i=1;i<=position-2 && currentNode != null;i++) {  
            currentNode=currentNode.next;  
        }  
    }  
}
```




```
        if(currentNode==null) {
            return headNode;
        }

        temp.next=currentNode.next;
        currentNode.next = temp;

        return headNode;
    }
    static void displayList(Node headNode) {
        //Copy from Lab5
    }
    static Node insertLast(Node headNode,int data) {
        //Copy from Lab5
    }
    public static void main(String[] args) {
        Node head = null;
        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);

        displayList(head);
        System.out.println("-----");

        head = insert(head,3,99);
        displayList(head);
        System.out.println("-----");
        head = insert(head,4,88);
        displayList(head);
    }
}
```



9.2.7. Delete the First Node of LinkedList

Lab7.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab7 {

    static Node deleteFirst(Node headNode) {

        if(headNode==null)
            return null;

        Node newHead = headNode.next;
        headNode.next = null;
        return newHead;
    }

    static Node insertLast(Node headNode,int data) {
        //Copy from Lab5
    }

    static void displayList(Node headNode) {
        //Copy from Lab5
    }

    public static void main(String[] args) {

        Node head = null;
        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        head = insertLast(head,50);
    }
}
```



```
        displayList(head);  
        System.out.println("-----");  
  
        head= deleteFirst(head);  
        displayList(head);  
        System.out.println("-----");  
        head= deleteFirst(head);  
        displayList(head);  
    }  
}
```



9.2.8. Delete the Last Node of LinkedList

Lab8.java

```
package com.jlcindia.linkedlist.singly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab8 {

    static Node deleteLast(Node headNode) {

        if (headNode == null)
            return null;

        if(headNode.next==null) {
            return null;
        }

        Node currentNode = headNode;
        while (currentNode.next.next != null) {
            currentNode = currentNode.next;
        }

        currentNode.next = null;
        return headNode;
    }
    static Node insertLast(Node headNode, int data) {

        //Copy from Previous Labs
    }

    static void displayList(Node headNode) {

        //Copy from Previous Labs
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    displayList(head);  
    System.out.println("-----");  
  
    head = deleteLast(head);  
    displayList(head);  
    System.out.println("-----");  
    head = deleteLast(head);  
    displayList(head);  
}  
}
```



9.2.9. Delete the Node at given position of S.L.L

Lab9.java

```
package com.jlcindia.linkedlist.singly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab9 {
    static Node delete(Node headNode, int postion) {

        if (headNode == null)
            return null;

        if (headNode.next == null)
            return null;

        if (postion == 1) {
            return headNode.next;
        }

        Node currentCode = headNode;

        for (int i = 1; i <= postion - 2 && currentCode!=null ; i++) {
            currentCode = currentCode.next;
        }

        if(currentCode==null) {
            return headNode;
        }

        currentCode.next=currentCode.next.next;

        return headNode;
    }
}
```



```
static Node insertLast(Node headNode, int data) {  
  
    //Copy from Previous Labs  
}  
  
static void displayList(Node headNode) {  
  
    //Copy from Previous Labs  
}  
public static void main(String[] args) {  
  
    Node head = null;  
  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    displayList(head);  
    System.out.println("-----");  
  
    head = delete(head, 2);  
    displayList(head);  
    System.out.println("-----");  
    head = delete(head, 3);  
    displayList(head);  
  
    }  
}
```



9.2.10. Search the Node in LinkedList

Lab10.java

```
package com.jlcindia.linkedlist.singly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab10 {

    static int search(Node headNode, int element) {

        int position=1;

        Node currentNode = headNode;
        while(currentNode!=null) {
            if(currentNode.data==element) {
                return position;
            }else {
                currentNode=currentNode.next;
                position++;
            }
        }

        return -1;
    }

    static Node insertLast(Node headNode, int data) {

        //Copy from Previous Labs
    }

    static void displayList(Node headNode) {

        //Copy from Previous Labs
    }
}
```




```
public static void main(String[] args) {  
  
    Node head = null;  
  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    displayList(head);  
    System.out.println("-----");  
  
    int x = search(head, 20);  
    System.out.println(x);  
    System.out.println("-----");  
    x = search(head, 40);  
    System.out.println(x);  
    System.out.println("-----");  
    x = search(head, 70);  
    System.out.println(x);  
  
}  
  
}
```

9.2.11. Design MySinglyLinkedList

Lab11.java

```
package com.jlcindia.linkedlist.singly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
public class MySinglyLinkedList {

    Node headNode = null;
    int nodeCount = 0;

    public String toString() {
        if (this.headNode == null) {
            return "[]";
        }

        String str = "[";
        Node currentNode = this.headNode;
        while (currentNode != null) {
            str = str + "" + currentNode.data + ", ";
            currentNode = currentNode.next;
        }
        str = str.substring(0, str.length() - 2);
        str = str + "]";
        return str;
    }
}
```



```
public boolean isEmpty() {  
  
    return nodeCount == 0;  
}  
  
public void clear() {  
  
    Node currentNode=headNode;  
    while(currentNode!=null) {  
        Node temp = currentNode.next;  
  
        currentNode.next=null;  
        currentNode=temp;  
    }  
  
    this.headNode = null;  
    this.nodeCount = 0;  
}  
  
public int size() {  
  
    return nodeCount;  
}  
  
public void insertFirst(int data) {  
  
    Node temp = new Node(data);  
    temp.next = this.headNode;  
    this.headNode = temp;  
    nodeCount++;  
  
}
```



```
public void insertLast(int data) {
```

```
    Node temp = new Node(data);
    if (this.headNode == null) {
        this.headNode = temp;
    }

    // To Reach the Last Node of LL
    Node currentNode = headNode;
    while (currentNode.next != null) {
        currentNode = currentNode.next;
    }

    currentNode.next = temp;
    nodeCount++;
}
```

```
public void insert(int position, int data) {
```

```
    Node temp = new Node(data);

    if (headNode == null) {
        this.headNode = temp;
        return;
    }

    if (position == 1) {
        temp.next = headNode;
        this.headNode = temp;
        return;
    }

    Node currentNode = headNode;
    for (int i = 1; i <= position - 2 && currentNode != null; i++) {
        currentNode = currentNode.next;
    }
}
```



```
        if (currentNode == null) {
            return;
        }

        temp.next = currentNode.next;
        currentNode.next = temp;

        nodeCount++;
    }

    public void deleteFirst() {

        if (this.headNode == null) {
            return;
        }

        this.headNode = headNode.next;
        nodeCount--;
    }

    public void deleteLast() {

        // When No Nodes
        if (this.headNode == null)
            return;

        // When One Node is there
        if (headNode.next == null) {
            this.headNode = null;
            nodeCount--;
            return;
        }

        // When two or more Nodes are there
        Node currentNode = headNode;
        while (currentNode.next.next != null) {
```



```
        currentNode = currentNode.next;
    }

    currentNode.next = null;
    nodeCount--;

}

public void delete(int postion) {

    // 1.Empty List
    if (headNode == null) {
        this.headNode = null;
        return;
    }

    // 2.List with One Node
    if (headNode.next == null) {
        this.headNode = null;
        nodeCount--;
        return;
    }

    // 3. List with 2 or more Nodes
    // Deleting 1st Node
    if (postion == 1) {
        this.headNode = headNode.next;
        nodeCount--;
        return;
    }

    // 4. List with 2 or more Nodes
    Node currentCode = headNode;

    for (int i = 1; i <= postion - 2 && currentCode != null; i++) {
        currentCode = currentCode.next;
    }
}
```



```
// 5. Position is not Present
if (currentCode == null) {
    return;
}

currentCode.next = currentCode.next.next;
nodeCount--;
}

public int search(int element) {

    int position = 1;

    Node currentNode = this.headNode;
    while (currentNode != null) {
        if (currentNode.data == element) {
            return position;
        } else {
            currentNode = currentNode.next;
            position++;
        }
    }

    return -1;

}
}
```



```
public class Lab11 {  
  
    public static void main(String[] args) {  
  
        MySinglyLinkedList mylist=new MySinglyLinkedList();  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
  
        mylist.insertFirst(11);  
        mylist.insertFirst(22);  
        mylist.insertFirst(33);  
  
        System.out.println("-----");  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
  
        mylist.insertLast(40);  
        mylist.insertLast(50);  
  
        System.out.println("-----");  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
  
        mylist.insert(4, 99);  
        mylist.insert(5, 88);  
  
        System.out.println("-----");  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
  
        mylist.deleteFirst();  
        System.out.println("-----");  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
    }  
}
```




```
mylist.deleteLast();
System.out.println("-----");
System.out.println(mylist);
System.out.println(mylist.size());

mylist.delete(2);
System.out.println("-----");
System.out.println(mylist);
System.out.println(mylist.size());
System.out.println("-----");

System.out.println(mylist.search(99));
System.out.println(mylist.search(77));

System.out.println("-----");
System.out.println(mylist.isEmpty());
System.out.println(mylist.size());

mylist.clear();

System.out.println(mylist.isEmpty());
System.out.println(mylist.size());

}

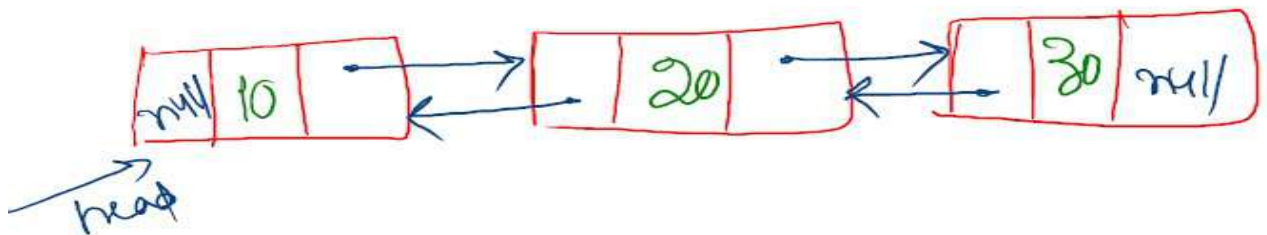
}
```



9.3. Doubly Linked List

- ◆ Doubly Linked List consists of a number of nodes
- ◆ Each Node has will have 3 parts
 - a) Address of Previous Node
 - b) Data
 - c) Address of Next Node
- ◆ Next Address of the Last node in the list is NULL, which indicates that No Next Element.
- ◆ Previous Address of the First/Head node in the list is NULL, which indicates that No Previous element.

Ex:



- ◆ Following is a type declaration for a linked list:

```
public class Node {  
  
    int data;  
    Node next;  
    Node prev;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
        this.prev=null;  
    }  
}
```

- ◆ Main Doubly Linked Lists Operations
 - ✓ Traverse: Access the elements in the List
 - ✓ Count: returns the number of elements in the List
 - ✓ Find nth node from the end of the List
 - ✓ Insert: inserts an element into the List
 - ✓ Delete: removes the specified position element from the List
 - ✓ Reverse the List

9.3.1. Traverse the Doubly LinkedList in forward order

Lab12.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Node {

    int data;
    Node next;
    Node prev;

    Node(int data) {
        this.data = data;
        this.next = null;
        this.prev=null;
    }

}
```



```
public class Lab12 {

    static String toString(Node headNode) {
        if (headNode == null) {
            return "[]";
        }

        String str = "[";
        Node currentNode = headNode;
        while (currentNode != null) {
            str = str + "" + currentNode.data + " , ";
            currentNode = currentNode.next;
        }
        str = str.substring(0, str.length() - 2);
        str = str + "]";

        return str;
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        Node node2 = new Node(20);
        Node node3 = new Node(30);

        head.next = node2;
        node2.prev = head;

        node2.next = node3;
        node3.prev = node2;

        System.out.println(toString(head));
    }
}
```

9.3.2. Traverse the Doubly LinkedList in Reverse order

Lab13.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab13 {

    static String toString(Node headNode) {

        if (headNode == null) {
            return "[]";
        }

        Node tailNode = headNode;

        while(tailNode.next!=null) {
            tailNode=tailNode.next;
        }

        String str = "[";
        Node currentNode = tailNode;
        while (currentNode != null) {
            str = str + "" + currentNode.data + " , ";
            currentNode = currentNode.prev;
        }

        str = str.substring(0, str.length() - 2);
        str = str + "]";

        return str;
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = new Node(10);  
    Node node2 = new Node(20);  
    Node node3 = new Node(30);  
  
    head.next = node2;  
    node2.prev = head;  
  
    node2.next = node3;  
    node3.prev = node2;  
  
    System.out.println(toString(head));  
  
}  
}
```



9.3.3. Insert the Node at beginning of D.L.L

Lab14.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab14 {

    static Node insertFirst(Node headNode,int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            headNode=temp; //head node changed
            return headNode;
        }

        //2.List with 1 or more Nodes
        temp.next=headNode;
        headNode.prev=temp;

        headNode=temp; //head node changed

        return headNode;
    }
}
```



```
static String toString(Node headNode) {

    if (headNode == null) {
        return "[]";
    }

    String str = "[";
    Node currentNode = headNode;
    while (currentNode != null) {
        str = str + "" + currentNode.data + ", ";
        currentNode = currentNode.next;
    }
    str = str.substring(0, str.length() - 2);
    str = str + "]";

    return str;
}

public static void main(String[] args) {

    Node head=null;
    head = insertFirst(head,10);
    head = insertFirst(head,20);
    head = insertFirst(head,30);

    System.out.println(toString(head));

}

}
```




9.3.4. Insert the Node at end of D.L.L

Lab15.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab15 {

    static Node insertLast(Node headNode,int data) {

        Node temp = new Node(data);

        //1.Empty List
        if(headNode==null) {
            headNode=temp; //head node changed
            return headNode;
        }

        //2.List with 1 or more Nodes
        Node currentNode = headNode;
        while (currentNode.next != null) {
            currentNode = currentNode.next;
        }

        currentNode.next=temp;
        temp.prev=currentNode;
        return headNode;
    }
}
```



```
static String toString(Node headNode) {

    if (headNode == null) {
        return "[]";
    }

    String str = "[";
    Node currentNode = headNode;
    while (currentNode != null) {
        str = str + "" + currentNode.data + ", ";
        currentNode = currentNode.next;
    }
    str = str.substring(0, str.length() - 2);
    str = str + "]";

    return str;
}

public static void main(String[] args) {

    Node head=null;
    head = insertLast(head,10);
    head = insertLast(head,20);
    head = insertLast(head,30);
    head = insertLast(head,99);

    System.out.println(toString(head));

}

}
```



9.3.5. Insert the Node at given position in D.L.L

Lab16.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab16 {

    static Node insert(Node headNode, int position, int data) {

        //1. Create New Node
        Node temp = new Node(data);

        //2.when head is null
        if(headNode==null) {
            return temp;
        }

        //3. If position is 1
        if(position==1){
            temp.next = headNode;
            headNode.prev=temp;
            return temp;
        }

        //4. Move the position -1 Node
        Node currentNode = headNode;
        for(int i=1;i<=position-2 && currentNode != null;i++) {
            currentNode=currentNode.next;
        }

        //5. If Position is out of range
        if(currentNode==null) {
            return headNode;
        }
    }
}
```



```
//6. Insert at given Position
temp.next=currentNode.next;
temp.prev=currentNode;

currentNode.next.prev=temp;
currentNode.next = temp;

return headNode;

}

static Node insertLast(Node headNode,int data) {
    // Copy from Lab15
}

static String toString(Node headNode) {
    // Copy from Lab15
}

public static void main(String[] args) {

    Node head=null;
    head = insert(head,3,55);
    head = insertLast(head,10);
    head = insertLast(head,20);
    head = insertLast(head,30);
    head = insertLast(head,40);
    head = insertLast(head,50);

    System.out.println(toString(head));
    head = insert(head,3,99); //
    System.out.println(toString(head));
}
}
```

9.3.6. Delete the First Node in D.L.L

Lab17.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab17 {

    static Node deleteFirst(Node headNode) {

        //1. If head is null
        if(headNode==null) {
            return null;
        }

        //2. If One Node is there
        if(headNode.next==null){
            return null;
        }

        //3.If Head is Not Null
        Node temp = headNode;

        headNode = headNode.next;
        headNode.prev=null;

        temp.next = null;

        return headNode;
    }

    static Node insertLast(Node headNode,int data) {
        // Copy from Lab15
    }
}
```



```
static String toString(Node headNode) {  
    // Copy from Lab15  
}  
  
public static void main(String[] args) {  
  
    Node head=null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
  
    System.out.println(toString(head));  
    head = deleteFirst(head); //10  
    System.out.println(toString(head));  
    head = deleteFirst(head); //20  
    System.out.println(toString(head));  
  
    }  
}
```

9.3.7. Delete the Last Node in D.L.L

Lab18.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab18 {

    static Node deleteLast(Node headNode) {

        //1. If head is null
        if(headNode==null) {
            return null;
        }

        //2. If One Node is there
        if(headNode.next==null){
            return null;
        }

        //3. Move to Last but one
        Node currentNode = headNode;
        while (currentNode.next.next != null) {
            currentNode = currentNode.next;
        }

        Node temp = currentNode.next;
        temp.prev=null;

        currentNode.next = null;
        return headNode;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15
```

```
}
```

```
static String toString(Node headNode) {  
    // Copy from Lab15
```

```
}
```

```
public static void main(String[] args) {
```

```
    Node head=null;
```

```
    head = insertLast(head,10);
```

```
    head = insertLast(head,20);
```

```
    head = insertLast(head,30);
```

```
    head = insertLast(head,40);
```

```
    head = insertLast(head,50);
```

```
    System.out.println(toString(head));
```

```
    head = deleteLast(head); //50
```

```
    System.out.println(toString(head));
```

```
    head = deleteLast(head); //40
```

```
    System.out.println(toString(head));
```

```
}
```

```
}
```


9.3.8. Delete the Node at given position in D.L.L

Lab19.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab19 {

    static Node delete(Node headNode, int position) {

        // 1.head is null
        if (headNode == null)
            return null;

        // 2.One Node
        if (headNode.next == null)
            return null;

        // 3.delete first node
        if (position == 1) {
            Node temp = headNode;

            headNode = headNode.next;
            headNode.prev = null;

            temp.next = null;
            return headNode;
        }
    }
}
```



```
// 4. Move to Position -1
Node currentCode = headNode;

for (int i = 1; i <= position - 2 && currentCode != null; i++) {
    currentCode = currentCode.next;
}

// 5.Position is out of Range
if (currentCode == null || currentCode.next==null) {
    return headNode;
}

Node temp = currentCode.next;

currentCode.next = currentCode.next.next;
if (currentCode.next != null) {
    currentCode.next.prev = currentCode;
}

temp.next = null;
temp.prev = null;

return headNode;
}

static Node insertLast(Node headNode,int data) {
    // Copy from Lab15
}

static String toString(Node headNode) {
    // Copy from Lab15
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    head = delete(head, 3); // 30  
    System.out.println(toString(head));  
    head = delete(head, 4); // 50  
    System.out.println(toString(head));  
    head = delete(head, 4); // No Deletion  
    System.out.println(toString(head));  
    head = delete(head, 3); // 40  
    System.out.println(toString(head));  
}  
}
```

9.3.9. Search the Node in D.L.L from beginnig

Lab20.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab20{

    static int search(Node headNode, int element) {

        int position =1;

        Node currentNode = headNode;

        while(currentNode!=null) {
            if( currentNode.data == element) {
                return position;
            }else {
                position++;
                currentNode = currentNode.next;
            }
        }
        return -1;
    }

    static Node insertLast(Node headNode,int data) {
        // Copy from Lab15
    }

    static String toString(Node headNode) {
        // Copy from Lab15
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    int x = search(head, 30); //3  
    System.out.println(x);  
    x = search(head, 50); //5  
    System.out.println(x);  
    x = search(head, 60); //-1  
    System.out.println(x);  
}  
}
```

9.3.10. Search the Node in D.L.L from ending

Lab21.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab21{

    static int search(Node headNode, int element) {

        Node currentNode = headNode;
        int nodeCount = 1;

        //Move to Last Node
        while(currentNode.next != null) {
            currentNode = currentNode.next;
            nodeCount++;
        }

        while(currentNode!=null) {
            if( currentNode.data == element) {
                return nodeCount;
            }else {
                nodeCount--;
                currentNode = currentNode.prev;
            }
        }

        return -1;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15  
}
```

```
static String toString(Node headNode) {  
    // Copy from Lab15  
}
```

```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 30);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    int x = search(head, 30); //3  
    System.out.println(x);  
    x = search(head, 50); //5  
    System.out.println(x);  
    x = search(head, 70); //-1  
    System.out.println(x);  
}
```

```
}
```



9.3.11. Reverse the D.L.L

Lab22.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab22 {
    static Node reverse(Node headNode) {

        // 1. Empty List
        if (headNode == null) {
            return null;
        }
        // 2. One Node List
        if (headNode.next == null) {
            return headNode;
        }

        Node tempNode = null;

        Node currentNode = headNode;
        while(currentNode!=null) {
            //Swap
            tempNode = currentNode.prev;
            currentNode.prev= currentNode.next;
            currentNode.next = tempNode;

            currentNode = currentNode.prev;
        }

        headNode = tempNode.prev;
        return headNode;
    }
}
```




```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15  
}  
  
static String toString(Node headNode) {  
    // Copy from Lab15  
}  
  
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    head = reverse(head);  
    System.out.println(toString(head));  
  
}  
}
```



9.3.12. Design MyDoublyLinkedList

Lab23.java

```
package com.jlccindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Node {
    int data;
    Node next;
    Node prev;

    Node(int data) {
        this.data = data;
        this.next = null;
        this.prev=null;
    }
}

public class MyDoublyLinkedList {

    Node headNode = null;
    int nodeCount = 0;

    public MyDoublyLinkedList() {
    }

    public String toString() {
        if (this.headNode == null) {
            return "[]";
        }

        String str = "[";
        Node currentNode = this.headNode;
```



```
        while (currentNode != null) {
            str = str + "" + currentNode.data + ", ";
            currentNode = currentNode.next;
        }
        str = str.substring(0, str.length() - 2);
        str = str + "]";

        return str;
    }

    public boolean isEmpty() {
        return this.nodeCount == 0;
    }

    public int size() {
        return this.nodeCount;
    }

    public void clear() {

        Node currentNode = this.headNode;
        while (currentNode != null) {
            Node temp = currentNode.next;

            currentNode.next = null;
            currentNode.prev = null;

            currentNode = temp;
        }

        this.headNode = null;
        this.nodeCount = 0;
    }
}
```



```
public void insertFirst(int data) {
```

```
    Node temp = new Node(data);
```

```
    // 1.Empty List
```

```
    if (this.headNode == null) {
```

```
        this.headNode = temp;
```

```
        nodeCount++;
```

```
        return;
```

```
    }
```

```
    // 2.List with 1 or more Nodes
```

```
    temp.next = this.headNode;
```

```
    this.headNode.prev = temp;
```

```
    this.headNode = temp;
```

```
    nodeCount++;
```

```
}
```

```
public void insertLast(int data) {
```

```
    Node temp = new Node(data);
```

```
    // 1.Empty List
```

```
    if (this.headNode == null) {
```

```
        this.headNode = temp;
```

```
        nodeCount++;
```

```
        return;
```

```
    }
```

```
    // 2.List with 1 or more Nodes
```

```
    Node currentNode = this.headNode;
```

```
    while (currentNode.next != null) {
```

```
        currentNode = currentNode.next;
```

```
    }
```



```
        currentNode.next = temp;
        temp.prev = currentNode;
        nodeCount++;
    }

    public void insert(int position, int data) {

        // 1. Create New Node
        Node temp = new Node(data);

        // 2.Empty List
        if (this.headNode == null) {
            this.headNode = temp;
            nodeCount++;
            return;
        }

        // 3. If position is 1
        if (position == 1) {
            temp.next = this.headNode;
            this.headNode.prev = temp;
            this.headNode = temp;
            nodeCount++;
            return;
        }

        // 4. Move the position -1 Node
        Node currentNode = this.headNode;
        for (int i = 1; i <= position - 2 && currentNode != null; i++) {
            currentNode = currentNode.next;
        }

        // 5. If Position is out of range
        if (currentNode == null) {
            return;
        }
    }
}
```



```
// 6. Insert at given Position
temp.next = currentNode.next;
temp.prev = currentNode;

currentNode.next.prev = temp;
currentNode.next = temp;

nodeCount++;

}

public void deleteFirst() {

    // 1. If head is null
    if (this.headNode == null) {
        this.headNode = null;
        return;
    }

    // 2. If One Node is there
    if (this.headNode.next == null) {
        this.headNode = null;
        nodeCount--;
        return;
    }

    // 3. when more nodes are there
    Node temp = this.headNode;

    this.headNode = this.headNode.next;
    headNode.prev = null;

    temp.next = null;
    nodeCount--;

}
```



```
public void deleteLast() {
```

```
    // 1. If head is null
    if (this.headNode == null) {
        return;
    }

    // 2. If One Node is there
    if (this.headNode.next == null) {
        this.headNode = null;
        nodeCount--;
        return;
    }

    // 3. When two or More Nodes
    Node currentNode = this.headNode;
    while (currentNode.next.next != null) {
        currentNode = currentNode.next;
    }

    Node temp = currentNode.next;
    temp.prev = null;

    currentNode.next = null;
    nodeCount--;

}
```

```
public void delete(int position) {
```

```
    // 1.head is null
    if (this.headNode == null) {
        this.headNode = null;
        return;
    }
}
```



```
// 2.One Node
if (this.headNode.next == null) {
    this.headNode = null;
    nodeCount--;
    return;
}

// 3.delete first node
if (position == 1) {
    Node temp = this.headNode;

    this.headNode = this.headNode.next;
    this.headNode.prev = null;

    temp.next = null;
    nodeCount--;

    return;
}

// 4. Move to Position -1
Node currentCode = this.headNode;

for (int i = 1; i <= position - 2 && currentCode != null; i++) {
    currentCode = currentCode.next;
}

// 5.Position is out of Range
if (currentCode == null || currentCode.next == null) {
    return;
}

Node temp = currentCode.next;

currentCode.next = currentCode.next.next;
if (currentCode.next != null) {
    currentCode.next.prev = currentCode;
}
```




```
    }

    temp.next = null;
    temp.prev = null;

    nodeCount--;

}

public int searchFirst(int element) {

    int position = 1;
    Node currentNode = headNode;

    while (currentNode != null) {
        if (currentNode.data == element) {
            return position;
        } else {
            position++;
            currentNode = currentNode.next;
        }
    }
    return -1;
}

public int searchLast(int element) {

    Node currentNode = headNode;
    int nodeCount = 1;

    // Move to Last Node
    while (currentNode.next != null) {
        currentNode = currentNode.next;
        nodeCount++;
    }
}
```



```
        while (currentNode != null) {
            if (currentNode.data == element) {
                return nodeCount;
            } else {
                nodeCount--;
                currentNode = currentNode.prev;
            }
        }
        return -1;
    }

    public void reverse() {
        // 1. Empty List
        if (this.headNode == null) {
            return;
        }
        // 2. One Node List
        if (this.headNode.next == null) {
            return;
        }

        Node tempNode = null;

        Node currentNode = this.headNode;
        while (currentNode != null) {
            // Swap
            tempNode = currentNode.prev;
            currentNode.prev = currentNode.next;
            currentNode.next = tempNode;

            currentNode = currentNode.prev;
        }
        this.headNode = tempNode.prev;
    }
}
```



```
public class Lab23 {  
  
    public static void main(String[] args) {  
  
        MyDoublyLinkedList mylist = new MyDoublyLinkedList();  
  
        System.out.println("-----1-----");  
        System.out.println(mylist);  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist.size());  
  
        mylist.insertFirst(10);  
        mylist.insertFirst(20);  
        mylist.insertFirst(30);  
  
        System.out.println("-----2-----");  
        System.out.println(mylist);  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist.size());  
  
        mylist.insertLast(88);  
        mylist.insertLast(99);  
  
        System.out.println("-----3-----");  
        System.out.println(mylist);  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist.size());  
  
        mylist.insert(4, 77);  
  
        System.out.println("-----4-----");  
        System.out.println(mylist);  
    }  
}
```



```
mylist.deleteFirst();

System.out.println("-----5-----");
System.out.println(mylist);

mylist.deleteLast();

System.out.println("-----6-----");
System.out.println(mylist);

mylist.delete(2);

System.out.println("-----7-----");
System.out.println(mylist);

mylist.insertLast(77);
mylist.insertLast(99);

System.out.println("-----8-----");
System.out.println(mylist);

System.out.println("-----9-----");
System.out.println(mylist.searchFirst(77));
System.out.println(mylist.searchLast(77));
System.out.println(mylist.searchFirst(55));
System.out.println(mylist.searchLast(55));

mylist.reverse();
System.out.println("-----10-----");
System.out.println(mylist);
}
```

```
}
```

DSA

Module 10

Stacks

Author
Srinivas Dande





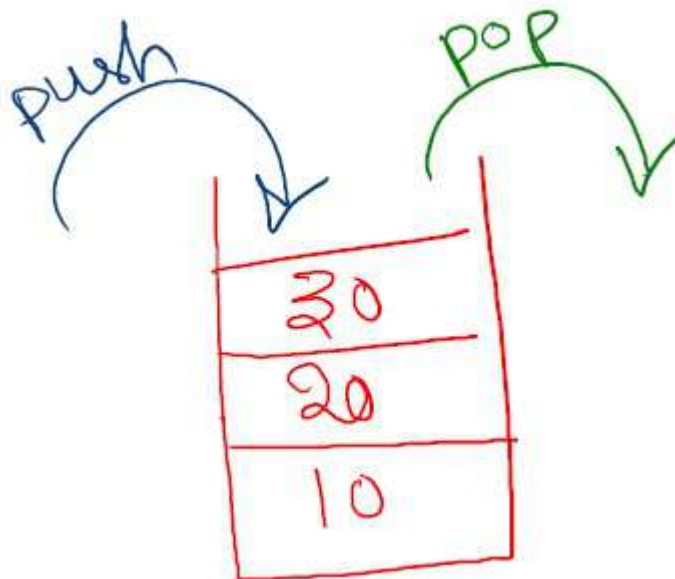
Java Learning Center

No.1 In Java Training & placement



10. Introduction to Stacks

- ♦ Stack is an ADT (Abstract Data Type).
- ♦ Stack is a **Linear Data Structure**.
- ♦ Stack follow the Strategy called **LIFO (Last In First Out)**.
- ♦ Stack is Closed at One End and Open at other end for the Operations
i.e Bottom of the Stack is closed and Top of the Stack is open for the Operations.
- ♦ You can do the Insert and Delete Operations on the Top End of the Stack
- ♦ Insert Operation on the Stack is called as **Push**
- ♦ Delete Operation on the Stack is called as **Pop**



- ♦ Stacks can be implemented using **Arrays and LinkedLists**

A stack is an ordered list in which insertion and deletion are done at one end, called top. The last element inserted is the first one to be deleted. Hence, it is called the Last in First out (LIFO) or First in Last out (FILO) list.



10.1. Stack Operations

- a) isEmpty()
- b) size()
- c) push(int)
- d) pop()
- e) peek()

a) isEmpty()

- ◆ Returns True if the Stack is Empty otherwise false.

b) size()

- ◆ Returns the size of the stack

c) push(object)

- ◆ Inserts the Element to the Top of the Stack

d) pop()

- ◆ Returns the Top Element and Removes that from the Stack

e) peek()

- ◆ Returns the Top Element from the Stack without Removing

10.2. Corner Conditions

- a) Stack Underflow
- b) Stack Overflow

a) Stack Underflow

- ◆ When pop() or peek() is called on the empty Stack.

b) Stack Overflow

- ◆ When push() is called on the full Stack.

10.3. Time Complexity of Stack Operations

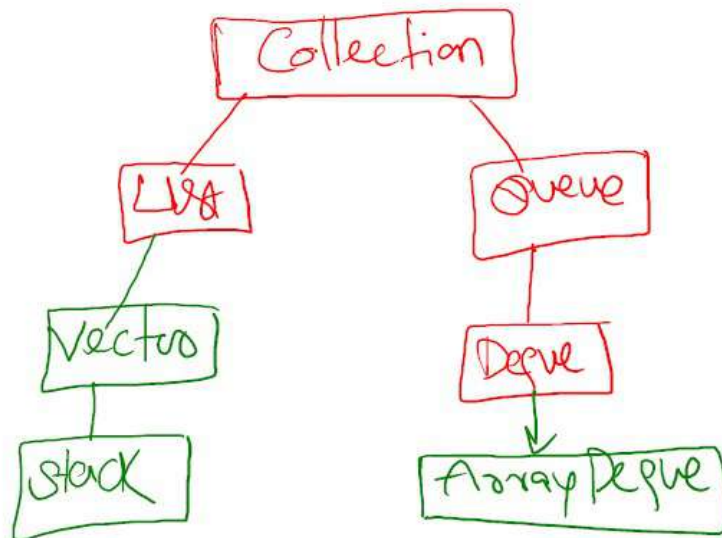
<u>Operation</u>	<u>Arrays</u>	<u>Dynamic Arrays</u>	<u>LinkedList</u>
push()	$O(1)$	Amortized $O(1)$	$O(1)$
pop()	$O(1)$	$O(1)$	$O(1)$
peek()	$O(1)$	$O(1)$	$O(1)$

10.4. Applications of Stacks

- ◆ Function Calls
- ◆ Balanced Parenthesis
- ◆ Reversing Items
- ◆ Infix to Postfix
- ◆ Infix to Prefix
- ◆ Evaluation of Postfix
- ◆ Evaluation of Prefix
- ◆ Stock Span Problems
- ◆ Undo/Redo
- ◆ Forward/backward
- etc

10.5. Stacks in Java Collection

- ♦ Java Collections has two classes for Stack use-cases.
 - a) Stack
 - b) ArrayDeque



a) Stack

- ♦ Stack is a legacy class from java.util package.
- ♦ All the methods of Stack are synchronized means Internal synchronization is provided
- ♦ Use the Stack class in multi-threaded environment

a) ArrayDeque

- ♦ ArrayDeque is a latest class from java.util package.
- ♦ All the methods of Stack are not synchronized means Internal synchronization is not provided
- ♦ Use the ArrayDeque class in single-threaded environment

- ♦ If you want to use ArrayDeque in multi-threaded environment then you need to provide external synchronization.
- ♦ More complete and consistent set of LIFO stack operations is provided in ArrayDeque class
- ♦ So Use ArrayDeque in preference to Stack class.
- ♦ Most ArrayDeque operations run in amortized $O(1)$.
- ♦ This class is likely to be faster than Stack when used as a stack

10.5.1. Using Stack and ArrayDeque API

Lab1.java

```
package com.jlcindia.stacks;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.ArrayDeque;
import java.util.Deque;

public class Lab1 {

    public static void main(String[] args) {

        Deque<Integer> mystack= new ArrayDeque<>();
        //Stack<Integer> mystack=new Stack<>();

        System.out.println(mystack);
        System.out.println(mystack.size());
        System.out.println(mystack.isEmpty());
    }
}
```



```
mystack.push(10);
mystack.push(20);
mystack.push(30);
mystack.push(40);
mystack.push(50);

System.out.println(mystack);
System.out.println(mystack.size());
System.out.println(mystack.isEmpty());

System.out.println(mystack.peek()); //50

mystack.pop();
mystack.pop();

System.out.println(mystack);
System.out.println(mystack.size());
System.out.println(mystack.isEmpty());

System.out.println(mystack.peek()); //30

}
}
```

10.6. Stack Implementation using Arrays

StackFullException.java

```
package com.jlcindia.stacks.arrays1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class StackFullException extends RuntimeException {

    @Override
    public String toString() {
        return "StackFullException [No Push Please ]";
    }
}
```

StackFullException.java

```
package com.jlcindia.stacks.arrays1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class StackEmptyException extends RuntimeException {

    @Override
    public String toString() {
        return "StackEmptyException [No pop or peek Please ]";
    }
}
```



MyStack.java

```
package com.jlcindia.stacks.arrays1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class MyStack {

    private Integer myarray[];
    private int capacity;
    private int top;

    public MyStack(int capacity) {
        this.capacity = capacity;
        this.top = -1;
        this.myarray = new Integer[capacity];
    }

    public int size() {
        return top + 1;
    }

    public boolean isEmpty() {
        return (top == -1);
    }

    public void push(Integer element) {

        if (top == capacity - 1) {
            throw new StackFullException();
        }

        top++;
        myarray[top] = element;
    }
}
```



```
public Integer pop() {  
  
    if (top == - 1) {  
        throw new StackEmptyException();  
    }  
    int element = myarray[top];  
    myarray[top] = null;  
    top--;  
    return element;  
}  
  
public Integer peek() {  
  
    if (top == - 1) {  
        throw new StackEmptyException();  
    }  
    int element = myarray[top];  
    return element;  
}  
  
public String toString() {  
    String str = "[";  
    if (top != -1) {  
        for (Integer x : myarray) {  
            if (x != null)  
                str = str + x + ",";  
        }  
    }  
    str = str + "];"  
    return str;  
}  
}
```



Lab2.java

```
package com.jlcindia.stacks.arrays1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {

    public static void main(String[] args) {

        MyStack mystack=new MyStack(5);

        System.out.println(mystack);
        System.out.println(mystack.size());
        System.out.println(mystack.isEmpty());

        mystack.push(10);
        mystack.push(20);
        mystack.push(30);
        mystack.push(40);
        mystack.push(50);
        //mystack.push(99);

        System.out.println(mystack);
        System.out.println(mystack.size());
        System.out.println(mystack.isEmpty());

        System.out.println(mystack.peek()); //50

        mystack.pop();
        mystack.pop();

        System.out.println(mystack);
        System.out.println(mystack.size());
        System.out.println(mystack.isEmpty());
```




```
System.out.println(mystack.peek()); //30
```

```
mystack.pop();  
mystack.pop();  
mystack.pop();  
//mystack.pop();
```

```
System.out.println(mystack);  
System.out.println(mystack.size());  
System.out.println(mystack.isEmpty());
```

```
}
```

```
}
```

10.7. Stack Implementation using Arrays

MyStack.java

```
package com.jlcindia.stacks.arrays2;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.EmptyStackException;

public class MyStack {

    private Integer myarray[];
    private int capacity;
    private int top;

    public MyStack(int capacity) {
        this.capacity = capacity;
        this.top = -1;
        this.myarray = new Integer[capacity];
    }
    public int capacity() {
        return capacity;
    }
    public int size() {
        return top + 1;
    }

    public boolean isEmpty() {
        return (top == -1);
    }
}
```



```
public void push(Integer element) {

    if (top == capacity - 1) {
        //Resize the Array
        int newCapacity= this.capacity * 2;
        Integer mynewArray[] = new Integer[newCapacity];
        System.arraycopy(myarray, 0, mynewArray, 0, capacity);
        myarray=mynewArray;
        capacity=newCapacity;
    }

    top++;
    myarray[top] = element;

}

public Integer pop() {

    if (top == - 1) {
        throw new EmptyStackException();
    }
    int element = myarray[top];
    myarray[top] = null;
    top--;
    return element;
}

public Integer peek() {

    if (top == - 1) {
        throw new EmptyStackException();
    }
    int element = myarray[top];
    return element;
}
```



```
public String toString() {  
    String str = "[";  
    if (top != -1) {  
        for (Integer x : myarray) {  
            if (x != null)  
                str = str + x + ",";  
        }  
    }  
    str = str + "];"  
    return str;  
}  
}
```

Lab3.java

```
package com.jlcindia.stacks.arrays2;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public class Lab3 {  
    public static void main(String[] args) {  
  
        MyStack mystack=new MyStack(5);  
  
        System.out.println(mystack);  
        System.out.println(mystack.size());  
        System.out.println(mystack.isEmpty());  
        System.out.println(mystack.capacity());//5  
  
        mystack.push(10);  
        mystack.push(20);  
        mystack.push(30);  
        mystack.push(40);  
        mystack.push(50);  
    }  
}
```



```
System.out.println(mystack);
System.out.println(mystack.size());
System.out.println(mystack.isEmpty());
System.out.println(mystack.capacity()); //5

mystack.push(60);
System.out.println(mystack);
System.out.println(mystack.size());
System.out.println(mystack.isEmpty());
System.out.println(mystack.capacity()); //10

}
}
```

10.8. Stack Implementation using LinkedLists

Node.java

```
package com.jlcindia.stacks.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Node {

    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```



MyStack.java

```
package com.jlcindia.stacks.linkedlist;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.EmptyStackException;

public class MyStack {

    private Node headNode;
    private int size;

    public MyStack() {
        this.headNode = null;
        this.size = 0;
    }

    public int size() {
        return this.size;
    }

    public boolean isEmpty() {
        return this.headNode == null;
    }

    public void push(int element) {
        Node temp = new Node(element);
        temp.next = headNode;
        headNode = temp;
        size++;
    }
}
```



```
public Integer pop() {  
  
    if(headNode==null) {  
        throw new EmptyStackException();  
    }  
  
    int element = headNode.data;  
  
    headNode = headNode.next;  
    size--;  
  
    return element;  
}  
  
public Integer peek() {  
  
    if(headNode==null) {  
        throw new EmptyStackException();  
    }  
  
    int element = headNode.data;  
  
    return element;  
}  
  
public String toString() {  
    if (this.headNode == null) {  
        return "[]";  
    }  
  
    String str = "[";  
    Node currentNode = this.headNode;  
    while (currentNode != null) {  
        str = str + "" + currentNode.data + " , ";  
        currentNode = currentNode.next;  
    }  
}
```



```
        str = str.substring(0, str.length() - 2);
        str = str + "];";

        return str;
    }
}
```

Lab4.java

```
package com.jlcindia.stacks.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab4 {

    public static void main(String[] args) {

        MyStack mystack=new MyStack();

        System.out.println(mystack);
        System.out.println(mystack.size());
        System.out.println(mystack.isEmpty());

        mystack.push(10);
        mystack.push(20);
        mystack.push(30);
        mystack.push(40);
        mystack.push(50);

        System.out.println(mystack);
        System.out.println(mystack.size());
        System.out.println(mystack.isEmpty());
        System.out.println(mystack.peek()); //50
    }
}
```




```
mystack.pop();
mystack.pop();

System.out.println(mystack);
System.out.println(mystack.size());
System.out.println(mystack.isEmpty());
System.out.println(mystack.peek()); //30

}
}
```

10.9. Two Stacks Implementation in One Array

MyStack.java

```
package com.jlcindia.twostacks;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class TwoStacks {

    private Integer myarray[];
    private int capacity;
    private int top1;
    private int top2;

    public TwoStacks(int capacity) {
        this.capacity = capacity;
        this.top1 = -1;
        this.top2 = capacity;
        this.myarray = new Integer[capacity];
    }
}
```



```
public boolean push1(Integer element) {

    if (top1 < top2 - 1) {
        top1++;
        myarray[top1] = element;
        return true;
    }

    return false;
}

public boolean push2(Integer element) {

    if (top1 < top2 - 1) {
        top2--;
        myarray[top2] = element;
        return true;
    }

    return false;
}

public Integer pop1() {
    if (top1 > -1) {
        int element = myarray[top1];
        myarray[top1] = null;
        top1--;
        return element;
    }
    return null;
}

public Integer pop2() {
    if (top2 < capacity) {
        int element = myarray[top2];
        myarray[top2] = null;
        top2++;
    }
}
```



```
        return element;
    }
    return null;
}

public Integer peek1() {
    if (top1 > -1) {
        int element = myarray[top1];
        return element;
    }
    return null;
}

public Integer peek2() {
    if (top2 < capacity) {
        int element = myarray[top2];
        return element;
    }
    return null;
}

public int size1() {
    return top1 + 1;
}

public int size2() {
    return capacity - top2;
}

public void printStacks() {
    for (int i = 0; i < capacity; i++) {
        System.out.print(myarray[i] + " ");
    }
    System.out.println();
}
}
```



Lab5.java

```
package com.jlcindia.twostacks;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5 {

    public static void main(String[] args) {

        TwoStacks mystack=new TwoStacks(5);

        mystack.printStacks();
        System.out.println(mystack.size1());
        System.out.println(mystack.size2());

        mystack.push1(10);
        mystack.push1(20);
        mystack.push2(99);
        mystack.push2(88);
        mystack.push2(77);

        mystack.push1(30);

        System.out.println("-----");
        mystack.printStacks();
        System.out.println(mystack.size1());
        System.out.println(mystack.size2());
        System.out.println(mystack.peek1());
        System.out.println(mystack.peek2());

        mystack.pop1();
        mystack.pop2();
        mystack.pop2();
    }
}
```



```
System.out.println("-----");
mystack.printStacks();
System.out.println(mystack.size1());
System.out.println(mystack.size2());
System.out.println(mystack.peek1());
System.out.println(mystack.peek2());
}

}
```

10.10. Palindrome with Stacks

Lab6.java

```
package com.jlcindia.stacks;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.ArrayDeque;
import java.util.Deque;

public class Lab6 {

    static boolean isPalimdrome(String str) {
        Deque<Character> mystack= new ArrayDeque<>();
        StringBuilder sb=new StringBuilder(str.length());
        str = str.toLowerCase();
        for(int i=0;i<str.length();i++) {
            char ch= str.charAt(i);
            if(ch>='a' && ch<='z') {
                sb.append(ch);
                mystack.push(ch);
            }
        }
    }
}
```



```
        StringBuilder revSb=new StringBuilder(str.length());
        while(!mystack.isEmpty()) {
            revSb.append(mystack.pop());
        }

        return sb.toString().equals(revSb.toString());

    }

    public static void main(String[] args) {
        String str = "ab@$cb%3a";
        boolean x= isPalimdrome(str);
        System.out.println(x);
    }
}
```

DSA

Module 11

Queues

Author
Srinivas Dande





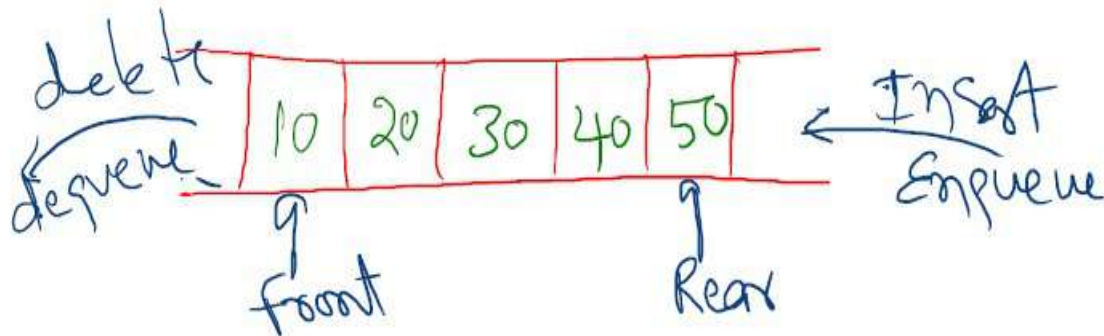
Java Learning Center

No.1 In Java Training & placement



11. Introduction to Queues

- ♦ Queue is an ADT (Abstract Data Type).
- ♦ Queue is a **Linear Data Structure**.
- ♦ Queue follow the Strategy called **FIFO (First In First Out)**.
- ♦ Queue is Open at both the Ends.
i.e You can do operations on both the ends.
- ♦ You can do **Insert Operation** in the **Back(Rear)** of the Queue
- ♦ You can do **Delete Operation** in the **Front** of the Queue
- ♦ **Insert Operation** on the Queue is called as **Enqueue**
- ♦ **Delete Operation** on the Queue is called as **Dequeue**



- ♦ Queues can be implemented using **Arrays and LinkedLists**

- ✓ A queue is an ordered list in which insertions are done at one end (**rear**) and deletions are done at other end (**front**).
- ✓ The first element to be inserted is the first one to be deleted. Hence, it is called First in First out (**FIFO**) or Last in Last out (**LILO**) list.



11.1. Queue Operations

- a) isEmpty()
- b) size()
- c) offer(int)
- d) poll()
- e) peek()

a) isEmpty()

- ♦ Returns True if the Queue is Empty otherwise false..

b) size()

- ♦ Returns the size of the Queue

c) offer(object)

- ♦ Inserts the Element at the Back of the Queue

d) poll()

- ♦ Removes the Front Element and returns the same

e) peek()

- ♦ Returns the Front Element from the Queue without Removing

11.2. Corner Conditions

- a) Queue Underflow
- b) Queue Overflow

a) Queue Underflow

- ♦ When poll() or peek() is called on the empty Queue.

b) Queue Overflow

- ♦ When offer() is called on the full Queue.

11.3. Time Complexity of Stack Operations

<u>Operation</u>	<u>Fixed Arrays</u>	<u>Circular Fixed Arrays</u>	<u>Circular Dynamic Arrays</u>	<u>LinkedList</u>
offer()	$O(1)$	$O(1)$	Amortized $O(1)$	$O(1)$
poll()	$O(n)$	$O(1)$	$O(1)$	$O(1)$
peek()	$O(1)$	$O(1)$	$O(1)$	$O(1)$

11.4. Applications of Queues

- ♦ Queues can be used in many of the real-world applications
 - a) Single Resource and Multiple Consumers
 - b) Sync between slow and fast devices

a) Single Resource and Multiple Consumers:

1) Web servers:

- ♦ Web servers uses queue to manage incoming requests from web clients.

2) CPU scheduling:

- ♦ OS uses queue to schedule the CPU based on FCFS Scheduling

3) Printers:

- ♦ Printers uses queue to manage the order in which the documents are printed

b) Sync between slow and fast devices

1) Keyboard buffer:

- ♦ Buffers the Keyboard input in queue and process it later

2) Message Buffering:

- ♦ In Distributed Systems, Messages will be placed in Queue and later consumers will consume it slowly.

3) Audio and Video Players:

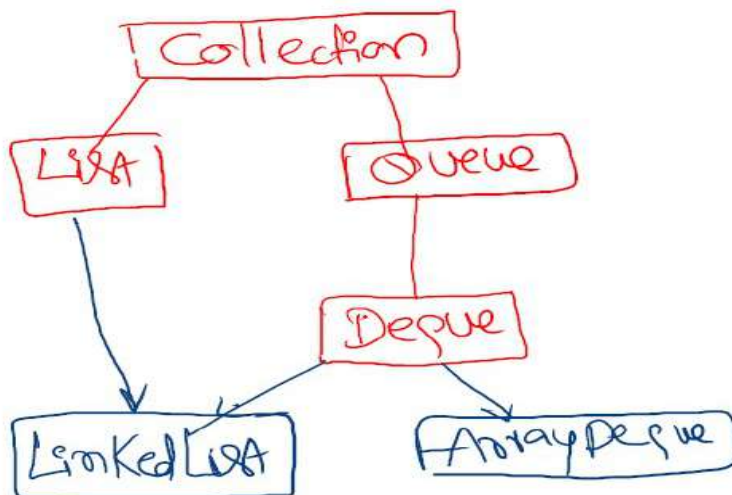
- ♦ Buffers the content and Plays it slowly

4) Event Handling:

- ♦ Queues can be used to handle events in event-driven systems

11.5. Queues in Java Collection

- ♦ Java Collections has two classes for Queues use-cases.
 - a) ArrayDeque
 - b) LinkedList



a) ArrayDeque

- ♦ ArrayDeque is a latest class from java.util package.
- ♦ ArrayDeque is implemented with Arrays.
- ♦ Use the ArrayDeque class in single-threaded environment
- ♦ If you want to use ArrayDeque in multi-threaded environment then you need to provide external synchronization.
- ♦ Most ArrayDeque operations run in amortized $O(1)$.
- ♦ ArrayDeque class is likely to be faster than LinkedList
- ♦ ArrayDeque can be used as
 - ♦ Stack
 - ♦ Queue

b) LinkedList

- ♦ LinkedList is the class from java.util package.
- ♦ LinkedList is implemented with Nodes.
- ♦ Use the LinkedList class in single-threaded environment
- ♦ If you want to use LinkedList in multi-threaded environment then you need to provide external synchronization.
- ♦ Most LinkedList operations run in $O(1)$.
- ♦ LinkedList class is likely to be slower than ArrayDeque
- ♦ LinkedList can be used as
 - ♦ List
 - ♦ Queue



11.5.1. Using Queue Built-In Methods

Lab1.java

```
package com.jlcindia.queue;

import java.util.ArrayDeque;
import java.util.LinkedList;
import java.util.Queue;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {
    public static void main(String[] args) {

        //Queue<Integer> myque= new LinkedList<>();
        Queue<Integer> myque= new ArrayDeque<>();

        System.out.println(myque);
        System.out.println(myque.size());
        System.out.println(myque.isEmpty());

        myque.offer(10);
        myque.offer(20);
        myque.offer(30);
        myque.offer(40);
        myque.offer(50);

        System.out.println("-----");
        System.out.println(myque);
        System.out.println(myque.size());
        System.out.println(myque.isEmpty());

        System.out.println("-----");
        System.out.println(myque.peek());
        System.out.println("-----");
    }
}
```



```
        myque.poll();
        myque.poll();
        System.out.println("-----");
        System.out.println(myque.peek());
        System.out.println("-----");
    }
}
```

11.5.2. Using Queue Built-In Methods

Lab2.java

```
package com.jlcindia.queues;

import java.util.ArrayDeque;
import java.util.LinkedList;
import java.util.Queue;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab2 {
    public static void main(String[] args) {

        Queue<Integer> myque= new LinkedList<>();
        //Queue<Integer> myque= new ArrayDeque<>();

        System.out.println(myque);
        System.out.println(myque.size());
        System.out.println(myque.isEmpty());

        myque.add(10);
        myque.add(20);
        myque.add(30);
        myque.add(40);
        myque.add(50);
    }
}
```




```
System.out.println("-----");
System.out.println(myque);
System.out.println(myque.size());
System.out.println(myque.isEmpty());

System.out.println("-----");
System.out.println(myque.element());
System.out.println("-----");

myque.remove();
myque.remove();

System.out.println("-----");
System.out.println(myque.element());
System.out.println("-----");
}
}
```

11.5.3. Traversing Queue elements in Forward Order

Lab3.java

```
package com.jlcindia.queues;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    public static void main(String[] args) {

        Queue<Integer> myque= new LinkedList<>();
        //Queue<Integer> myque= new ArrayDeque<>();
```




```
myque.add(10);
myque.add(20);
myque.add(30);
myque.add(40);
myque.add(50);

System.out.println(myque);
System.out.println("-----");

Iterator<Integer> it = myque.iterator();
while(it.hasNext()) {
    System.out.print(it.next()+"\t");
}

System.out.println("\n-----");
for(Integer x:myque) {
    System.out.print(x+"\t");
}
}
```

11.5.4. Reverse the Queue using Iteration

Lab4.java

```
package com.jlcindia.queues;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
public class Lab4 {  
    static void reverseQueue(Queue<Integer> myque) {  
        Stack<Integer> mystack = new Stack<>();  
  
        while(!myque.isEmpty()) {  
            mystack.push(myque.poll());  
        }  
  
        while(!mystack.isEmpty()) {  
            myque.offer(mystack.pop());  
        }  
    }  
  
    public static void main(String[] args) {  
  
        Queue<Integer> myque= new LinkedList<>();  
  
        myque.add(10);  
        myque.add(20);  
        myque.add(30);  
        myque.add(40);  
        myque.add(50);  
  
        System.out.println(myque);  
        System.out.println("-----");  
        reverseQueue(myque);  
        System.out.println(myque);  
  
    }  
}
```

11.5.5. Reverse the Queue using Recursion

Lab5.java

```
package com.jlcindia.queue;

import java.util.LinkedList;
import java.util.Queue;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab5 {
    static void reverseQueue(Queue<Integer> myque) {
        if(myque.isEmpty()) {
            return;
        }

        int x= myque.poll();
        reverseQueue(myque);
        myque.offer(x);
    }
    public static void main(String[] args) {
        Queue<Integer> myque= new LinkedList<>();

        myque.add(10);        myque.add(20);
        myque.add(30);        myque.add(40);
        myque.add(50);

        System.out.println(myque);
        System.out.println("-----");
        reverseQueue(myque);
        System.out.println(myque);
    }
}
```

11.6. Queue Implementation using Arrays

MyQueue.java

```
package com.jlcindia.queuearrays1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyQueue {

    int capacity; // size of array
    int size; //size of the Queue
    Integer myarray[];

    public MyQueue(int capacity) {
        this.capacity = capacity;
        this.size = 0;
        this.myarray = new Integer[capacity];
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return (size == 0);
    }

    public boolean isFull() {
        return (size == capacity);
    }
}
```



```
public boolean offer(int element) {
    if (isFull()) {
        return false;
    }

    myarray[size] = element;
    size++;
    return true;
}

public Integer poll() {
    if(isEmpty()) {
        return null;
    }
    int element = myarray[0];
    for(int i=0;i<size-1;i++) {
        myarray[i] = myarray[i+1];
    }
    size--;
    myarray[size] = null;
    return element;
}

public Integer peek() {
    if(isEmpty()) {
        return null;
    }

    return myarray[0];
}

public int getFront() {
    if(isEmpty())
        return -1;

    return 0;
}
```



```
public int getRear() {
    if(isEmpty())
        return -1;

    return size-1;
}

public String toString() {
    String str = "[";
    if (size != -1) {
        for (Integer x : myarray) {
            if (x != null)
                str = str + x + ",";
            else
                str= str+" null , ";
        }
    }
    str = str + "]";
    return str;
}
}
```

Lab6.java

```
package com.jlcindia.queuearrays1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab6 {
    public static void main(String[] args) {
        MyQueue myque = new MyQueue(5);

        System.out.println(myque);
        System.out.println(myque.size());
        System.out.println(myque.isEmpty());
        System.out.println(myque.isFull());
    }
}
```



```
myqueue.offer(10);
myqueue.offer(20);
myqueue.offer(30);
myqueue.offer(40);
myqueue.offer(50);

System.out.println(myqueue);
System.out.println(myqueue.size());
System.out.println(myqueue.isEmpty());
System.out.println(myqueue.isFull());
System.out.println("-----");
System.out.println(myqueue.peek()); //10
System.out.println("-----");
myqueue.poll();
myqueue.poll();
System.out.println(myqueue);
System.out.println("-----");
System.out.println(myqueue.peek()); //30
System.out.println("-----");
```

```
}
```

```
}
```



11.7. Queue Implementation using Arrays

MyQueue.java

```
package com.jlcindia.queues.arrays2;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyQueue {
    private int capacity; // size of array
    private int size; // size of the Queue
    private int front;
    private Integer myarray[];

    public MyQueue(int capacity) {
        this.capacity = capacity;
        this.size = 0;
        this.front = 0;
        this.myarray = new Integer[capacity];
    }
    public int size() {
        return size;
    }
    public boolean isEmpty() {
        return (size == 0);
    }
    public boolean isFull() {
        return (size == capacity);
    }
    public int getFront() {
        if (isEmpty())
            return -1;

        return front;
    }
}
```




```
public int getRear() {
    if (isEmpty())
        return -1;

    return (front+size-1)%capacity;
}

public boolean offer(int element) {
    if (isFull()) {
        //Resize Code
        //Dont return false
        return false;
    }

    int currRear= getRear();
    int newRear= (currRear+1)%capacity;

    myarray[newRear] = element;
    size++;
    return true;
}

public Integer poll() {
    if (isEmpty()) {
        return null;
    }

    int element = myarray[front];

    size--;
    myarray[front] = null;
    front = (front+1)%capacity;
    return element;
}
```



```
public Integer peek() {
    if (isEmpty()) {
        return null;
    }

    return myarray[front];
}

public String toString() {
    String str = "[";
    if (size != -1) {
        for (Integer x : myarray) {
            if (x != null)
                str = str + x + ",";
            else
                str = str + " null , ";
        }
    }
    str = str + "]";
    return str;
}
}
```

Lab7.java

```
package com.jlcindia.queues.arrays2;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab7 {
    public static void main(String[] args) {
        MyQueue myque = new MyQueue(5);

        System.out.println(myque);
        System.out.println(myque.size());
        System.out.println(myque.isEmpty());
    }
}
```



```
System.out.println(myqueue.isFull());
System.out.println(myqueue.getFront());
System.out.println(myqueue.getRear());

myqueue.offer(10);
myqueue.offer(20);
myqueue.offer(30);
myqueue.offer(40);
myqueue.offer(50);

System.out.println(myqueue);
System.out.println(myqueue.size());
System.out.println(myqueue.isEmpty());
System.out.println(myqueue.isFull());
System.out.println(myqueue.getFront());
System.out.println(myqueue.getRear());
System.out.println("-----");
System.out.println(myqueue.peek()); //10
System.out.println("-----");

myqueue.poll();
myqueue.poll();

System.out.println(myqueue);
System.out.println("-----");
System.out.println(myqueue.peek()); //30
System.out.println(myqueue.getFront()); //2
System.out.println(myqueue.getRear()); //4
System.out.println("-----");

myqueue.offer(55);
boolean x =myqueue.offer(66);
System.out.println(x);

System.out.println(myqueue);
```

```
        System.out.println("-----");
        System.out.println(myque.peek()); //30
        System.out.println(myque.getFront()); //2
        System.out.println(myque.getRear()); //1
        System.out.println("-----");

        boolean b= myque.offer(99);
        System.out.println(b);
    }
}
```

11.8. Queue Implementation using LinkedLists

Node.java

```
package com.jlcindia.queue.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Node {

    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```



MyQueue.java

```
package com.jlcindia.queues.linkedlists;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyQueue {

    private int size; // size of the Queue
    private Node frontNode;
    private Node rearNode;

    public MyQueue() {
        this.size = 0;
        this.frontNode=null;
        this.rearNode=null;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        //return (size == 0);
        return (frontNode==null && rearNode==null);
    }

    public Integer peek() {
        if (isEmpty()) {
            return null;
        }

        return frontNode.data;
    }
}
```



```
public void offer(int element) {

    Node temp = new Node(element);

    //1.Empty List
    if(isEmpty()) {
        frontNode=temp;
        rearNode=temp;
        size++;
        return;
    }
    //2.Non-Empty List
    rearNode.next=temp;
    rearNode=temp;
    size++;
}

public Integer poll() {
    if (isEmpty()) {
        return null;
    }

    int element = frontNode.data;
    Node temp = frontNode;
    frontNode=frontNode.next;
    temp.next=null;
    size--;

    if(frontNode==null) {
        rearNode=null;
    }

    return element;
}
```



```
public String toString() {
    if (this.frontNode == null) {
        return "[]";
    }

    String str = "[";
    Node currentNode = this.frontNode;
    while (currentNode != null) {
        str = str + "" + currentNode.data + ", ";
        currentNode = currentNode.next;
    }
    str = str.substring(0, str.length() - 2);
    str = str + "]";

    return str;
}
}
```

Lab4.java

```
package com.jlcindia.queues.linkedlists;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab8 {
    public static void main(String[] args) {

        MyQueue myque = new MyQueue();

        System.out.println(myque);
        System.out.println(myque.size());
        System.out.println(myque.isEmpty());

        myque.offer(10);
        myque.offer(20);
    }
}
```



```
myque.offer(30);
myque.offer(40);
myque.offer(50);

System.out.println(myque);
System.out.println(myque.size());
System.out.println(myque.isEmpty());

System.out.println("-----");
System.out.println(myque.peek()); //10
System.out.println("-----");

myque.poll();
myque.poll();

System.out.println(myque);
System.out.println("-----");
System.out.println(myque.peek()); //30
System.out.println("-----");

myque.offer(60);
myque.offer(70);

System.out.println(myque);
System.out.println("-----");
System.out.println(myque.peek()); //30
System.out.println("-----");

myque.offer(99);
}
```


DSA

Module 12

Dequeues

Author
Srinivas Dande





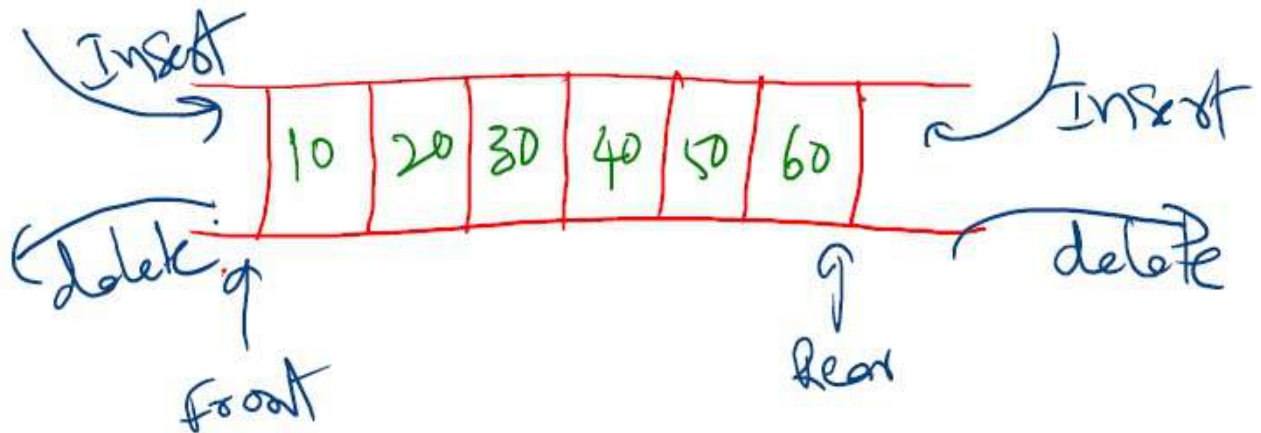
Java Learning Center

No.1 In Java Training & placement



12. Introduction to Deques

- ◆ Deque - Doubly Ended Queues.
- ◆ Deque is a special type of Queue.
- ◆ Deque does not follow FIFO Rule.
- ◆ You can do **Insert** and **Delete** Operations in the **Front of the Deque**
- ◆ You can do **Insert** and **Delete** Operations in the **Back(Rear)** of the Deque
- ◆ **Insert Operation** on the Deque is called as **Enqueue**
- ◆ **Delete Operation** on the Deque is called as **Deque**



- ◆ Deques can be implemented using **Arrays** and **LinkedLists**

✓ A **Deque** is a Special Queue which is an ordered list in which insertions are done at both **front** and **rear** and deletions are also done at both **front** and **rear**.



12.1. Queue Operations

- a) isEmpty()
- b) size()
- c) offerFirst()
- d) offerLast()
- e) pollFirst()
- f) pollLast()
- g) peekFirst()
- h) peekLast()

a) isEmpty():

- ♦ Returns True if the Queue is Empty otherwise false..

b) size():

- ♦ Returns the size of the Queue

c) offerFirst():

- ♦ Inserts the Element at the Front of the Queue

d) offerLast():

- ♦ Inserts the Element at the Rear of the Queue

e) pollFirst():

- ♦ Removes the Front Element from the Queue and returns the same

f) pollLast():

- ♦ Removes the Rear Element from the Queue and returns the same

g) peekFirst():

- ♦ Removes the Front Element from the Queue and returns the same

h) peekLast():

- ♦ Removes the Front Element from the Queue and returns the same

12.2. Corner Conditions

- a) Deque Underflow
- b) Deque Overflow

a) Deque Underflow

- ♦ When poll() or peek() is called on the empty Deque.

b) Deque Overflow

- ♦ When offer() is called on the full Deque.

12.3. Time Complexity of Stack Operations

<u>Operation</u>	<u>Fixed Arrays</u>	<u>Circular Fixed Arrays</u>	<u>Circular Dynamic Arrays</u>	<u>LinkedList</u>
offerFirst()	O(n)	O(1)	Amortized O(1)	O(1)
offerLast()	O(1)	O(1)	Amortized O(1)	O(1)
pollFirst()	O(n)	O(1)	O(1)	O(1)
pollLast()	O(1)	O(1)	O(1)	O(1)
peekFirst()	O(1)	O(1)	O(1)	O(1)
peekLast()	O(1)	O(1)	O(1)	O(1)

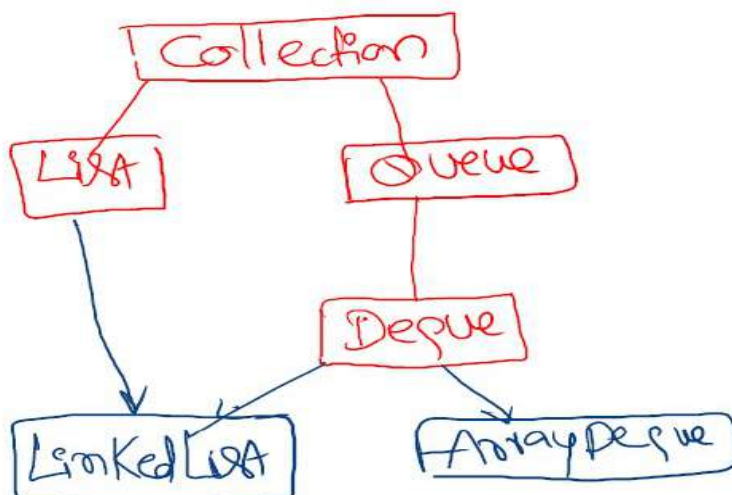


12.4. Applications of Queues

- ♦ Deques can be used in many of the real-world applications
 - a) Deque can be used as both Stack and Queue
 - b) Deque can be used to maintain recently visited URLs in the Browser History
 - c) Deque can be used to store list of Undo Operations
 - d) Deque is used in Steal Process Scheduling Algorithm
 - e) Clockwise and anti-clockwise operations in deque are very faster and will be useful in many problems
- etc

12.5. Deques in Java Collection

- ♦ Java Collections has two classes for Deque use-cases.
 - a) ArrayDeque
 - b) LinkedList



a) ArrayDeque

- ♦ ArrayDeque is a latest class from java.util package.
- ♦ ArrayDeque is implemented with Arrays.
- ♦ Use the ArrayDeque class in single-threaded environment
- ♦ If you want to use ArrayDeque in multi-threaded environment then you need to provide external synchronization.
- ♦ Most ArrayDeque operations run in amortized $O(1)$.
- ♦ ArrayDeque class is likely to be faster than LinkedList
- ♦ ArrayDeque can be used as
 - ♦ Stack
 - ♦ Queue

b) LinkedList

- ♦ LinkedList is the class from java.util package.
- ♦ LinkedList is implemented with Nodes.
- ♦ Use the LinkedList class in single-threaded environment
- ♦ If you want to use LinkedList in multi-threaded environment then you need to provide external synchronization.
- ♦ Most LinkedList operations run in $O(1)$.
- ♦ LinkedList class is likely to be slower than ArrayDeque
- ♦ LinkedList can be used as
 - ♦ List
 - ♦ Queue



12.5.1. Using Deque Built-In Methods

Lab1.java

```
package com.jlcindia.deques;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.LinkedList;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {
    public static void main(String[] args) {

        //Deque<Integer> mydeque = new ArrayDeque<>();
        Deque<Integer> mydeque = new LinkedList<>();

        System.out.println(mydeque);
        System.out.println(mydeque.size());
        System.out.println(mydeque.isEmpty());

        mydeque.offerFirst(10);
        mydeque.offerFirst(20);
        mydeque.offerFirst(30);
        System.out.println(mydeque);

        mydeque.offerLast(55);
        mydeque.offerLast(66);
        mydeque.offerLast(77);
        System.out.println(mydeque);
        System.out.println(mydeque.size());
        System.out.println(mydeque.isEmpty());

        System.out.println("-----");
    }
}
```




```
System.out.println(mydeque.peekFirst()); //30
System.out.println(mydeque.peekLast()); //77
System.out.println("-----");
mydeque.pollFirst(); //Deletes 30
mydeque.pollLast(); // Deletes 77
System.out.println("-----");
System.out.println(mydeque.peekFirst()); //20
System.out.println(mydeque.peekLast()); //66
System.out.println("-----");

    }
}
```

12.5.2. Using Deque Built-In Methods

Lab2.java

```
package com.jlcindia.deques;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.LinkedList;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab2 {
    public static void main(String[] args) {

        //Deque<Integer> mydeque = new ArrayDeque<>();
        Deque<Integer> mydeque = new LinkedList<>();

        System.out.println(mydeque);
        System.out.println(mydeque.size());
        System.out.println(mydeque.isEmpty());
    }
}
```



```
mydeque.addFirst(10);
mydeque.addFirst(20);
mydeque.addFirst(30);
System.out.println(mydeque);

mydeque.addLast(55);
mydeque.addLast(66);
mydeque.addLast(77);
System.out.println(mydeque);
System.out.println(mydeque.size());
System.out.println(mydeque.isEmpty());
System.out.println("-----");
System.out.println(mydeque.getFirst()); //30
System.out.println(mydeque.getLast()); //77
System.out.println("-----");
mydeque.removeFirst(); //Deletes 30
mydeque.removeLast(); // Deletes 77
System.out.println("-----");
System.out.println(mydeque.getFirst()); //20
System.out.println(mydeque.getLast()); //66
System.out.println("-----");

    }
}
```



12.5.3. Traversing Deque elements in Forward Order

Lab3.java

```
package com.jlcindia.deques;

import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    public static void main(String[] args) {
        Deque<Integer> mydeque = new LinkedList<>();
        System.out.println(mydeque);

        mydeque.addFirst(10);
        mydeque.addFirst(20);
        mydeque.addFirst(30);
        mydeque.addLast(55);
        mydeque.addLast(66);
        mydeque.addLast(77);
        System.out.println(mydeque);

        //1. Using Iterator
        Iterator<Integer> it = mydeque.iterator();
        while(it.hasNext()) {
            System.out.print(it.next()+"\t");
        }
        System.out.println("\n-----");
        //2.Using enhanced for loop
        for(Integer x: mydeque) {
            System.out.print(x+"\t");
        }
    }
}
```



12.5.4. Reverse the Deque Elements

Lab4.java

```
package com.jlcindia.deques;

import java.util.Deque;
import java.util.Iterator;
import java.util.LinkedList;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    public static void main(String[] args) {

        Deque<Integer> mydeque = new LinkedList<>();

        System.out.println(mydeque);

        mydeque.addFirst(10);
        mydeque.addFirst(20);
        mydeque.addFirst(30);
        mydeque.addLast(55);
        mydeque.addLast(66);
        mydeque.addLast(77);
        System.out.println(mydeque);

        //1. Using Iterator
        Iterator<Integer> it = mydeque.descendingIterator();
        while(it.hasNext()) {
            System.out.print(it.next()+"\t");
        }

    }
}
```

12.5.5. ArrayDeque can be used as Stack

Lab5.java

```
package com.jlcindia.deques;

import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab5 {
    public static void main(String[] args) {

        Deque<Integer> mystack = new ArrayDeque<>();
        System.out.println(mystack);

        mystack.push(10);
        mystack.push(20);
        mystack.push(30);
        mystack.push(40);
        mystack.push(50);

        System.out.println(mystack);
        System.out.println(mystack.peek());

        mystack.pop();
        System.out.println(mystack);
        System.out.println(mystack.peek());

    }
}
```

12.5.6. ArrayDeque can be used as Queue

Lab6.java

```
package com.jlcindia.deques;

import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab6 {
    public static void main(String[] args) {

        Deque<Integer> myqueue = new ArrayDeque<>();
        System.out.println(myqueue);

        myqueue.offer(10);
        myqueue.offer(20);
        myqueue.offer(30);
        myqueue.offer(40);
        myqueue.offer(50);

        System.out.println(myqueue);
        System.out.println(myqueue.peek());

        myqueue.poll();
        System.out.println(myqueue);
        System.out.println(myqueue.peek());

    }
}
```



12.6. Deque Implementation using Arrays

MyDeque.java

```
package com.jlcindia.deques.arrays1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Design Deque using Arrays
// Non-Circular Deque or Non-Circular Array

class MyDeque {

    int capacity;
    int size;
    Integer myarray[];

    public MyDeque(int capacity) {
        this.capacity = capacity;
        this.size = 0;
        this.myarray = new Integer[capacity];
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return (size==0);
    }

    public boolean isFull() {
        return (size==capacity);
    }
}
```



```
public int getFront() {
    if(isEmpty()) {
        return -1;
    }

    return 0;
}

public int getRear() {
    if(isEmpty()) {
        return -1;
    }

    return size-1;
}

public boolean offerFirst(int element) {

    if(isFull()) {
        return false;
    }

    for(int i= size-1;i>=0;i--) {
        myarray[i+1] = myarray[i];
    }

    myarray[0] = element;
    size++;
    return true;
}
```




```
public boolean offerLast(int element) {
    if(isFull())
        return false;

    myarray[size] = element;
    size++;
    return true;
}

public Integer pollFirst() {
    if(isEmpty())
        return null;

    int element = myarray[0];

    for(int i=0;i<size-1;i++) {
        myarray[i] = myarray[i+1];
    }

    myarray[size-1] = null;
    size--;
    return element;
}

public Integer pollLast() {
    if(isEmpty())
        return null;

    int element = myarray[size-1];

    myarray[size-1] = null;
    size--;
    return element;
}
```



```
public Integer peekFirst() {
    if(isEmpty()) {
        return null;
    }

    return myarray[0];
}

public Integer peekLast() {
    if(isEmpty()) {
        return null;
    }

    return myarray[size-1];
}

public String toString() {
    String str = "[";
    if (size != -1) {
        for (Integer x : myarray) {
            if (x != null)
                str = str + x + ",";
            else
                str= str+" null , ";
        }

        str = str + "]";
    }
    return str;
}
}
```



Lab7.java

```
package com.jlcindia.queues.arrays1;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab7 {
    public static void main(String[] args) {

        MyDeque mydeque = new MyDeque(6);

        System.out.println(mydeque);
        System.out.println(mydeque.size());
        System.out.println(mydeque.isEmpty());
        System.out.println(mydeque.isFull());
        System.out.println(mydeque.getFront());
        System.out.println(mydeque.getRear());
        System.out.println("-----");

        mydeque.offerFirst(10);
        mydeque.offerFirst(20);
        mydeque.offerFirst(30);

        System.out.println(mydeque);
        System.out.println(mydeque.isEmpty());
        System.out.println(mydeque.isFull());
        System.out.println(mydeque.getFront()); //0
        System.out.println(mydeque.getRear()); //2
        System.out.println("-----");

        mydeque.offerLast(55);
        mydeque.offerLast(66);
        mydeque.offerLast(77);
```



```
System.out.println(mydeque);
System.out.println(mydeque.size());
System.out.println(mydeque.isEmpty());
System.out.println(mydeque.isFull());
System.out.println(mydeque.getFront()); //0
System.out.println(mydeque.getRear()); //5

System.out.println("-----");

System.out.println(mydeque.peekFirst()); //30
System.out.println(mydeque.peekLast()); //77
System.out.println("-----");

mydeque.pollFirst(); //Deletes 30
mydeque.pollLast(); // Deletes 77
System.out.println(mydeque);

System.out.println("-----");
System.out.println(mydeque.peekFirst()); //20
System.out.println(mydeque.peekLast()); //66
System.out.println("-----");

    }
}
```



12.7. Deque Implementation using Arrays

MyDeque.java

```
package com.jlcindia.deques.arrays2;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Design Circular Deque using Arrays

class MyDeque {

    int front;
    int rear;
    int capacity;
    int size;
    Integer myarray[];

    public MyDeque(int capacity) {
        this.capacity = capacity;
        this.myarray = new Integer[capacity];

        this.size = 0;
        this.front = -1;
        this.rear = -1;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return (size == 0);
    }
}
```



```
public boolean isFull() {
    return (size == capacity);
}

public int getFront() {
    if (isEmpty())
        return -1;

    return front;
}

public int getRear() {
    if (isEmpty())
        return -1;

    return rear;
}

public boolean offerFirst(int element) {

    if (isFull())
        return false;

    if (front == -1) {
        front = 0;
        rear = 0;
    } else if (front == 0) {
        front = capacity - 1;
    } else {
        front = front - 1;
    }

    myarray[front] = element;
    size++;
    return true;
}
```



```
public boolean offerLast(int element) {
    if (isFull())
        return false;

    if (rear == -1) {
        front = 0;
        rear = 0;
    }
    if (rear == capacity - 1) {
        rear = 0;
    } else {
        rear = rear + 1;
    }

    myarray[rear] = element;
    size++;
    return true;
}
```

```
public Integer pollFirst() {
    if (isEmpty())
        return null;

    int element = myarray[front];
    myarray[front] = null;
    size--;

    if (front == rear) {
        front = -1;
        rear = -1;
    } else if (front == capacity - 1) {
        front = 0;
    } else {
        front = front + 1;
    }
    return element;
}
```



```
public Integer pollLast() {
    if (isEmpty()) {
        return null;
    }

    int element = myarray[rear];
    myarray[rear] = null;
    size--;

    if(front==rear) {
        front=-1;
        rear=-1;
    }else if (rear == 0) {
        rear = capacity-1;
    } else {
        rear = rear - 1;
    }

    return element;
}

public Integer peekFirst() {
    if (isEmpty())
        return null;

    return myarray[front];
}

public Integer peekLast() {
    if (isEmpty())
        return null;

    return myarray[rear];
}
```




```
public String toString() {
    String str = "[";
    if (size != -1) {
        for (Integer x : myarray) {
            if (x != null)
                str = str + x + ",";
            else
                str = str + " null , ";
        }
    }
    str = str + "]";
    return str;
}
}
```

Lab8.java

```
package com.jlcindia.deques.arrays2;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab8 {
    public static void main(String[] args) {
        MyDeque mydeque = new MyDeque(6);

        System.out.println(mydeque);
        System.out.println(mydeque.size());
        System.out.println(mydeque.isEmpty());
        System.out.println(mydeque.isFull());
        System.out.println(mydeque.getFront()); //-1
        System.out.println(mydeque.getRear()); //-1
        System.out.println("-----");

        mydeque.offerFirst(10);
        mydeque.offerFirst(20);
        mydeque.offerFirst(30);
        System.out.println(mydeque);
    }
}
```



```
System.out.println(mydeque.isEmpty());
System.out.println(mydeque.isFull());
System.out.println(mydeque.getFront()); //4
System.out.println(mydeque.getRear()); //0
System.out.println("-----");
```

```
mydeque.offerLast(55);
mydeque.offerLast(66);
mydeque.offerLast(77);
```

```
System.out.println(mydeque);
System.out.println(mydeque.size());
System.out.println(mydeque.isEmpty());
System.out.println(mydeque.isFull());
System.out.println(mydeque.getFront()); //4
System.out.println(mydeque.getRear()); //3
```

```
System.out.println("-----");
```

```
System.out.println(mydeque.peekFirst()); //30
System.out.println(mydeque.peekLast()); //77
System.out.println("-----");
```

```
mydeque.pollFirst();
mydeque.pollFirst();
mydeque.pollLast();
mydeque.pollLast();
System.out.println(mydeque);
```

```
System.out.println(mydeque.getFront()); //4
System.out.println(mydeque.getRear()); //3
System.out.println("-----");
System.out.println(mydeque.peekFirst()); //10
System.out.println(mydeque.peekLast()); //55
System.out.println("-----");
```

```
}
```

```
}
```

12.8. Deque Implementation using LinkedLists

Node.java

```
package com.jlcindia.deques.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Node{

    int data;
    Node next;
    Node prev;

    Node(int data){
        this.data=data;
        this.next=null;
        this.prev=null;
    }
}
```

MyDeque.java

```
package com.jlcindia.deques.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyDeque {

    Node front;
    Node rear;
    int size;
```



```
public MyDeque() {
    this.size = 0;
    this.front = null;
    this.rear = null;
}

public int size() {
    return size;
}

public boolean isEmpty() {
    return (size == 0);
}

public boolean offerFirst(int element) {

    Node temp =new Node(element);

    if(front==null) {
        front=temp;
        rear=temp;
        size++;
        return true;
    }

    temp.next = front;
    front.prev=temp;
    front=front.prev;

    size++;
    return true;
}
```



```
public boolean offerLast(int element) {
```

```
    Node temp = new Node(element);
```

```
        if(front==null) {
            front=temp;
            rear=temp;
            size++;
            return true;
        }
```

```
        rear.next = temp;
        temp.prev = rear;
        rear = rear.next;
```

```
        size++;
        return true;
```

```
    }
```

```
public Integer pollFirst() {
```

```
    if(size==0) {
        return null;
    }
```

```
    Node temp = front;
    front = front.next;
    front.prev=null;
```

```
    temp.next=null;
    size--;
```

```
    return temp.data;
```

```
}
```



```
public Integer pollLast() {  
  
    if(size==0) {  
        return null;  
    }  
  
    Node temp = rear;  
    rear = rear.prev;  
    rear.next=null;  
  
    temp.prev=null;  
    size--;  
  
    return temp.data;  
}  
  
public Integer peekFirst() {  
    if (isEmpty()) {  
        return null;  
    }  
  
    return front.data;  
}  
  
public Integer peekLast() {  
    if (isEmpty()) {  
        return null;  
    }  
  
    return rear.data;  
}
```



```
public String toString() {
    if (this.front == null) {
        return "[]";
    }

    String str = "[";
    Node currentNode = this.front;
    while (currentNode != null) {
        str = str + "" + currentNode.data + " , ";
        currentNode = currentNode.next;
    }
    str = str.substring(0, str.length() - 2);
    str = str + "]";

    return str;
}
}
```

Lab9.java

```
package com.jlcindia.deques.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab9 {
    public static void main(String[] args) {

        MyDeque mydeque = new MyDeque();

        System.out.println(mydeque);
        System.out.println(mydeque.size());
        System.out.println(mydeque.isEmpty());
        System.out.println("-----");
    }
}
```



```
mydeque.offerFirst(10);
mydeque.offerFirst(20);
mydeque.offerFirst(30);

System.out.println(mydeque);
System.out.println(mydeque.size());
System.out.println(mydeque.isEmpty());
System.out.println("-----");

mydeque.offerLast(55);
mydeque.offerLast(66);
mydeque.offerLast(77);

System.out.println(mydeque);

System.out.println("-----");
System.out.println(mydeque.peekFirst()); //30
System.out.println(mydeque.peekLast()); //77
System.out.println("-----");

mydeque.pollFirst();
mydeque.pollFirst();
mydeque.pollLast();
mydeque.pollLast();
System.out.println(mydeque);

System.out.println("-----");
System.out.println(mydeque.peekFirst()); //10
System.out.println(mydeque.peekLast()); //55
System.out.println("-----");
```

```
}
```

```
}
```


DSA

Module 13

Searching

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

13. Searching

- ◆ There are two Algorithms for searching an element
 - ✓ Linear Search
 - ✓ Binary Search

Linear Search

- ◆ Linear search is a sequential searching algorithm where we start from one end and check every element of the list until the required element is found.
- ◆ If the required element is found then we return the index of that element otherwise we return -1.
- ◆ It is the simplest searching algorithm.

Binary Search

- ◆ Binary Search can be implemented only on a Sorted list of items.
- ◆ If the elements are not sorted already, then we need to sort them first.
- ◆ In Binary Search, Element is always searched in the middle of a portion of an array and repeatedly dividing the search interval in half.
- ◆ If the required element is found then we return the index of that element otherwise we return -1.

13.1. Time Complexity

<u>Operation</u>	<u>Using Iteration</u>	<u>Using Recursion</u>
Linear Search	$O(n)$	$O(n)$
Binary Search	$O(\log n)$	$O(\log n)$

13.2.1. Linear Search using Iteration

Lab1.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static int linearSearch(int arr[],int element) {

        for(int i=0;i<arr.length;i++) {
            if(element == arr[i]){
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50,60,70};
        int element = 50;

        int result = linearSearch(arr,element);
        System.out.println(result);
    }
}
```

13.2.2. Linear Search using Recursion

Lab2.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {

    public static int linearSearch(int arr[],int element,int size) {

        if(size==0)
            return -1;
        else if(arr[size-1]==element)
            return size-1;

        return linearSearch(arr, element, size-1);
    }

    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50,60,70};
        int element = 50;
        int size=arr.length;

        int result = linearSearch(arr,element,size);
        System.out.println(result);
    }
}
```



13.3.1. Binary Search using Iteration

Lab3.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    public static int binarySearch(int arr[],int element) {

        int low=0;
        int high=arr.length-1;

        while(low<=high) {
            int mid = (low+high)/2;

            if(element == arr[mid])
                return mid;
            else if(element > arr[mid])
                low = mid+1;
            else if(element < arr[mid])
                high = mid-1;
        }

        return -1;
    }
    public static void main(String[] args) {
        int arr[] = {10,20,30,40,50,60,70};
        int element = 25;

        int result = binarySearch(arr,element);
        System.out.println(result);
    }
}
```



13.3.2. Binary Search using Recursion

Lab4.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    public static int binarySearch(int arr[],int element,int low,int high) {
        if(low>high)
            return -1;

        int mid = (low+high)/2;
        if(element == arr[mid]) {
            return mid;
        }
        else if(element > arr[mid]) {
            low = mid+1;
            return binarySearch(arr,element,low,high);
        }
        else if(element < arr[mid]) {
            high = mid-1;
            return binarySearch(arr,element,low,high);
        }
        return -1;
    }
    public static void main(String[] args) {
        int arr[] = {10,20,30,40,50,60,70};
        int element = 10;
        int low = 0;          int high=arr.length-1;

        int result = binarySearch(arr,element,low,high);
        System.out.println(result);
    }
}
```



13.4. First Occurrence of element in Sorted Array

Lab5A.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5A {

    public static int firstOccurence(int arr[],int element) {

        for(int i=0;i<arr.length;i++) {
            if(element == arr[i]){
                return i;
            }
        }

        return -1;
    }

    public static void main(String[] args) {

        int arr[] = {10,20,20,30,30,30,50};
        int element = 20;

        int result = firstOccurence(arr,element);
        System.out.println(result);

    }
}
```


13.4. First Occurrence of element in Sorted Array

Lab5.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5 {

    public static int firstOccurence(int arr[], int element) {

        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (element > arr[mid])
                low = mid + 1;
            else if (element < arr[mid])
                high = mid - 1;
            else if (element == arr[mid]) {
                if (mid == 0 || arr[mid - 1] != arr[mid])
                    return mid;
                else {
                    high = mid - 1;
                }
            }
        }

        return -1;
    }
}
```



```
public static void main(String[] args) {  
    int arr[] = { 10, 20, 20, 30, 30, 30, 50 };  
    int element = 20;  
    int result = firstOccurence(arr, element);  
    System.out.println(result);  
}  
}
```

13.5. Last Occurence of element in Sorted Array

Lab6A.java

```
package com.jlcindia.searching;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public class Lab6A {  
    public static int lastOccurence(int arr[],int element) {  
        for(int i=arr.length-1;i>=0;i--) {  
            if(element == arr[i]){  
                return i;  
            }  
        }  
        return -1;  
    }  
    public static void main(String[] args) {  
        int arr[] = {10,20,20,30,30,30,50};  
        int element = 30;  
        int result = lastOccurence(arr,element);  
        System.out.println(result);  
    }  
}
```



13.5. Last Occurrence of element in Sorted Array

Lab6.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab6 {
    public static int lastOccurence(int arr[], int element) {
        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (element > arr[mid])
                low = mid + 1;
            else if (element < arr[mid])
                high = mid - 1;
            else if (element == arr[mid]) {
                if (mid == arr.length - 1 || arr[mid] != arr[mid + 1])
                    return mid;
                else {
                    low = mid + 1;
                }
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int arr[] = { 10, 20, 20, 30, 30, 30, 50 };
        int element = 30;
        int result = lastOccurence(arr, element);
        System.out.println(result);
    }
}
```



13.6. Count Occurrence of element in Sorted Array

Lab7.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab7 {

    public static int firstOccurence(int arr[], int element) {

        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (element > arr[mid])
                low = mid + 1;
            else if (element < arr[mid])
                high = mid - 1;
            else if (element == arr[mid]) {
                if (mid == 0 || arr[mid - 1] != arr[mid])
                    return mid;
                else {
                    high = mid - 1;
                }
            }
        }

        return -1;
    }
}
```



```
public static int lastOccurence(int arr[], int element) {
    int low = 0;
    int high = arr.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;

        if (element > arr[mid])
            low = mid + 1;
        else if (element < arr[mid])
            high = mid - 1;
        else if (element == arr[mid]) {
            if (mid == arr.length - 1 || arr[mid] != arr[mid + 1])
                return mid;
            else {
                low = mid + 1;
            }
        }
    }
    return -1;
}

public static int countOccurence(int arr[], int element) {
    int first = firstOccurence(arr, element);
    if (first == -1)
        return 0;

    int last = lastOccurence(arr, element);
    int count = last - first + 1;
    return count;
}

public static void main(String[] args) {
    int arr[] = { 10, 20, 20, 30, 30, 30, 50 };
    int element = 15;
    int result = countOccurence(arr, element);
    System.out.println(result);
}
}
```

DSA

Module 14

Sorting

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

14. Sorting

- ♦ Sorting Algorithm is used to arrange elements of an array/list in an order specified.
- ♦ Following are the list of Sorting Algorithms

- 1) Bubble Sort
- 2) Selection Sort
- 3) Insertion Sort
- 4) Merge Sort
- 5) Quicksort
- 6) Heap Sort
- 7) Counting Sort
- 8) Radix Sort
- 9) Bucket Sort
- 10) Shell Sort



14.1. Complexity of Sorting Algorithms

<u>Sorting Algorithm</u>	<u>Time Complexity Best</u>	<u>Time Complexity Worst</u>	<u>Time Complexity Average</u>	<u>Space Complexity</u>
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Max
Radix Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Max
Bucket Sort	$O(n+k)$	$O(n^2)$	$O(n)$	$O(n+k)$
Shell Sort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(1)$

14.2. Stability of Sorting Algorithms

- a) Stable Sorting Algorithms
- b) Unstable Sorting Algorithms

a) Stable Sorting Algorithms:

- ♦ If equal elements are not reordered as a result of the sort then that sorting algorithm is Stable.

Ex:

Bubble Sort
Insertion Sort
Merge Sort
Counting Sort
Radix Sort
Bucket Sort

b) Unstable Sorting Algorithms:

- ♦ If equal elements are reordered as a result of the sort then that sorting algorithm is Unstable.

Ex:

Selection Sort
Quicksort
Heap Sort
Shell Sort

14.3. Sorting in Java

- a) Arrays.sort()
- b) Collections.sort()

a) Arrays.sort():

- ♦ Arrays.sort() can be used in two ways.

- 1) For Array of Primitives(int,char,double etc)
- 2) For Array of Objects(Integer, Student, Customer etc)

1) For Array of Primitives(int,char,double etc):

- ♦ Sorts the specified array into ascending numerical order.
- ♦ Sorting algorithm is a Dual-Pivot Quicksort
- ♦ This algorithm offers $O(n \log(n))$ performance on many data sets that cause other quicksorts to degrade to $O(n^2)$ performance

2) For Array of Objects(Integer, Student, Customer etc)

- ♦ Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.
 - ♦ All elements in the array must implement the Comparable interface.
 - ♦ This sort is guaranteed to be stable: equal elements will not be reordered as a result of the sort.
 - ♦ Sorting algorithm is a MergeSort
 - ♦ This algorithm offers $O(n \log(n))$ performance
-
- ✓ **Arrays.sort() uses QuickSort for Array of Primitives**
 - ✓ **Arrays.sort() uses MergeSort for Array of Objects**

b) Collections.sort()

1) For Lists (ArrayList, LinkedList etc)

- ♦ Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.
 - ♦ All elements in the array must implement the Comparable interface.
 - ♦ This sort is guaranteed to be stable: equal elements will not be reordered as a result of the sort.
 - ♦ Sorting algorithm is a MergeSort
 - ♦ This algorithm offers $O(n \log(n))$ performance
- ✓ **Collections.sort() uses MergeSort for Collection of Objects**

14.4. Labs on Arrays.sort()

14.4.1. Sort the Array

Lab1.java

```
package com.jlccindia.arrays.sort;

import java.util.Arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {

    public static void main(String[] args) {

        int arr1[] = {10,5,20,15,30,25};
        char arr2[] = {'C','D','E','B','A'};

        Arrays.sort(arr1);
        System.out.println(Arrays.toString(arr1));

        Arrays.sort(arr2);
        System.out.println(Arrays.toString(arr2));

    }
}
```

14.4.2. Sort the Sub-Array

Lab2.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {

    public static void main(String[] args) {

        int arr1[] = {10,5,20,15,30,25};
        char arr2[] = {'C','D','E','B','A'};

        Arrays.sort(arr1,2,6);
        System.out.println(Arrays.toString(arr1));

        Arrays.sort(arr2,3,5);
        System.out.println(Arrays.toString(arr2));

    }
}
```

14.4.3. Sort Array in ASC and DESC Order

Lab3.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;
import java.util.Collections;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab3 {

    public static void main(String[] args) {

        Integer arr[] = {10,5,20,15,30,25};

        Arrays.sort(arr);
        System.out.println(Arrays.toString(arr));

        Arrays.sort(arr,Collections.reverseOrder());
        System.out.println(Arrays.toString(arr));

    }
}
```



14.4.4. Sort Array in ASC and DESC Order

Lab4.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;
import java.util.Comparator;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyIntegerComparator implements Comparator<Integer>{

    @Override
    public int compare(Integer a, Integer b) {
        return b-a;
    }
}

public class Lab4{

    public static void main(String[] args) {

        Integer arr[] = {10,5,20,15,30,25};

        Arrays.sort(arr);
        System.out.println(Arrays.toString(arr));

        Arrays.sort(arr,new MyIntegerComparator());
        System.out.println(Arrays.toString(arr));

    }
}
```




14.4.5. Arrange as Odd First and Even Next

Lab5.java

```
package com.jlcindia.arrays.sort;
import java.util.Arrays;
import java.util.Comparator;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyEvenComparator implements Comparator<Integer>{
    @Override
    public int compare(Integer a, Integer b) {
        return a%2-b%2;
    }
}

class MyOddComparator implements Comparator<Integer>{
    @Override
    public int compare(Integer a, Integer b) {
        return b%2-a%2;
    }
}

public class Lab5{
    public static void main(String[] args) {

        Integer arr[] = {10,5,20,15,30,25};

        Arrays.sort(arr,new MyEvenComparator());
        System.out.println(Arrays.toString(arr));

        Arrays.sort(arr,new MyOddComparator());
        System.out.println(Arrays.toString(arr));
    }
}
```

14.4.6. Using Comparable

Lab6.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Customer implements Comparable<Customer>{

    int cid;
    String cname;
    long phone;

    public Customer() {}

    public Customer(int cid, String cname, long phone) {
        super();
        this.cid = cid;
        this.cname = cname;
        this.phone = phone;
    }

    @Override
    public String toString() {
        return "[" + cid + ", " + cname + ", " + phone + "]";
    }

    @Override
    public int compareTo(Customer cust) {
        return this.cid-cust.cid;
    }
}
```

```
public class Lab6{  
  
    public static void main(String[] args) {  
  
        Customer cust1 = new Customer(102,"sd",222);  
        Customer cust2 = new Customer(103,"ds",333);  
        Customer cust3 = new Customer(101,"sri",111);  
  
        Customer customers[] = {cust1,cust2,cust3};  
  
        Arrays.sort(customers);  
  
        for(Customer cust:customers) {  
            System.out.println(cust);  
        }  
    }  
}
```

14.4.7. Using Comparator

Lab7.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;
import java.util.Comparator;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyMarksComparator implements Comparator<Student>{

    @Override
    public int compare(Student stu1, Student stu2) {
        return (int) (stu1.marks-stu2.marks);
    }
}

class Student implements Comparable<Student>{

    int sid;
    String sname;
    double marks;

    public Student() {}

    public Student(int sid, String sname, double marks) {
        super();
        this.sid = sid;
        this.sname = sname;
        this.marks = marks;
    }
}
```



```
@Override
public String toString() {
    return "[" + sid + ", " + sname + ", " + marks + "]";
}

@Override
public int compareTo(Student stu) {
    return this.sid-stu.sid;
}
}

public class Lab7{

    public static void main(String[] args) {

        Student stu1 = new Student(102,"sd",70);
        Student stu2 = new Student(103,"ds",60);
        Student stu3 = new Student(101,"sri",50);

        Student students[] = {stu1,stu2,stu3};

        Arrays.sort(students);

        for(Student stu:students) {
            System.out.println(stu);
        }

        Arrays.sort(students,new MyMarksComparator());

        for(Student stu:students) {
            System.out.println(stu);
        }

    }
}
```



14.5. Labs on Collections.sort()

14.5.1. Sort the List in ASC and DESC

Lab8.java

```
package com.jlcindia.collections.sort;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab8 {
    public static void main(String[] args) {

        List<Integer> mylist = new ArrayList<>();
        mylist.add(10);
        mylist.add(5);
        mylist.add(20);
        mylist.add(25);
        mylist.add(15);

        Collections.sort(mylist);
        System.out.println(mylist);

        Collections.sort(mylist,Collections.reverseOrder());
        System.out.println(mylist);

    }
}
```

14.5.2. Using Comparable

Lab9.java

```
package com.jlcindia.collections.sort;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Customer implements Comparable<Customer>{
    int cid;
    String cname;
    long phone;

    public Customer() {}
    public Customer(int cid, String cname, long phone) {
        super();
        this.cid = cid;
        this.cname = cname;
        this.phone = phone;
    }

    @Override
    public String toString() {
        return "[" + cid + ", " + cname + ", " + phone + "]";
    }

    @Override
    public int compareTo(Customer cust) {
        return this.cid-cust.cid;
    }
}
```



```
public class Lab9{  
  
    public static void main(String[] args) {  
  
        Customer cust1 = new Customer(102,"sd",222);  
        Customer cust2 = new Customer(103,"ds",333);  
        Customer cust3 = new Customer(101,"sri",111);  
  
        List<Customer> customers = new ArrayList<>();  
        customers.add(cust1);  
        customers.add(cust2);  
        customers.add(cust3);  
  
        Collections.sort(customers);  
  
        for(Customer cust:customers) {  
            System.out.println(cust);  
        }  
  
    }  
}
```


14.5.3. Using Comparator

Lab10.java

```
package com.jlcindia.collections.sort;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyMarksComparator implements Comparator<Student> {

    @Override
    public int compare(Student stu1, Student stu2) {
        return (int) (stu1.marks - stu2.marks);
    }
}

class Student implements Comparable<Student> {

    int sid;
    String sname;
    double marks;

    public Student() { }
    public Student(int sid, String sname, double marks) {
        super();
        this.sid = sid;
        this.sname = sname;
        this.marks = marks;
    }
}
```



```
@Override
public String toString() {
    return "[" + sid + ", " + sname + ", " + marks + "]";
}

@Override
public int compareTo(Student stu) {
    return this.sid - stu.sid;
}
}

public class Lab10 {
    public static void main(String[] args) {

        Student stu1 = new Student(102, "sd", 70);
        Student stu2 = new Student(103, "ds", 60);
        Student stu3 = new Student(101, "sri", 50);

        List<Student> students = new ArrayList<>();
        students.add(stu1);
        students.add(stu2);
        students.add(stu3);

        Collections.sort(students);

        for (Student stu : students) {
            System.out.println(stu);
        }
        System.out.println("-----");

        Collections.sort(students, new MyMarksComparator());

        for (Student stu : students) {
            System.out.println(stu);
        }
    }
}
```

14.6. Exploring Bubble Sort

- ♦ Bubble sort is the simplest sorting algorithm.
- ♦ It works by iterating the input array from the first element to the last, comparing each pair of elements and swapping them if needed.
- ♦ Bubble sort continues its iterations until no more swaps are needed.
- ♦ **The only significant advantage that bubble sort has over other implementations is that it can detect whether the input list is already sorted or not.**

Complexity of Bubble Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Worst	$O(n^2)$
Time Complexity - Average	$O(n^2)$
Space Complexity	$O(1)$

14.6.1. Bubble Sort Implementation

Lab11.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab11 {
    public static void bubbleSort(int arr[]) {

        int n= arr.length;
```



```
        for(int i=0;i<n;i++) {
            for(int j=0;j<n-i-1;j++) {
                if(arr[j]>arr[j+1]) {
                    int temp = arr[j];
                    arr[j]=arr[j+1];
                    arr[j+1]=temp;
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr[] = {5,9,15,8,20,7,10};
        System.out.println(Arrays.toString(arr));

        bubbleSort(arr);
        System.out.println(Arrays.toString(arr));
    }
}
```

14.6.2. Optimized Bubble Sort

Lab12.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab12 {

    public static void bubbleSort(int arr[]) {

        int n= arr.length;
```



```
        for(int i=0;i<n;i++) {  
            boolean swaped=false;  
            for(int j=0;j<n-i-1;j++) {  
                if(arr[j]>arr[j+1]) {  
                    int temp = arr[j];  
                    arr[j]=arr[j+1];  
                    arr[j+1]=temp;  
                    swaped=true;  
                }  
            }  
            if(swaped==false) {  
                break;  
            }  
        }  
    }  
  
    public static void main(String[] args) {  
        int arr[] = {5,9,15,8,20,7,10};  
        System.out.println(Arrays.toString(arr));  
        bubbleSort(arr);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```

14.7. Exploring Selection Sort

- ♦ Easy to implement the Selection sort
- ♦ Selection sort works well for small data sets.
- ♦ Selection sort is an in-place sorting algorithm.
- ♦ Selection sort is not a Stable sorting algorithm.

Algorithm

- 1) Find the minimum value in the list
- 2) Swap it with the value in the current position
- 3) Repeat this process for all the elements until the entire array is sorted

- ♦ This algorithm is called selection sort since it repeatedly selects the smallest element.

Complexity of Selection Sort:

Time Complexity - Best	$O(n^2)$
Time Complexity - Average	$O(n^2)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(1)$

14.7.1. Selection Sort Implementation

Lab13.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
public class Lab13 {  
  
    public static void selectionSort(int arr[]) {  
  
        int n = arr.length;  
  
        for (int i = 0; i < n-1; i++) {  
            int min_index = i;  
            for (int j = i + 1; j < n; j++) {  
                if (arr[j] < arr[min_index])  
                    min_index = j;  
            }  
  
            int temp = arr[min_index];  
            arr[min_index] = arr[i];  
            arr[i] = temp;  
        }  
    }  
  
    public static void main(String[] args) {  
  
        int arr[] = { 10, 5, 30, 25, 15, 20 };  
  
        System.out.println(Arrays.toString(arr));  
        selectionSort(arr);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```

14.8. Exploring Insertion Sort

- ♦ Insertion sort is a simple and efficient comparison sort.
- ♦ Insertion sort is an in-place sorting algorithm.
- ♦ Insertion sort is a Stable sorting algorithm.
- ♦ In this algorithm, each iteration
 - ✓ Removes an element from the input data,
 - ✓ Inserts it into the correct position in the already-sorted list until no input elements remain.

Algorithm

- 1) Consider some part is already Sorted
 - 2) Take i^{th} element and Insert it in the Correct position
 - 3) Repeat this process for all the elements until the entire array is sorted
- ♦ This algorithm is called Insertion Sort since it repeatedly Inserts the Element in Correct position in the Sorted part.

Complexity of Insertion Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Average	$O(n^2)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(1)$

14.8.1. Insertion Sort Implementation

Lab14.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```




```
*/  
public class Lab14 {  
  
    public static void insertionSort(int arr[]) {  
  
        int n = arr.length;  
  
        for (int i = 1; i < n; i++) {  
            int key = arr[i];  
            int j=i-1;  
            while(j>=0 && arr[j]>key) {  
                arr[j+1] = arr[j];  
                j--;  
            }  
  
            arr[j+1] = key;  
  
        }  
    }  
  
    public static void main(String[] args) {  
  
        int arr[] = { 10, 5, 30, 25, 15, 20 };  
  
        System.out.println(Arrays.toString(arr));  
        insertionSort(arr);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```

14.9. Exploring Merge Sort

- ♦ Merge sort is an example of the divide and conquer strategy.
- ♦ Merging is the process of combining two sorted arrays into one bigger sorted array
- ♦ Merge sort is Quick sort's complement
- ♦ Merge sort accesses the data in a sequential manner
- ♦ This algorithm is used for sorting a linked list
- ♦ Merge sort divides the list into two parts; then each part is conquered individually.
- ♦ Merge sort starts with the small arrays and finishes with the largest one.
- ♦ Merge sort is not an in-place sorting algorithm.
- ♦ Merge sort is a Stable sorting algorithm.

Algorithm

- 1) Divide the Input List into two parts.
- 2) Solve each part recursively.
- 3) After solving the sub problems, they are merged by scanning the resultant sub problems.

This algorithm is called Merge Sort since it is merging two sorted arrays to make one bigger sorted array.

Complexity of Merge Sort:

Time Complexity - Best	$O(n \log n)$
Time Complexity - Average	$O(n \log n)$
Time Complexity - Worst	$O(n \log n)$
Space Complexity	$O(n)$



14.9.1. Merge two Sorted Arrays

Lab15.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab15 {
    public static void mergeArrays(int a[],int b[]) {
        int m = a.length;
        int n = b.length;

        int i=0;
        int j=0;
        while(i<m && j<n) {
            if(a[i]<=b[j]) {
                System.out.println(a[i]);
                i++;
            }
            else {
                System.out.println(b[j]);
                j++;
            }
        }
        while(i<m ) {
            System.out.println(a[i]);
            i++;
        }
        while( j<n) {
            System.out.println(b[j]);
            j++;
        }
    }
}
```



```
public static void main(String[] args) {  
  
    int a[] = { 10, 20, 50 };  
    int b[] = { 5, 15, 50 };  
    mergeArrays(a,b);  
  
}  
}
```

14.9.2. Implement Merge Function

Lab16.java

```
package com.jlcindia.sorting;  
  
import java.util.Arrays;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
  
public class Lab16 {  
  
    public static void merge(int arr[], int low, int mid, int high) {  
  
        int m = mid - low + 1;  
        int n = high - mid;  
  
        int left[] = new int[m];  
        for (int i = 0; i < m; i++) {  
            left[i] = arr[low + i];  
        }  
  
        int right[] = new int[n];  
        for (int j = 0; j < n; j++) {  
            right[j] = arr[mid + j+1];  
        }  
  
    }  
  
}
```



```
int i = 0;
int j = 0;
int k=low;

while (i < m && j < n) {
    if (left[i] <= right[j]) {
        arr[k] = left[i];
        i++;
        k++;
    } else {
        arr[k] = right[j];
        j++;
        k++;
    }
}
while (i < m) {
    arr[k] = left[i];
    i++;
    k++;
}
while (j < n) {
    arr[k] = right[j];
    j++;
    k++;
}
}

public static void main(String[] args) {
    int arr[] = { 10, 20, 50, 5, 15, 50 };
    int low = 0;    int high = 5;    int mid = 2;

    System.out.println(Arrays.toString(arr));
    merge(arr, low, mid, high);
    System.out.println(Arrays.toString(arr));
}
}
```



14.9.3. Merge Sort Algorithm

Lab17.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab17 {
    public static void merge(int arr[], int low, int mid, int high) {

        int m = mid - low + 1;
        int n = high - mid;

        int left[] = new int[m];
        for (int i = 0; i < m; i++) {
            left[i] = arr[low + i];
        }

        int right[] = new int[n];
        for (int j = 0; j < n; j++) {
            right[j] = arr[mid + j + 1];
        }

        int i = 0;
        int j = 0;
        int k = low;

        while (i < m && j < n) {
            if (left[i] <= right[j]) {
                arr[k] = left[i];
                i++;
                k++;
            } else {
                arr[k] = right[j];
                j++;
                k++;
            }
        }
    }
}
```



```
                j++;
                k++;
            }
        }

        while (i < m) {
            arr[k] = left[i];
            i++;
            k++;
        }

        while (j < n) {
            arr[k] = right[j];
            j++;
            k++;
        }

    }

    public static void mergeSort(int arr[], int low, int high) {
        if(high>low) {
            int mid = low + (high - low)/2;
            mergeSort(arr,low,mid);
            mergeSort(arr,mid+1,high);
            merge(arr, low, mid, high);

        }
    }

    public static void main(String[] args) {
        int arr[] = { 10, 35, 20, 15, 5, 25 };
        int low = 0;           int high = 5;
        System.out.println(Arrays.toString(arr));
        mergeSort(arr, low, high);
        System.out.println(Arrays.toString(arr));
    }
}
```

14.10. Exploring Quicksort

- ♦ Quick sort is an example of the divide and conquer strategy.
- ♦ It is one of the famous algorithms among comparison-based sorting algorithms
- ♦ It is also called partition exchange sort.
- ♦ It uses recursive calls for sorting the elements,
- ♦ Quick sort is an in-place sorting algorithm.
- ♦ Quick sort is a Non-Stable sorting algorithm.

Algorithm

- ♦ **The recursive algorithm consists of 4 steps:**
 - 1) If there are one or no elements in the array to be sorted, return.
 - 2) Pick an element in the array to serve as the “pivot” point.
 - 3) Split the array into two parts – one with elements larger than the pivot and the other with elements smaller than the pivot.
 - 4) Recursively repeat the algorithm for both halves of the original array.

Complexity of Quick Sort:

Time Complexity - Best	$O(n \log n)$
Time Complexity - Average	$O(n \log n)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(n)$

14.10.1. Partitioning the given Array

Lab18.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
```




* @Company: Java Learning Center

**/

```
public class Lab18 {  
    public static void simplePartition(int arr[], int low, int high, int pivot) {  
  
        int n = high - low + 1;  
        int temp[] = new int[n];  
        int k = 0;  
  
        for(int i = low; i <= high; i++) {  
            if(arr[i] <= arr[pivot]) {  
                temp[k++] = arr[i];  
            }  
        }  
  
        for(int i = low; i <= high; i++) {  
            if(arr[i] > arr[pivot]) {  
                temp[k++] = arr[i];  
            }  
        }  
  
        for(int i = low; i <= high; i++) {  
            arr[i] = temp[i - low];  
        }  
    }  
  
    public static void main(String[] args) {  
        int arr[] = { 5, 13, 6, 9, 12, 4, 8, 7};  
        int low = 0;  
        int high = arr.length - 1;  
        int pivot = high;  
        System.out.println(Arrays.toString(arr));  
        simplePartition(arr, low, high, pivot);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```



14.10.2. Lomuto Partition

Lab19.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab19 {

    public static int lomutoPartition(int arr[], int low, int high) {

        int pivot = arr[high];
        int i=low-1;

        for(int j=low;j<=high-1;j++) {
            if(arr[j]<pivot) {
                i++;
                int temp= arr[i];
                arr[i]=arr[j];
                arr[j]=temp;
            }
        }

        int temp= arr[i+1];
        arr[i+1]=arr[high];
        arr[high]=temp;

        return (i+1);
    }

    public static void main(String[] args) {

        int arr[] = { 5,13,6,9,12,4,8,7};
        int low = 0;
```



```
        int high = arr.length-1;

        System.out.println(Arrays.toString(arr));
        int pivotIndex= lomutoPartition(arr, low, high);
        System.out.println(Arrays.toString(arr));
        System.out.println(pivotIndex);
    }
}
```

14.10.3. Hoares Partition

Lab20.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab20 {

    public static int hoaresPartition(int arr[], int low, int high) {

        int pivot = arr[low];
        int i = low - 1;
        int j = high + 1;

        while (true) {
            do {
                i++;
            } while (arr[i] < pivot);

            do {
                j--;
            } while (arr[j] > pivot);
```



```
        if (i >= j) {
            return j;
        }

        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

public static void main(String[] args) {

    int arr[] = {5,3,8,4,2,7,1,10};
    int low = 0;
    int high = arr.length-1;

    System.out.println(Arrays.toString(arr));
    int pivotIndex = hoaresPartition(arr, low, high);
    System.out.println(Arrays.toString(arr));
    System.out.println(pivotIndex);
}
}
```

14.10.4. Quick Sort with Lomuto Partition

Lab21.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab21 {
    public static int lomutoPartition(int arr[], int low, int high) {
```



```
int pivot = arr[high];
int i=low-1;

for(int j=low;j<=high-1;j++) {
    if(arr[j]<pivot) {
        i++;
        int temp= arr[i];
        arr[i]=arr[j];
        arr[j]=temp;
    }
}

int temp= arr[i+1];
arr[i+1]=arr[high];
arr[high]=temp;

return (i+1);
}

public static void quickSort(int arr[], int low, int high) {
    if(low<high) {
        int pivotIndex=lomutoPartition(arr, low, high);
        quickSort(arr, low, pivotIndex-1);
        quickSort(arr, pivotIndex+1, high);
    }
}

public static void main(String[] args) {
    int arr[] = {8,4,7,9,3,10,5};
    int low = 0;
    int high = arr.length-1;
    System.out.println(Arrays.toString(arr));
    quickSort(arr,low,high);
    System.out.println(Arrays.toString(arr));
}
}
```



14.10.5. Quick Sort with Hoares Partition

Lab22.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab22 {
    public static int hoaresPartition(int arr[], int low, int high) {

        int pivot = arr[low];
        int i = low - 1;
        int j = high + 1;

        while (true) {
            do {
                i++;
            } while (arr[i] < pivot);

            do {
                j--;
            } while (arr[j] > pivot);

            if (i >= j) {
                return j;
            }

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}
```



```
public static void quickSort(int arr[], int low, int high) {  
  
    if(low<high) {  
        int pivotIndex=hoaresPartition(arr, low, high);  
        quickSort(arr, low, pivotIndex);  
        quickSort(arr, pivotIndex+1, high);  
    }  
  
}  
  
public static void main(String[] args) {  
  
    int arr[] = {8,4,7,9,3,10,5};  
    int low = 0;  
    int high = arr.length-1;  
  
    System.out.println(Arrays.toString(arr));  
    quickSort(arr,low,high);  
    System.out.println(Arrays.toString(arr));  
  
}  
}
```



14.11. Exploring Cycle Sort

- ♦ Cycle Sort does the Sorting with minimum memory writes and can be usefull for cases where the memory writes are costly.
- ♦ Selection sort is an in-place sorting algorithm.
- ♦ Selection sort is not a Stable sorting algorithm.

Complexity of Cycle Sort:

Time Complexity - Best	$O(n^2)$
Time Complexity - Average	$O(n^2)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(1)$

14.11.1. Cycle Sort with Distinct elements

Lab23.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab23 {

    public static void cycleSort(int arr[]) {

        int n = arr.length;
        for (int cycle = 0; cycle < n - 1; cycle++) {
            int item = arr[cycle];
            int position = cycle;
            for (int j = cycle + 1; j < n; j++) {
                if (arr[j] < item)
                    position++;
            }
        }
    }
}
```




```
        int temp = item;
        item = arr[position];
        arr[position] = temp;

        while (position != cycle) {
            position = cycle;
            for (int j = cycle + 1; j < n; j++) {
                if (arr[j] < item)
                    position++;
            }

            temp = item;
            item = arr[position];
            arr[position] = temp;
        }
    }

    public static void main(String[] args) {

        int arr[] = { 20,40,50,10,30 };

        System.out.println(Arrays.toString(arr));
        cycleSort(arr);
        System.out.println(Arrays.toString(arr));

    }
}
```

14.12. Exploring Heap Sort

- ◆ Basic Idea of Heap Sort is based on Selection Sort
- ◆ Heap Sort is an Improvement of Selection Sort

In Selection Sort

- a) We find the Max Element
- b) We swap Max Element with Last element

- ◆ In Selection Sort, we do Linear Search to find the max element.
- ◆ That why Time Complexity with Selection Sort is $O(n^2)$

In Heap Sort

- a) We build Max Heap
- b) We swap Root with Last element - So the Max Element goes to End.
- c) Reduce the Heap Size
- d) Heapify the Array
- e) Repeat Steps b,c,d

- ◆ Heap Sort does the Optimization over the selection sort. Uses the same Idea but for finding the Max Element, Uses Max Heap Data Structures instead of Linear Search
- ◆ That why Time Complexity with Heap Sort is $O(n \log n)$

Complexity of Heap Sort:

Time Complexity - Best	$O(n \log n)$
Time Complexity - Average	$O(n \log n)$
Time Complexity - Worst	$O(n \log n)$
Space Complexity	$O(1)$

14.12.1. Heap Sort Implementation

Lab24.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab24 {
    public static void heapify(int arr[], int n, int i) {

        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            // Recursively heapify the affected sub-tree
            heapify(arr, n, largest);
        }
    }
}
```



```
public static void buildHeap(int arr[]) {  
  
    int n = arr.length;  
    for (int i = n / 2 - 1; i >= 0; i--) {  
        heapify(arr, n, i);  
    }  
}  
  
public static void heapSort(int arr[]) {  
  
    int n = arr.length;  
    buildHeap(arr);  
    for (int i = n - 1; i >= 0; i--) {  
        int temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
  
        heapify(arr, i, 0);  
    }  
}  
  
public static void main(String[] args) {  
  
    int arr[] = { 10, 15, 50, 5, 20, 60, 2 };  
  
    System.out.println(Arrays.toString(arr));  
    heapSort(arr);  
    System.out.println(Arrays.toString(arr));  
}  
}
```

14.13. Exploring Counting Sort

- ♦ Counting Sort is not a Comparison based Algorithm
- ♦ Counting Sort counts the occurrences of elements instead of comparing the elements.
- ♦ Counting Sort is stable sorting Algorithm
- ♦ Counting Sort works well when you have range of Integer Values
- ♦ Counting sort assumes that each of the elements is an integer in the range 1 to K, for some integer K.

Limitations or When I can not use Counting Sort:

- ♦ When Array contains Negative Values
- ♦ When Array contains Floating point Values
- ♦ When Array contains Objects like Students, Customers

Complexity of Counting Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Average	$O(n)$
Time Complexity - Worst	$O(n)$
Space Complexity	$O(k)$

14.13.1. Counting Sort Implementation

Lab25.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
public class Lab25 {  
    public static void countingSort(int arr[],int k) {  
  
        int n = arr.length;  
        int count[] = new int[k];  
  
        for (int i = 0; i < n; i++) {  
            count[arr[i]]++;  
        }  
  
        for (int i = 1; i < k; i++) {  
            count[i] = count[i] + count[i-1];  
        }  
  
        int output[] = new int[n];  
        for (int i = n-1; i >= 0; i--) {  
            output[count[arr[i]]-1] = arr[i];  
            count[arr[i]]--;  
        }  
  
        for (int i = 0; i < n; i++) {  
            arr[i] = output[i];  
        }  
    }  
  
    public static void main(String[] args) {  
  
        int arr[] = { 2,5,5,2,0,1,2};  
  
        System.out.println(Arrays.toString(arr));  
        countingSort(arr,6);  
        System.out.println(Arrays.toString(arr));  
  
    }  
}
```

14.14. Exploring Radix Sort

- ♦ Use Radix Sort when you have large Range Values
- ♦ Radix Sort is not a Comparison based Algorithm,
- ♦ It counts the occurrences of elements instead of comparing the elements
- ♦ Radix Sort is stable sorting Algorithm
- ♦ Radix Sort uses Counting Sort as Sub-Function

Algorithm:

1. Take the least significant digit of each element.
2. Sort the list of elements based on that digit, but keep the order of elements with the same digit.
3. Repeat the sort with each more significant digit.

Complexity of Cycle Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Average	$O(n)$
Time Complexity - Worst	$O(n)$
Space Complexity	$O(d)$

14.14.1. Radix Sort Implementation

Lab26.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab26 {
    public static void radixSort(int arr[]) {

        int n = arr.length;
        int max= arr[0];
```



```
        for(int i=1;i<n;i++) {
            if(arr[i]>max) {
                max= arr[i];
            }
        }
        for(int exp =1;max/exp>0;exp = exp*10) {
            countingSort(arr,exp);
        }
    }
    public static void countingSort(int arr[],int exp) {

        int n = arr.length;
        int count[] = new int[10];

        for (int i = 0; i < n; i++) {
            count[(arr[i]/exp)%10]++;
        }

        for (int i = 1; i < 10; i++) {
            count[i] = count[i] + count[i-1];
        }

        int output[]= new int[n];

        for (int i = n-1; i>=0; i--) {
            output[count[(arr[i]/exp)%10]-1] = arr[i];
            count[(arr[i]/exp)%10]--;
        }

        for (int i = 0; i < n; i++) {
            arr[i] = output[i];
        }
    }
}
```



```
public static void main(String[] args) {  
  
    int arr[] = { 319,212,6,8,100,50};  
  
    System.out.println(Arrays.toString(arr));  
    radixSort(arr);  
    System.out.println(Arrays.toString(arr));  
  
}  
}
```

14.15. Exploring Bucket Sort

- ◆ When Values are uniformly distributed then use Bucket Sort.
- ◆ Like Counting sort, Bucket sort also imposes restrictions on the input to improve the performance.
- ◆ In other words, Bucket sort works well if the input is drawn from fixed set.

Ex:

Assume that all the input elements from $\{0, 1, \dots, K - 1\}$,
i.e., the set of integers in the interval $[0, K - 1]$.

That means, K is the number of distant elements in the input.

Complexity of Bucket Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Average	$O(n)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(n)$



14.15.1. Bucket Sort Implementation

Lab27.java

```
package com.jlcindia.sorting;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab27 {
    public static void bucketSort(int arr[], int k) {
        int n = arr.length;

        //1. Find max Value
        int max_value=arr[0];
        for(int i=1;i<n;i++) {
            max_value=Math.max(max_value, arr[i]);
        }
        max_value++;

        //2. Create Buckets
        ArrayList<ArrayList<Integer>> mybuckets =
            new ArrayList<ArrayList<Integer>>();

        for(int i=0;i<k;i++) {
            mybuckets.add(new ArrayList<Integer>());
        }

        //3. Fill Buckets
        for(int i=0;i<n;i++) {
            int bucketIndex=(k*arr[i])/max_value;
            mybuckets.get(bucketIndex).add(arr[i]);
        }
    }
}
```



//4. Sort Buckets

```
for(int i=0;i<k;i++) {  
Collections.sort(mybuckets.get(i));  
}
```

//5. Join Buckets

```
int myindex=0;  
for(int i=0;i<k;i++) {  
    for(int j=0;j<mybuckets.get(i).size();j++) {  
        arr[myindex++]=mybuckets.get(i).get(j);  
    }  
}  
  
public static void main(String[] args) {  
    int arr[]={30,40,10,80,5,12,70,45};  
  
    System.out.println(Arrays.toString(arr));  
    bucketSort(arr, 4);  
    System.out.println(Arrays.toString(arr));  
}  
}
```